Fundamentele programării 2.

Sorin Milutinovici
sorin.milutinovici@prof.utm.ro

Input/Output

Operatori

Instrucțiuni

Funcții

Tablouri de date

Input / Output

input/output

printf	int printf(const char *fmt, ...)	funcțiile de acest tip se numesc “variadice”

Declarația funcției ne arată:

1. Există un argument obligatoriu de tip char * (pointer către char adică un șir de caractere)
2. Acesta poate fi urmat de unul sau mai multe argumente.

Argumentul obligatoriu este un **șir de caractere** numit “format”. Șirul respectiv poate avea o serie de coduri speciale pentru afișarea variabilelor care urmează ca argumente suplimentare.

```
ex: printf(“număr întreg: %i si număr flotant %f”, 23, 23.4567);
```

scanf	int scanf(const char *fmt, ...)	funcțiile de acest tip se numesc “variadice”

Declarația funcției ne arată:

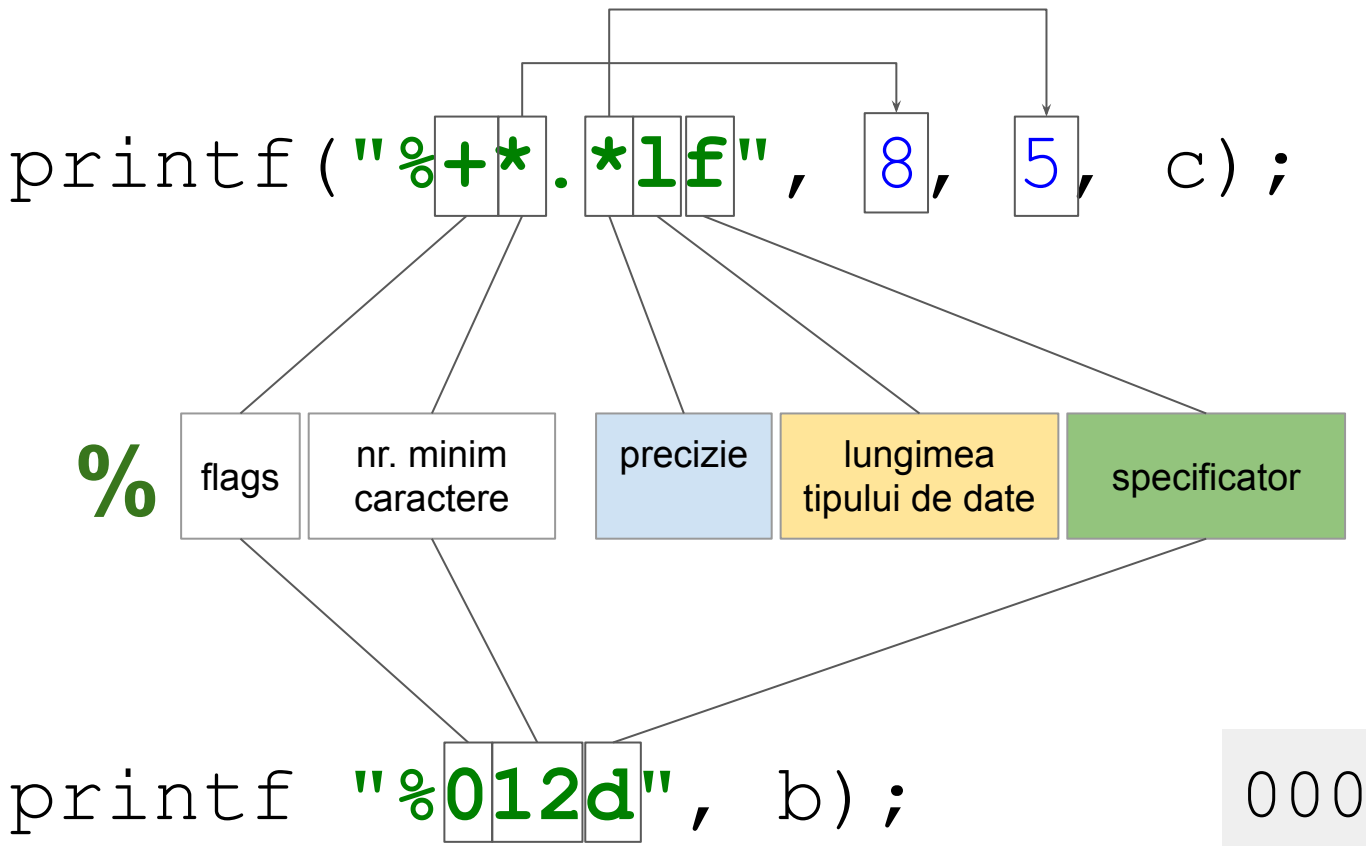
1. Există un argument obligatoriu de tip char * (pointer către char adică un șir de caractere)
2. Acesta poate fi urmat de unul sau mai multe argumente.

```
ex: scanf(“%i”, &intvar);
```

Pentru a putea plasa valoarea obținută de la consolă într-o variabilă definită în afara funcției scanf, trebuie să trimitem o **referință** către o variabilă **deja existentă**.

printf

```
long double c = 342.43543453;
int b = 12345;
```



printf - exemple pentru floats

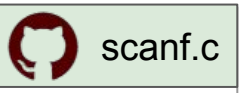
printf_floats.c

```c
#include <stdio.h>

int main()
{
   float c = 342.43543453;
   //Utilizand doar specificatorul %f
   printf("%f\n", c);
   //Specificatorul de float cu semnul fortat
   printf("%+f\n", c);
   //Specificatorul de float cu minus sau spatiu
   printf("% f\n", c);
   //Specificatorul de float cu numar si precizie
   printf("%5.2f\n", c);
   //Specificatorul de float cu partea intreaga si precizie
   printf("%7.2f\n", c);
   //Specificatorul de float cu partea intreaga si precizie ca parametrii
   printf("%*.*f\n", 8, 5, c);
   return 0;
}
```

https://onlinegdb.com/rk2WMPcn7

```
342.435425
+342.435425
 342.435425
342.44
 342.44
342.43542
```

scanf




```c
#include <stdio.h>

int main()
{
   int s;
   float b;
   char c;
   printf("Introduceti un numar intreg:");
   scanf ("%d", &s);
   printf ("Ati introdus numarul %d\n",s);
   printf("Introduceti un numar rational:");
   scanf ("%f", &b);
   printf ("Ati introdus numarul %f\n",b);
   printf("Introduceti un caracter:");
   scanf (" %c", &c);
   printf ("Ati introdus caracterul %c\n",c);
   return 0;
}
```

https://onlinegdb.com/ry_y_P92X

```
Introduceti un numar intreg:12
Ati introdus numarul 12
Introduceti un numar rational:32.45
Ati introdus numarul 32.450001
Introduceti un caracter:d
Ati introdus caracterul d
```

Operatorul & plasat înaintea numelui variabilei specifică o **referință** către acea variabilă.

Referința conține **adresa segmentului de memorie** în care este stocată variabila.

Astfel, scanf poate prelua datele introduse de la consolă și le poate **plasa direct în memorie** acolo unde restul programului știe că va găsi variabilele noastre.

Dacă am trimite variabila, în locul referinței, aceasta ar fi **copiată în altă adresă de memorie** la intrarea în funcție, adresă care ar fi **ștearsă** la părăsirea funcției iar informația s-ar pierde.

Funcțiile sunt tipizate în C, de aceea nu putem returna din scanf datele pentru că nu știm ce fel de date ne trebuie.

~~`float cc = scanf(...)`~~

ar însemna să avem un scanf pentru char, unul pentru int, unul pentru float… etc.

Operatori

Operatori aritmetici

```
int a = 10;
int b = 26;
```

Observați problemele de precizie!!

Op	Descriere	int a = 10; int b = 26;	float a = 10.12; float b = 26.32;	double a = 10.12; double b = 26.32;
+	Adaugă două variabile	a + b = 36	36.439999	36.439999999999998
-	Scade variabila a doua din prima	b - a = 16	16.200001	16.200000000000003
*	Înmulțește două variabile	a * b = 260	266.358398	266.358399999999961
/	Împarte două variabile	b / a = 2	2.600791	2.600790513833992
%	Restul împărțirii pentru numere întregi	b % a = 6	Nu e cazul	Nu e cazul
++	Incrementează operatorul cu 1	a++ = 11	11.120000	11.119999999999999
--	Decrementează operatorul cu 1	a-- = 9	9.120000	9.119999999999999

Operatori logici

Adevărat sau fals în C.

- Orice valoare care este diferită de 0 este considerată adevărată
- Valoarea 0 este considerată falsă
- Rezultatul operatorilor logici poate fi 0 (fals) sau 1 (adevărat).

Operator	Descriere	`int a = 1;` `int b = 0;`
&&	Operatorul AND. Daca ambele variabile sunt non-zero atunci rezultatul devine “adevărat” (1)	a && b este: 0
||	Operatorul OR. Dacă oricare dintre cele doua variabile este diferită de zero atunci rezultatul devine “adevărat” (1)	a || b este: 1
!	Operatorul NOT. Inversează condiția pe care o precede.	! a || b este: 0

Regulile se păstrează și pentru alt fel de tipuri de date - float, double.

Operatori relaționali

Rezultatul operatorilor relaționali poate fi 0 (fals) sau 1 (adevărat).

Operator	Descriere	**int** a = 10; **int** b = 26;
==	Verifică dacă cele două valori sunt egale sau nu. Dacă da, condiția devine 1 (adevărată). Dacă nu, condiția devine 0 (falsă)	a == b este: 0
!=	Verifică dacă cele două valori sunt diferite. Dacă da, condiția devine 1 (adevărată). Dacă nu, condiția devine 0 (falsă)	a != b este: 1
>	Verifică dacă valoarea variabilei din stânga este mai mare decât cea a celei din dreapta. Dacă da, condiția devine 1 (adevărată). Dacă nu, condiția devine 0 (falsă)	a > b este: 0
<	Verifică dacă valoarea variabilei din stânga este mai mică decât cea a celei din dreapta. Dacă da, condiția devine 1 (adevărată). Dacă nu, condiția devine 0 (falsă)	a < b este: 1
>=	Verifică dacă valoarea variabilei din stânga este mai mare SAU EGALĂ decât cea a celei din dreapta. Dacă da, condiția devine 1 (adevărată). Dacă nu, condiția devine 0 (falsă)	a >= b este: 0
<=	Verifică dacă valoarea variabilei din stânga este mai mică SAU EGALA decât cea a celei din dreapta. Dacă da, condiția devine 1 (adevărată). Dacă nu, condiția devine 0 (falsă)	a <= b este: 1

Sau când 0,7 nu e egal cu 0.7 ...

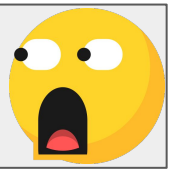
	semn		exponent		mantisa		total
float	0	1b	01111110	8b	01100110011001100110011	23b	32 biți
double	0	1b	01111111110	11b	0110011001100110011001100110011001100110011001100110	52b	64 biți

float	0.69999998807907104492187 5
double	0.69999999999999999555910790149 9

Valorile stocate în realitate

```
#include <stdio.h>

void main()
{
    float a = 0.7;
    if (a == 0.7) {
        printf("egale");
    } else {

printf("inegale");
    }
}
```

inegale



definim variabila a de tip float și încercăm să stocăm 0.7 în ea. În realitate stocăm 0.69999998807907104492187 5

0.7 în condiția if devine o **variabilă anonimă**. Ce tip are? float, double? Programatorul nu a asociat un tip dar 0.7 din if trebuie să fie stocat în memorie și pentru aceasta compilatorul **trebuie să decidă un tip** doar pe baza evaluării numărului 0.7. Compilatorul vede că nu este număr întreg și alege **cel mai mare tip flotant comun** (double).

deci a == 0.7 compară valoarea lui 0.7 float (stocată în variabila a) cu valoarea lui 0.7 double, stocată în variabila anonimă.

a **este automat promovat la double** (dar nu 0.7 ci 0.69999998807907104492187 5) și comparat cu 0.7 stocat în double (0.69999999999999999555910790149 9). Cele două valori NU sunt egale.

Dacă scriam `a == 0.7f` (ca în slide-ul anterior) atunci îi comunicam compilatorului că 0.7 este float și atunci nu ar fi fost stocat automat ca double. Egalitatea ar fi fost adevărată.

Operatori la nivel de bit

Operatorii la nivel de bit operează direct cu fiecare bit al variabilelor.

Operator	Descriere
&	AND binar - copiază un bit în rezultat dacă există în ambele variabile
|	OR binar - copiază un bit dacă există în oricare dintre variabile
^	Exclusive OR (XOR)
~	NOT (Inversează fiecare bit)
<<	Left Shift - mută patternul de biți cu n caractere la stânga
>>	Right Shift - mută patternul de biți cu n caractere la dreapta

Operatori la nivel de bit

1	1	0	1	0	0	1	1	211
& (AND)								
1	0	1	1	0	1	0	1	181
=								
1	0	0	1	0	0	0	1	145

1	1	0	1	0	0	1	1	211
| (OR)								
1	0	1	1	0	1	0	1	181
=								
1	1	1	1	0	1	1	1	247

1	1	0	1	0	0	1	1	211
^ (XOR)								
1	0	1	1	0	1	0	1	181
=								
0	1	1	0	0	1	1	0	102

~ (NOT)								
1	1	0	1	0	0	1	1	211
=								
0	0	1	0	1	1	0	0	44

1	1	0	1	0	0	1	1	211
>>3 (R Shift)								
=								
0	0	0	1	1	0	1	0	26

1	1	0	1	0	0	1	1	211
<<3 (L Shift)								
=								
1	0	0	1	1	0	0	0	152

Reprezentarea numerelor negative

Două variante:

1. "sign-magnitude" - cel mai din stânga bit este folosit ca semn
2. numerele negative sunt reprezentate ca **"complemente fata de 2"** ale numerelor pozitive

Presupunand ca avem un set de numere reprezentate printr-un numar fix de cifre si ca acestea sunt aliniate pe un interval linear, complementul unui număr este numărul care se afla la egală distanță de celălalt capăt al intervalului

Exemplu: Complementul față de 10

Complementul lui 2 fata de 10 este 8

0	1	2	3	4	5	6	7	8	9	10

Cum calculam complementul față de 10 a lui 2? Scadem 2 din 10
Cum calculam complementul numarului 23145 fata de 10? Scadem 23145 din 100000 si obtinem 76855. Putem calcula manual? Dificil. Exista o metoda mai simpla? Da, calculam complementul fata de 9 și adaugam 1

	9	9	9	9	9	
Numar	2	1	3	4	5	
Complementul fata de 9	7	8	6	5	4	+ 1
Complementul fata de 10	7	8	6	5	5	

Reprezentarea numerelor negative

Echivalent binar - cum calculam complementul fata de 2 al numărului 01010101?

Cel mai simplu: calculăm complementul față de 1 și adăugăm 1

Numar	0	1	0	1	0	1	0	1	
Complementul fata de 1	1	0	1	0	1	0	1	0	+1
Complementul fata de 2	1	0	1	0	1	0	1	1	

Conversie - 34 -> 00100010

Convertim pozitivul lui 34 la un număr binar	00100010
Inversam toti bitii	11011101
adaugam 1 la valoarea obtinuta anterior	11011110

Conversie 10001111 -> -113

Incepem cu reprezentarea binara a numarului	10001111
inversam toti bitii acesteia, obtinand complementul fata de 1	01110000
Adaugam 1 la complement obtinand complementul fata de 2	01110001
Convertim rezultatul la un numar zecimal pozitiv	113
Adaugam semnul -	-113

Dacă nu știm că numărul are semn:

$1\times128 + 0\times64 + 0\times32 + 0\times16 + 1\times8 + 1\times4 + 1\times2 + 1\times1 = 143$

De ce se folosește complementul?

1. Se pot folosi aceleași circuite de tip added pentru scădere.

10 - 4	
10	0000 1010
4	0000 0100
~4	1111 1011
$comp_2(4)$	1111 1011 + 0000 0001 = 1111 1100
10 + $comp_2(4)$	0000 1010 + 1111 1100 — 1 0000 0110
6	0000 0110

10 - 14	
10	0000 1010
14	0000 1110
~14	1111 0001
$comp_2(14)$	1111 0001 + 0000 0001 = 1111 0010
10 + $comp_2(14)$	0000 1010 + 1111 0010 — 1 1111 1100
252 (unsigned)	1111 1100
~252	0000 0011
~-252 + 1 4	0000 0011 + 0000 0001 — 0000 0100
-4	adaugam semnul minus

2. Un singur 0 (sign-magnitude are +0 (00000000) și -0 (10000000)

3. Mai simplu de detectat depășirile aritmetice (cel mai din stânga bit poate avea o schimbare falsă de semn datorită adunării numerelor mari).

4. Mai simplu de implementat operațiile de bit-shift

Numerele întregi trebuie marcate ca având semn sau nu

1 1 0 1 1 0 0 1

217 + 39 = 256

Numerele binare reprezentate in 8 biti, convertite in baza 10 se intind în intervalul 0 - 255. Complementaritatea se păstrează

Unsigned = 217

	1	1	0	1	1	0	0	1
inmultire cu puteri ale lui 2	$1 * 2^7$	$1 * 2^6$	$0 * 2^5$	$1 * 2^4$	$1 * 2^3$	$0 * 2^2$	$0 * 2^1$	$1 * 2^0$
	1 * 128	1 * 64	0 * 32	1 * 16	1 * 8	0 * 4	0 * 2	1 * 1
217	128	64	0	16	8	0	0	1

Signed = -39

	1	1	0	1	1	0	0	1	
complementul fata de 1	0	0	1	0	0	1	1	0	+
complementul fata de 2	0	0	1	0	0	1	1	1	
-39	0 * 128	0 * 64	1 * 32	0 * 16	0 * 8	1 * 4	1 * 2	1 * 1	

Operatori de atribuire

Op	Descriere	Exemplu
=	Atribuie valoarea din partea dreapta a operatorului variabilei din partea stângă	a = b + c
+=	Adaugă valoarea variabilei din dreapta celei din stânga și atribuie valoarea variabilei din stânga.	c += a este la fel cu c = c + a
-=	Scade valoarea variabilei din partea dreaptă din cea din stânga și atribuie rezultatul variabilei din partea stângă.	c -= a este la fel cu c = c - a
*=	Înmulțește valoarea variabilei din stânga operatorului cu valoarea celei din stânga și atribuie rezultatul variabilei din stânga.	c *= a este la fel cu c = c * a
/=	Împarte valoarea variabilei din stânga la cea din dreapta și atribuie rezultatul variabilei din stânga.	c /= a este la fel cu c = c / a
%=	Atribuie variabilei din stânga valoarea modulului împărțirii valorii din stânga la cea din dreapta	c %= a este la fel cu c = c % a
<<=	Left shift si atribuire	c <<= 2 este la fel cu c = c<<2
>>=	Right shift și atribuire	c <<= 2 este la fel cu c = c<<2
&=	Bitwise AND și atribuire	c &= a este la fel cu c = c & a
^=	Bitwise XOR și atribuire	c ^= a este la fel cu c = c ^ a
|=	Bitwise OR și atribuire	c |= a este la fel cu c = c | a

Postincrement, preincrement

https://onlinegdb.com/-kKLboS32

```c
#include <stdio.h>

int main() {
   int a = 5;
   printf("++A: %d\n", ++a);
   printf("noul a : %d\n", a);
   return 0;
}
```

```
++A: 6
noul a : 6
```

https://onlinegdb.com/EncuBa1GB

```c
#include <stdio.h>

int main() {
   int a = 5;
   printf("A++: %d\n", a++);
   printf("noul a : %d\n", a);
   return 0;
}
```

```
++A: 5
noul a : 6
```

https://onlinegdb.com/e6yHkIkEK

```c
#include <stdio.h>

int main() {
   int a = 5;
   int result = a++ % 3;
   printf("Rezultat: %d\n", result);
   printf("Noul a: %d\n", a);
   return 0;
}
```

```
Rezultat: 2
Noul a: 6
```

https://onlinegdb.com/ja-6hUOCGT

```c
#include <stdio.h>

int main() {
   int a = 5;
   int result = ++a % 3;
   printf("Rezultat: %d\n", result);
   printf("Noul a: %d\n", a);
   return 0;
}
```

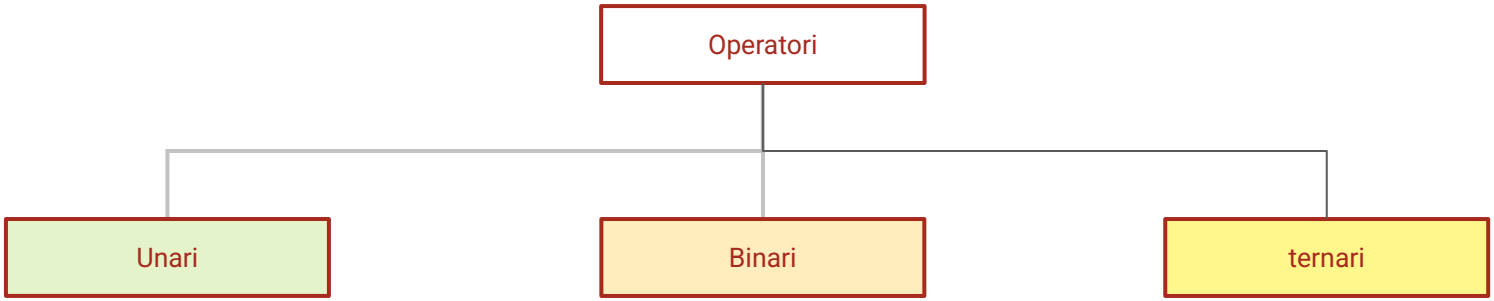
```
Rezultat: 0
Noul a: 6
```

Alți operatori

Operator	Descriere	Exemple
sizeof()	Returnează dimensiunea unei variabile.	sizeof(a), un număr
&	Returnează adresa unei variabile	&a = 0x7fffa9a5a92c
*	Returnează un pointer către o variabilă	*a;
?:	Expresie condițională	a > 23 ? 1 : 0 returnează 1 dacă variabila a conține un număr mai mare decât 23 și 0 dacă nu.

Operatori unari, binari, ternari

- Operatorii unari implică o singură variabilă.
- Operatorii binari implică două variabile.
- Operatorii ternari implică trei variabile.



- sizeof
- incrementare
  - post (a = x++)
  - pre (a = ++X)
- decrementare
  - pre (a = x--)
  - post (a = --x)

- aritmetic (+,-,*,/,%)
- relațional (==,!=,>,<,>=,<=)
- logic (&&,||,!)
- bitwise (&,^,|,~,<<,>>)
- atribuire (+,+=,-=...)
- speciali (&,*)

condițional: (?:)

Precedența operatorilor

Operator	Asociere
!,++,--, - unar	dreapta la stânga
*,/,%	stânga la dreapta
<, <=, > >=	stânga la dreapta
==, !=	stânga la dreapta
&&	stânga la dreapta
||	stânga la dreapta

```c
#include <stdio.h>

int main() {
   int result = 5 + 3 * 2;
   printf("Result: %d\n",
result);
   return 0;
}
```

```c
#include <stdio.h>

int main() {
   int a = 5;
   int result = a++ % 3;
   printf("Result: %d\n", result);
   printf("New value of a: %d\n",
a);
   return 0;
}
```

https://en.cppreference.com/w/c/language/operator_precedence

Instructiuni

Blocuri de cod

C (ca multe alte limbaje) permite definirea unor blocuri de cod care limitează vizibilitatea (eng. scope) variabilelor din interiorul acestora (cele declarate cu clasa de stocare auto). Acest comportament se numeste “lexical scoping”.

```
#include <stdio.h>


int main() {
  int a = 10;
  {                                          bloc 1
    int b = 18;
    printf("b in bloc este %i\n", b);
    {                                        bloc 2
      int c = 23;
      printf("b,c in bloc:  %i,%i\n", b,c);
    }
  }
   int d = 20;
   printf("a este %i\n", a);
   printf("b este %i\n", b);
   printf("d este %i\n", d);
   return 0;
}
```

```
#include <stdio.h>


int main() {
   int a = 10;
   {
     int b = 18;
     printf("b in bloc este %i\n", b);
     {
       int c = 23;
       printf("b,c in bloc:  %i,%i\n", b,c);
     }
   }
   int d = 20;
   printf("a este %i\n", a);
//    printf("b este %i\n", b);
   printf("d este %i\n", d);
   return 0;
}
```

https://onlinegdb.com/SJ2b2KKT7

```
main.c: In function 'main':
main.c:16:26: error: 'b' undeclared (first use
in this function)
    printf("b este %i\n", b);
```

```
b in bloc este 18
b,c in bloc:  18,23
a este 10
d este 20
```

Cum să luăm decizii

Deciziile sunt instrucțiuni care execută anumite blocuri de cod în funcție de răspunsul la o anumită întrebare. Întrebarea este evaluată și poate avea două răspunsuri: adevărat sau fals.

Adevărat sau fals in C.

- Orice valoare care este diferită de 0 este considerată adevărată
- Valoarea 0 este considerată falsă
- Rezultatul operatorilor logici poate fi 0 (fals) sau 1 (adevărat).

```c
int main()
{
   int N;
   printf("introduceti un numar\n");
   scanf("%i",&N);

   if (N > 0) {
       printf("Numarul e mai mare decat zero\n");
   } else {
       printf("Numarul e mai mic decat zero\n");
   }
   return 0;
}
```

```
introduceti un numar
12
Numarul e mai mare decat zero
```

```
introduceti un numar
-432
Numarul e mai mic decat zero
```

https://onlinegdb.com/B1RbH623X

if-else

Cu paranteze

```c
int main()
{
    int N;
    printf("numar ?\n");
    scanf("%i",&N);

    if (N > 0) {
        printf("> decat zero\n");
    } else {
        printf("< decat zero\n");
    }

    return 0;
}
```

https://onlinegdb.com/Sy2fbSw6X

Fără paranteze

if-no-braces.c

```c
int main()
{
  int N;
  printf("numar? \n");
  scanf("%i",&N);

  if (N > 0)
      printf("> decat zero\n");
  else
      printf("< decat zero\n");

   printf("restul ...");
  return 0;
}
```

https://onlinegdb.com/HyH9brDpm

În cazul în care după if sau else urmează o singură instrucțiune (doar una), se pot omite parantezele. Dar aceea trebuie sa fie instrucțiune și nu declarație.

Editoarele special concepute pentru scrierea codului vor pune paranteze și în cazul în care există o singură instrucțiune. Chiar dacă nu sunt necesare, acestea fac codul mult mai ușor de înțeles și urmărit.

Apple goto fail SSL bug

```
. . .
hashOut.data = hashes + SSL_MD5_DIGEST_LEN;
hashOut.length = SSL_SHA1_DIGEST_LEN;
if ((err = SSLFreeBuffer(&hashCtx)) != 0)
   goto fail;
if ((err = ReadyHash(&SSLHashSHA1, &hashCtx)) != 0)
   goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &clientRandom)) != 0)
   goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
   goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
   goto fail;
   goto fail;  /* MISTAKE! THIS LINE SHOULD NOT BE HERE */
if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
   goto fail;

err = sslRawVerify(...);
. . .
```

About the security content of iOS 7.0.6

This document describes the security content of iOS 7.0.6.

iOS 7.0.6

- Data Security

Available for: iPhone 4 and later, iPod touch (5th generation), iPad 2 and later

Impact: An attacker with a privileged network position may capture or modify data in sessions protected by SSL/TLS

Description: Secure Transport failed to validate the authenticity of the connection. This issue was addressed by restoring missing validation steps.

CVE-ID

CVE-2014-1266

Codul va sări peste funcția **sslRawVerify** în toate situațiile, permițând conexiunea SSL **fără verificarea reală a certificatelor**.

În Februarie 2014, Apple a publicat un update de securitate pentru toate dispozitivele mobile, în categoria "Data Security":
Description: Secure Transport failed to validate the authenticity of the connection. This issue was addressed by restoring missing validation steps.

Lipsa parantezelor poate fii "cool" dar poate avea consecințe dramatice. Există standarde de cod care încurajează evitarea parantezelor acolo unde este posibil, există altele care obligă la prezența parantezelor în toate cazurile.

https://www.synopsys.com/blogs/software-security/understanding-apple-goto-fail-vulnerability-2.html

if-else - condiții complexe

Operator	Descriere
&&	Operatorul AND. Daca ambele variabile sunt non-zero atunci rezultatul devine “adevarat” (1)
||	Operatorul OR. Daca oricare dintre cele doua variabile este diferita de zero atunci rezultatul devine “adevarat” (1)
!	Operatorul NOT. Inverseaza condiția pe care o precede.

```
if (a > 3 && a < 8 )
```

```
if ((a > 3 && a < 8 ) || (b > 19))
```

```
if (a > 3 || a < 8 )
```

```
if (!(a > 3 && a < 8 ) || (b > 19))
```

```
if (a == 3 || a > 8 )
```

```
if (!(a > 3 || a < 8 ))
```

Variabile locale

```c
#include <stdio.h>

int main()
{
  int N = 3;
  int a = 2;
  if (N >= 10) {
      a = 100;
      printf("N >= cu 10\n");
  } else if (N < 10 && N >= 0) {
      printf("N <= 10\n");
  } else {
      printf("N negativ\n");
  }
   printf("a dupa if %d\n", a);

  return 0;
}
```

if-imbricat_2.c

https://onlinegdb.com/By4TQSDT7

N <= 10
a dupa if 2

```c
#include <stdio.h>

int main()
{
  int N = 13;
  int a = 2;
  if (N >= 10) {
      a = 100;
      printf("N >= cu 10\n");
  } else if (N < 10 && N >= 0) {
      printf("N <= 10\n");
  } else {
      printf("N negativ\n");
  }
   printf("a dupa if %d\n", a);

  return 0;
}
```

Modificăm N

if-imbricat_2.c

https://onlinegdb.com/rJB4VSv6Q

N >= cu 10
a dupa if 100

Dacă programul ajunge la ramura care atribuie valoarea 100 variabilei a (definită în afara if-ului), variabila își menține valoarea după if.

Variabile locale

```c
#include <stdio.h>

int main()
{
  int N = 13;
  int a = 2;
  if (N >= 10) {
      a = 100;
      int b = 12;
      printf("N >= cu 10\n");
  } else if (N < 10 && N >= 0) {
      printf("N <= 10\n");
  } else {
      printf("N negativ\n");
  }
   printf("a dupa if %d\n", a);
   printf("b dupa if %d\n", b);
  return 0;
}
```

if-imbricat_3.c

https://onlinegdb.com/rJqIrBDam

```
main.c:17:30: error: 'b' undeclared (first use in this function)
    printf("b dupa if %d\n", b);
```

Variabila b nu este definită în afara if-ului

```c
#include <stdio.h>

int main()
{
  int N = 13;
  int a = 2;
  if (N >= 10) {
      a = 100;
      int b = 12;
      printf("N >= cu 10\n");
  } else if (N < 10 && N >= 0) {
      printf("N <= 10\n");
  } else {
      printf("N negativ\n");
  }
   printf("a dupa if %d\n", a);

  return 0;
}
```

if-imbricat_4.c

https://onlinegdb.com/S1e4PBDa7

```
N >= cu 10
a dupa if 100
```

Dacă b nu este apelată în afara blocului în care a fost definită, programul se compilează

ternary-hell.c

```c
#include<stdio.h>

int main()
{
  int a = 23;
   int b = 44;
   int c = 99;
   int max1 = a>b ? a>c?a:c : b>c?b:c;
   int max = 0;
   if (a > b) {
       if (a > c) {
           max = a;
       } else {
           max = c;
       }
   } else {
       if (b > c) {
           max = b;
       } else {
           max = c;
       }
   }
   printf ("Max 1: %i\n", max1);
   printf ("Max 2: %i\n", max);
   return 0;
}
```

https://onlinegdb.com/H13zUtsTX

```c
a>b ? a>c?a:c : b>c?b:c;
```

```
Max 1: 99
Max 2: 99
```

switch - case

switch-case_2.c

```c
int main () {
  char gr = 'B';
  switch(gr) {
     case 'A' :
     case 'B' :
     case 'C' :
        printf("%c: OK\n", gr);
        break;
     case 'D' :
     case 'F' :
        printf("%c: Not OK\n", gr);
        break;
     default :
        printf("%c: Invalid grade\n", gr);
  }
  return 0;
}
```

https://onlinegdb.com/r1VndrPa7

Se pot grupa mai multe cazuri pentru același cod.

De aceea switch-ul trece la următorul caz în lipsa break-ului.

```
A: OK
```

```
B: OK
```

```
C: OK
```

```
D: Not OK
```

```
F: Not OK
```

switch - case față de if - else if

- Orice construcție de tip **switch/case** poate fi scrisă folosind **if/else if**.

- **Nu orice construcție** de tip if/else if poate fi scrisă cu switch/case (pentru că nu putem pune expresii la cazuri în c).

- Switch/case este în general **mai rapid** decât aceeași construcție if/else (deși depinde de compilator și de construcție în sine).

- Uneori switch/case generează cod mai mare decat if/else if()
  `/* A bunch of if...else if... gives smaller code than switch...case ! */` (ARDUINO)

- Construcțiile de tip switch/case sunt **mai clare și mai ușor de citit** decat if/else.

- În general switch/case sunt folosite la listele lungi, cu multe cazuri și cu condiții simple (ex: coduri de eroare diferite care declanșează comportamente diferite).

- If/else if se folosește mai degrabă în situațiile în care sunt cazuri puține cu condiții complexe.

Bucla for

sintaxa instrucțiunii for:

```
for ( init; condition; increment )
{
   statement(s);
}
```

Bucla for are incrementarea inclusă în declarație

```
for( a = 10; a < 15; a = a + 1 )
{
     printf("a = %d\n", a);
}
```

```
int main () {

  int a;

  for( a = 10; a < 15; a = a + 1 ){
     printf("a = %d\n", a);

  }
  return 0;
}
```

```
int main () {

  int a;

  for( a = 10; a < 15; a = a + 1 ){
     printf("a = %d\n", a);
     a++;
  }
  return 0;
}
```

!

```
a = 10
a = 11
a = 12
a = 13
a = 14
```

```
a = 10
a = 12
a = 14
```

Bucla while față de bucla do while

do-while.c

```
int main()
{
    int a = 10;

    while (a < 10) {
        printf("while a = %d\n", a);
        a++;
    }

    a = 10;

    do {
        printf("do a = %d\n", a);
        a++;
    } while (a < 10);

    return 0;
}
```

https://onlinegdb.com/B1co2rv6m

```
do a = 10
```

Buclele while si do while obligă la adăugarea instrucțiunilor de incrementare a contorului

Bucla while se execută doar dacă expresia din paranteză este adevarată. Condiția este testată înainte de execuția blocului din interiorul buclei while.

Bucla do while se execută înainte de evaluarea expresiei din paranteză. De aceea, indiferent de condiția de adevăr a expresiei din paranteza, bucla se execută cel puțin o dată.

Bucla while față de bucla do while

sugerat de dl. Alexandru Palcau

Funcții

Funcții

printf și **scanf** sunt funcții externe (nu fac parte propriu-zis din limbajul C). Sunt implementate în bibliotecile asociate oricărui compilator C, împreună cu alte funcții similare.

Programarea în orice limbaj implică, pe lângă cunoașterea instrucțiunilor limbajului respectiv, cunoașterea funcțiilor disponibile în bibliotecile de sistem precum și a funcțiilor disponibile în alte biblioteci disponibile.

Biblioteca asociată sistemelor Linux (numita glibc) are peste 1600 de funcții.

Utilizarea funcțiilor din bibliotecile externe se face prin folosirea macro-ului **include** care va adăuga la programul curent un fișier special numit header în care se găsesc definițiile funcțiilor respective.

Pentru folosirea funcțiilor printf și scanf a trebuit sa folosim instrucțiunea:

#include <stdio.h> în care stdio.h este headerul.

fragment din fișierul stdio.h

```
/* Write formatted output to STREAM.

  This function is a possible cancellation point
and therefore not
  marked with   THROW.  */
extern int fprintf (FILE *  restrict   stream,
         const char *  restrict   format, ...);
/* Write formatted output to stdout.

  This function is a possible cancellation point
and therefore not
  marked with   THROW.  */
extern int printf (const char *__restrict __format,
...);
/* Write formatted output to S.  */
extern int sprintf (char *  restrict   s,
         const char *__restrict __format, ...)
__THROWNL;
```

Headere ale funcțiilor din glibc.

<assert.h>	<stdatomic.h>
<complex.h>	<stdbool.h>
<ctype.h>	<stddef.h>
<errno.h>	<stdint.h>
<fenv.h>	<stdio.h>
<float.h>	<stdlib.h>
<inttypes.h>	<stdnoreturn.h>
<limits.h>	<string.h>
<locale.h>	<tgmath.h>
<math.h>	<threads.h>
<setjmp.h>	<time.h>
<signal.h>	<uchar.h>
<stdalign.h>	<wchar.h>
<stdarg.h>	<wctype.h>

Headere frecvent utilizate

Acestea sunt disponibile în marea majoritate a compilatoarelor c

complex.h	Funcții pentru aritmetica numerelor complexe
limits.h	Funcții și macro-uri pentru specificarea valorilor minime și maxime ale tipurilor (ex: funcția sizeof())
math.h	Funcții matematice (sqrt, sin, cos)
stdio.h	Funcții legate de input/output
stdarg.h	Funcții care permit procesarea unui număr variabil de argumente trimise unei funcții.
string.h	Funcții care permit procesarea șirurilor de caractere
time.h	Funcții care permit operații cu structuri de tip dată si oră
uchar.h	Funcții care permit operații cu caractere unicode.

math.h

https://en.wikipedia.org/wiki/C_mathematical_functions

În sistemele de operare tip Linux, funcțiile definite în headerul math.h sunt separate într-o altă billiotecă numită libm. În cazul compilării cu compilatorul gcc (Gnu C Compiler), trebuie adăugată biblioteca libm la instrucțiunea de compilare.

```
gcc -Wall sqrt.c -o sqrt -lm
```

sqrt.c

```c
#include <stdio.h>
#include <math.h>

int main()
{
    float x = 4.0, y = 2.0;
    float z1, z2;
    z1 = sqrt(x);
    z2 = sqrt(y);
    printf("%f %f\n", z1, z2);
    return 0;
}
```

https://onlinegdb.com/r1vdABvTX

```
2.000000 1.414214
```

sincos.c

```c
#include <stdio.h>
#include <math.h>

const double pi= 3.14159265;
int main()
{
    double x = pi/2;
    double y = sin(x);
    double z = cos(x);
    printf("sin pi/2 = %f\n", y);
    printf("cos pi/2 = %f\n", z);
    return 0;
}
```

https://onlinegdb.com/BJQAArDT7

```
sin pi/2 = 1.000000
cos pi/2 = 0.000000
```

Cum definim o funcție?

functii_putere.c

```c
#include <stdio.h>

int putere(int m, int n);

int main()
{
   int a=3, b=5;
   int c = putere(a, b);
   printf("%d la puterea %d este %d\n",a,b,c);
   return 0;
}

int putere(int m, int n)
{
   int i, p=1;
   for (i=1; i<=n; i++) {
       p = p * m;
   }
   return p;
}
```

Funcția trebuie declarată înainte de a fi utilizată

Apelarea functiei

Definirea funcției

https://onlinegdb.com/ByTCZ6hTQ

```
3 la puterea 5 este 243
```

Variabile locale

functii_putere.c

```c
#include <stdio.h>

int putere(int m, int n);

int main()
{
   int a=3, b=5;
   int c = putere(a, b);
   printf("%d la puterea %d este %d\n",a,b,c);
   return 0;
}

int putere(int m, int n)
{
   int i, p=1;
   for (i=1; i<=n; i++) {
       p = p * m;
   }
   return p;
}
```

Variabilele a, b și c sunt definite în interiorul funcției “main” și sunt accesibile doar în interiorul funcției “main”

Variabilele i, p sunt definite în interiorul funcției **putere**. Acestea sunt variabile locale. Ele nu sunt accesibile în afara funcției în care au fost declarate

https://onlinegdb.com/ByTCZ6hTQ

```
3 la puterea 5 este 243
```

Variabile globale

functii_putere_vars.c

```c
#include <stdio.h>

int putere(int m, int n);
int glb = 121;

int main()
{
   int a=3, b=5;
   int c = putere(a, b);
   printf("in functia main: %i\n", glb);
   printf("%d la puterea %d este %d\n",a,b,c);
   return 0;
}

int putere(int m, int n)
{
   int i, p=1;
   printf("in functia putere: %i\n", glb);
   for (i=1; i<=n; i++) {
       p = p * m;
   }
   return p;
}
```

https://onlinegdb.com/B14OWUwpm

Variabilă globală

Variabilă globală accesibilă în funcția “main”

Variabilă globală accesibilă în funcția “putere”

```
in functia putere: 121
in functia main: 121
3 la puterea 5 este 243
```

Modificarea variabilelor globale

functii_putere_vars_2.c

```c
#include <stdio.h>

int putere(int m, int n);
int glb = 121;

int main()
{
   int a=3, b=5;
   printf("main: 1 %i\n", glb);
   int c = putere(a, b);
   printf("main: 2 %i\n", glb);
   glb = 1234;
   printf("main: 3 %i\n", glb);
   printf("%d la puterea %d este %d\n",a,b,c);
   return 0;
}

int putere(int m, int n)
{
   int i, p=1;
   glb = 4321;
   printf("putere: 1 %i\n", glb);
   for (i=1; i<=n; i++) {
       p = p * m;
   }
   return p;
}
```

https://onlinegdb.com/SJabWUDpX

```
main: 1 121
putere: 1 4321
main: 2 4321
main: 3 1234
3 la puterea 5 este 243
```

Variabilele globale pot fi modificate în orice funcție a programului. Odată modificate, celelalte apeluri ale aceleiași variabile vor utiliza valoarea modificată.

Funcțiile trebuie declarate înainte de a fi utilizate

functii_putere.c

```c
#include <stdio.h>

int putere(int m, int n);

int main()
{
   int a=3, b=5;
   int c = putere(a, b);
   printf("%d la puterea %d este
%d\n",a,b,c);
   return 0;
}

int putere(int m, int n)
{
   int i, p=1;
   for (i=1; i<=n; i++) {
       p = p * m;
   }
   return p;
}
```

https://onlinegdb.com/H1A97UPTm

global_const.c

```c
#include <stdio.h>

const double PI=3.14159265;

double aria(double raza)
{
   return PI * raza * raza;
}

double lungime(double raza)
{
   return 2 * PI * raza;
}

int main()
{
   double raza = 12;
   printf("Aria cercului: %f\n",
aria(raza));
   printf("Lungimea cercului: %f\n",
lungime(raza));
   return 0;
}
```

https://onlinegdb.com/H1wP7UvpX

Declarația poate fi însoțită de corpul funcției sau nu. În cazul în care corpul funcției este specificat dincolo de funcțiile în care este apelată, ea poate fi declarată la începutul programului.

Universul unui program C

Stack	Spațiul în care sunt stocate variabilele non-statice și apelurile de funcții

memorie distribuită, biblioteci dinamice.

Heap	spațiul disponibil pentru alocare dinamică a memoriei în timpul rulării programului (malloc, calloc)
BSS	Spațiul în care sunt stocate varabilele statice neinițializate (static int x) - care vor fi automat initializate cu 0
Data	Spațiul în care sunt stocate varabilele statice inițializate (static int x = 12)
Text	Spațiul în care se găsește programul (instrucțiunile binare ale acestuia)

Stack-ul crește sau scade la apelul funcțiilor. Fiecare apel de funcție este reprezentat în stack sunt forma unei structuri de date numita “stack frame”. Managementul stack-ului este făcut de sistemul de operare cu ajutorul unor registre ale procesorului.

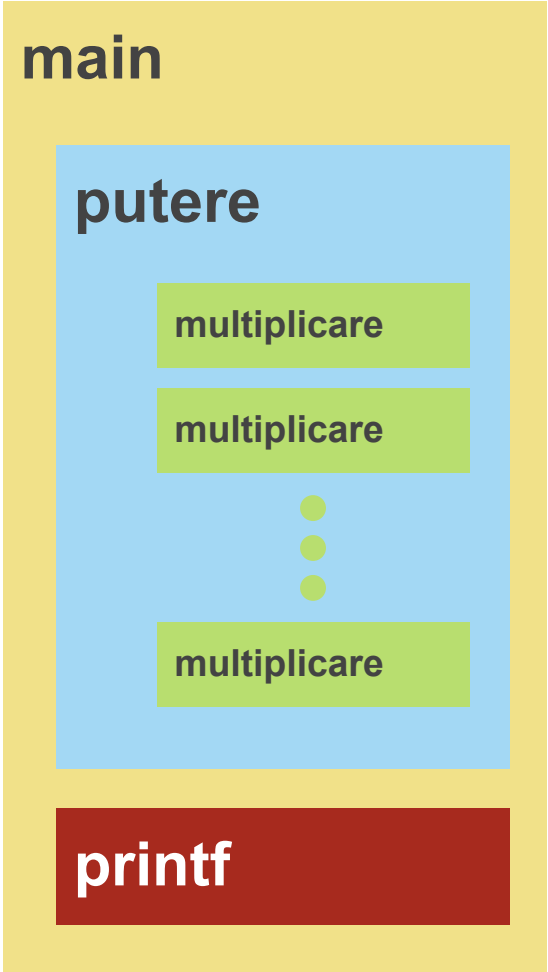
Heap-ul este variabil, crește (sau scade) în timpul execuției programului în funcție de memoria alocată

Data nu este read-only (sau are cel puțin o secțiune modificabila) - variabilele se pot schimba

De obicei segmentul text este read-only dar poate fi distribuit între mai multe procese

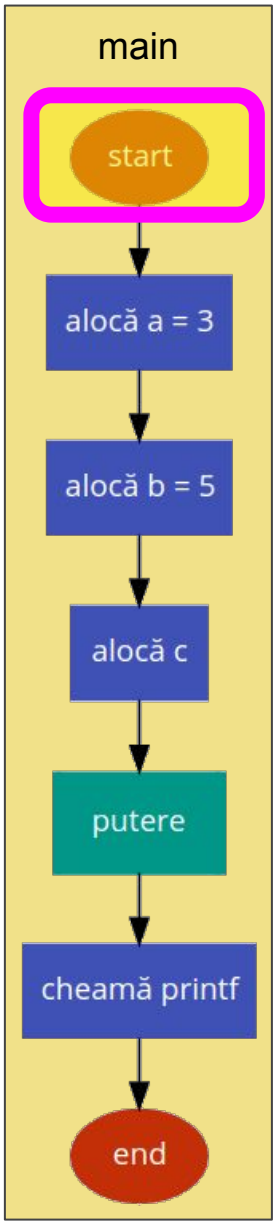
Viața unui program

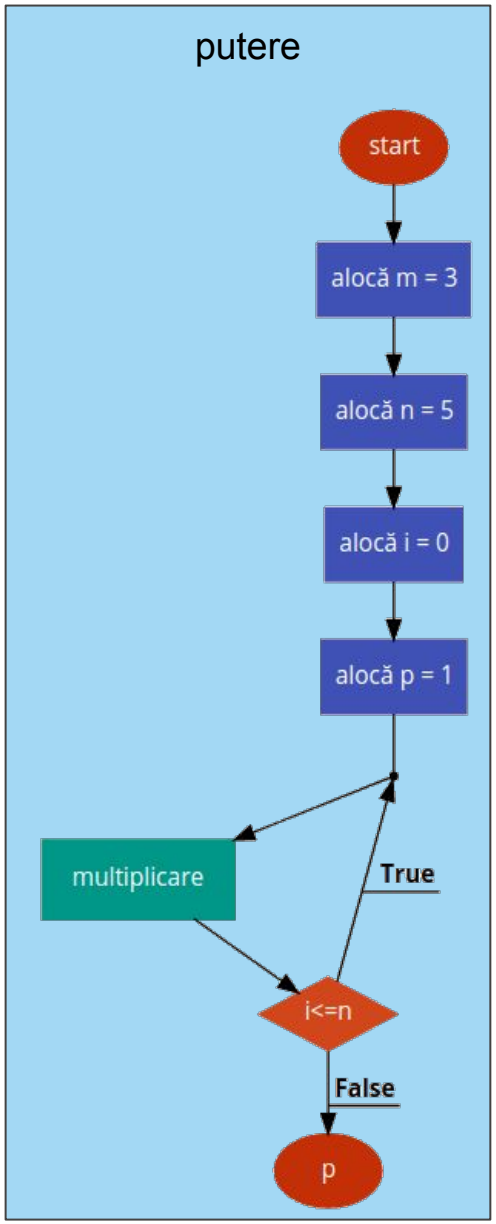
```
#include <stdio.h>

int multiplicare(int m, int n) {
   int result = 0;
   result = m * n;
   return result;
}

int putere(int m, int n)
{
  int i;
  int p=1;
  for (i=1; i<=n; i++) {
      p = multiplicare(p,m);
  }
  return p;
}

int main(int argc, char *argv[])
{
  int a=3;
  int b=5;
  int c = putere(a, b);
  printf("%d la puterea %d este %d\n",a,b,c);
  return 0;
}
```

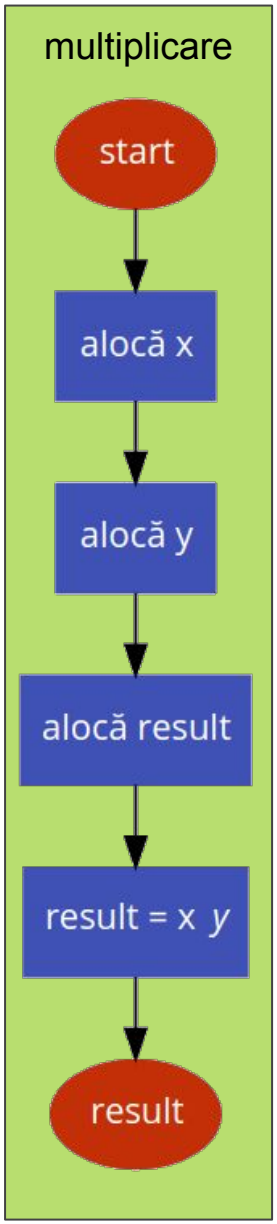



```
3 la puterea 5 este 243
```

Viața unui program

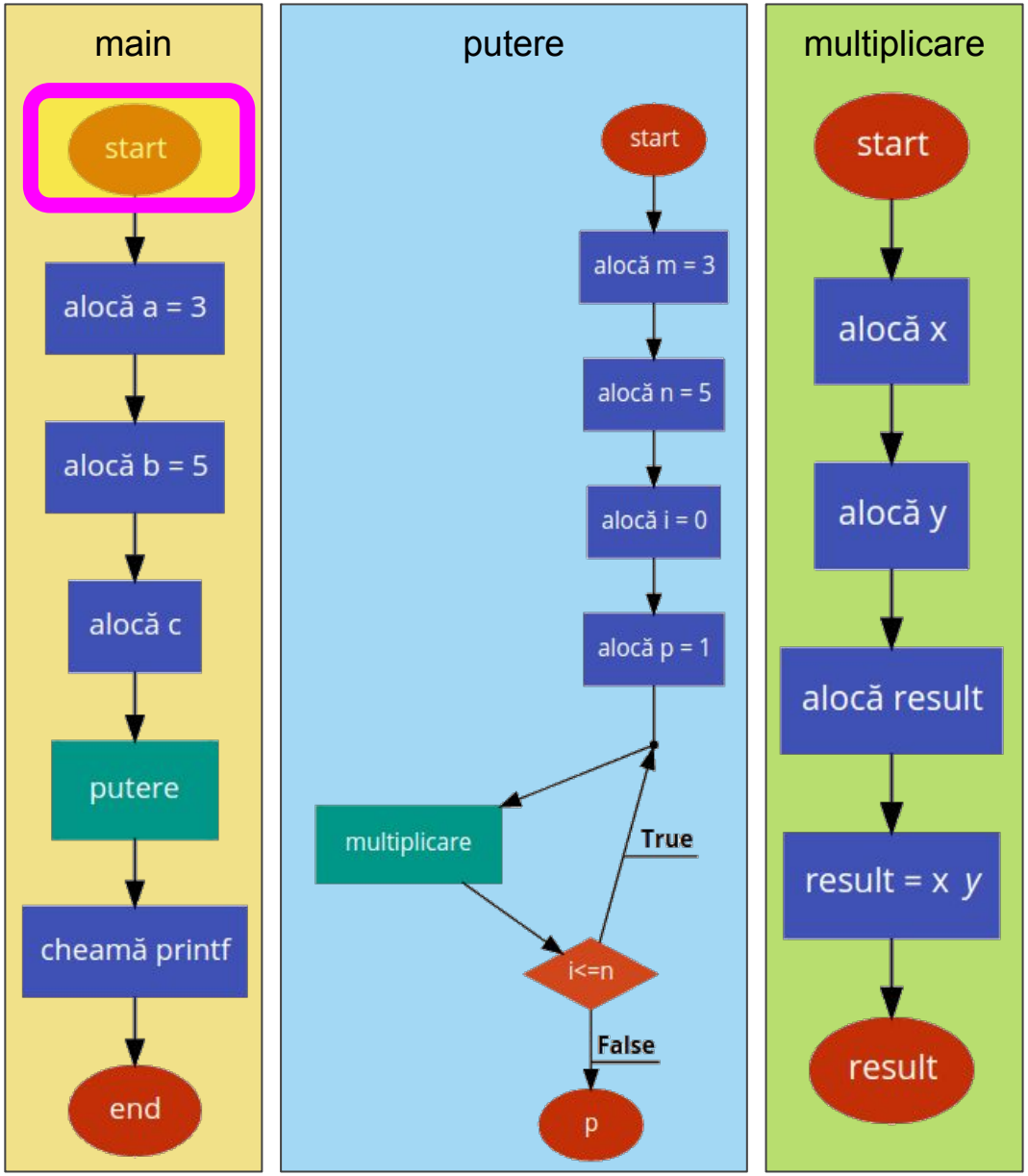


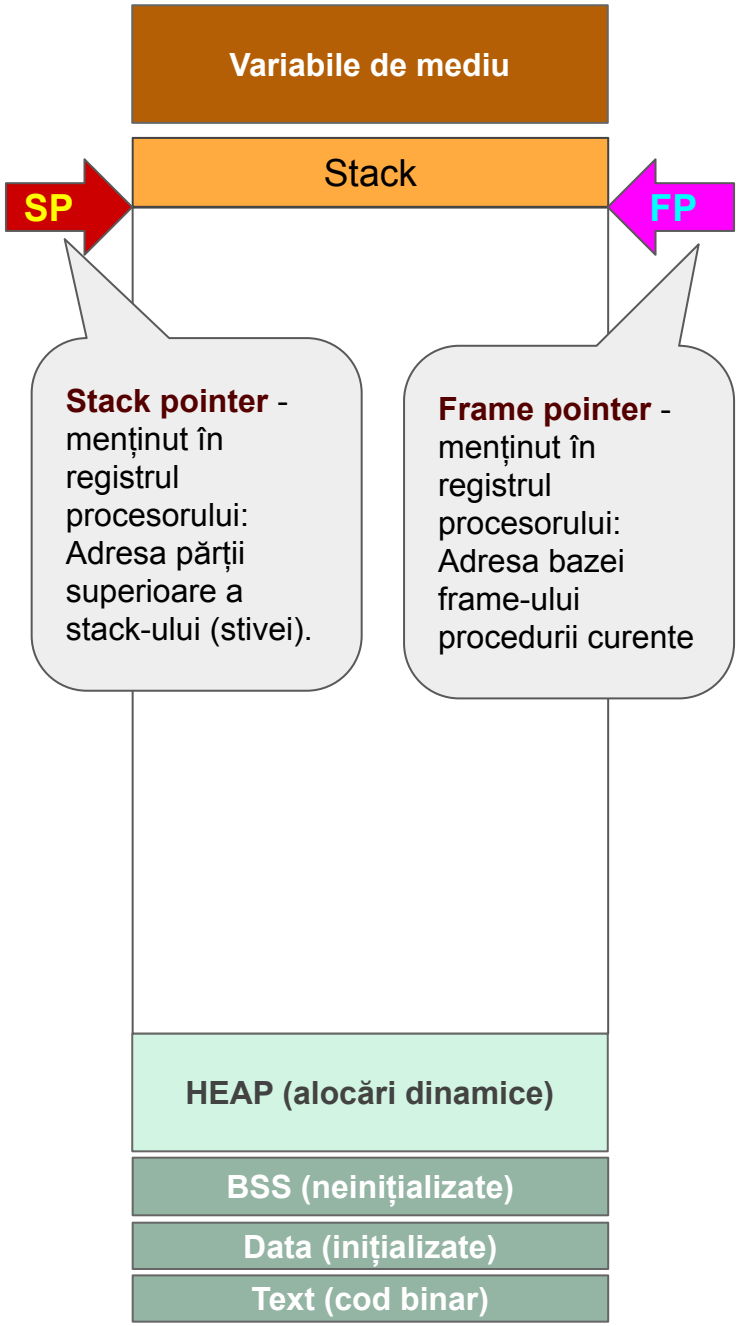




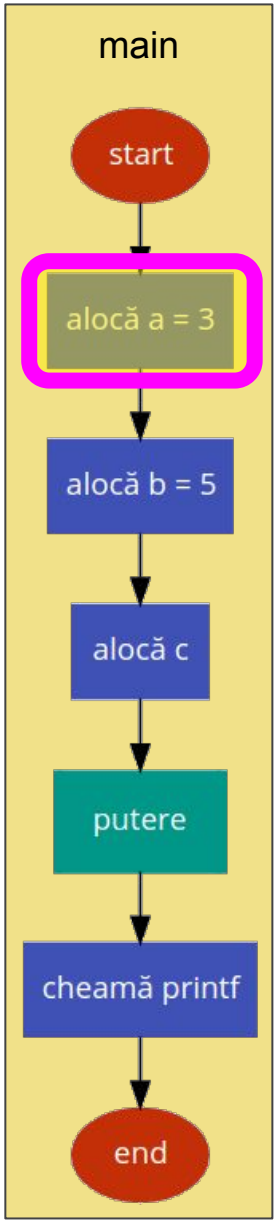

```
#include <stdio.h>

int multiplicare(int m, int n) {
   int result = 0;
   result = m * n;
   return result;
}

int putere(int m, int n)
{
  int i;
  int p=1;
  for (i=1; i<=n; i++) {
      p = multiplicare(p,m);
  }
  return p;
}

int main(int argc, char *argv[])
{
  int a=3;
  int b=5;
  int c = putere(a, b);
  printf("%d la puterea %d este
%d\n",a,b,c);
  return 0;
}
```

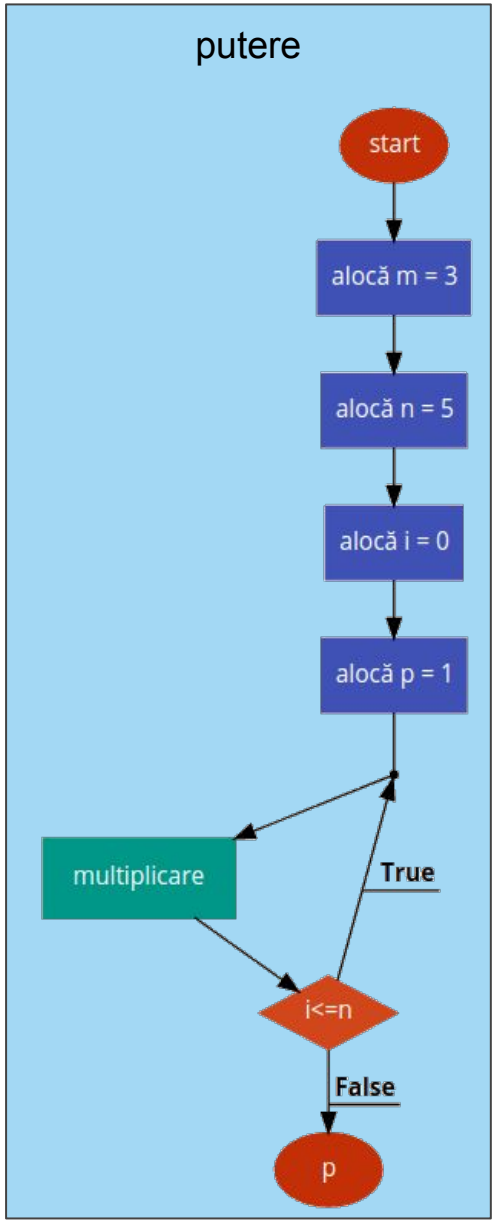
Viața unui program

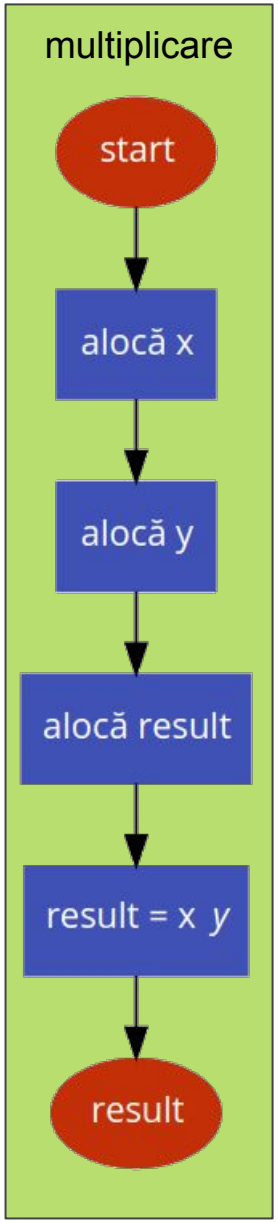


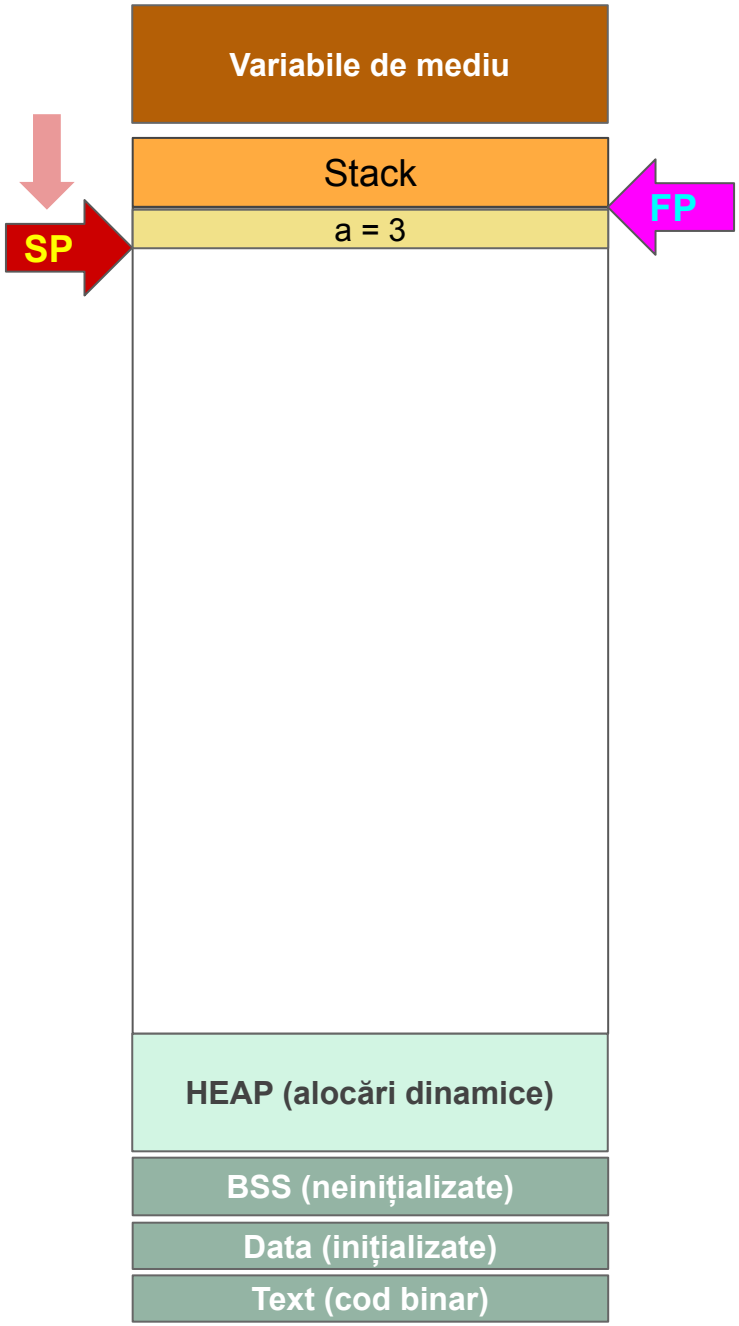


Viața unui program

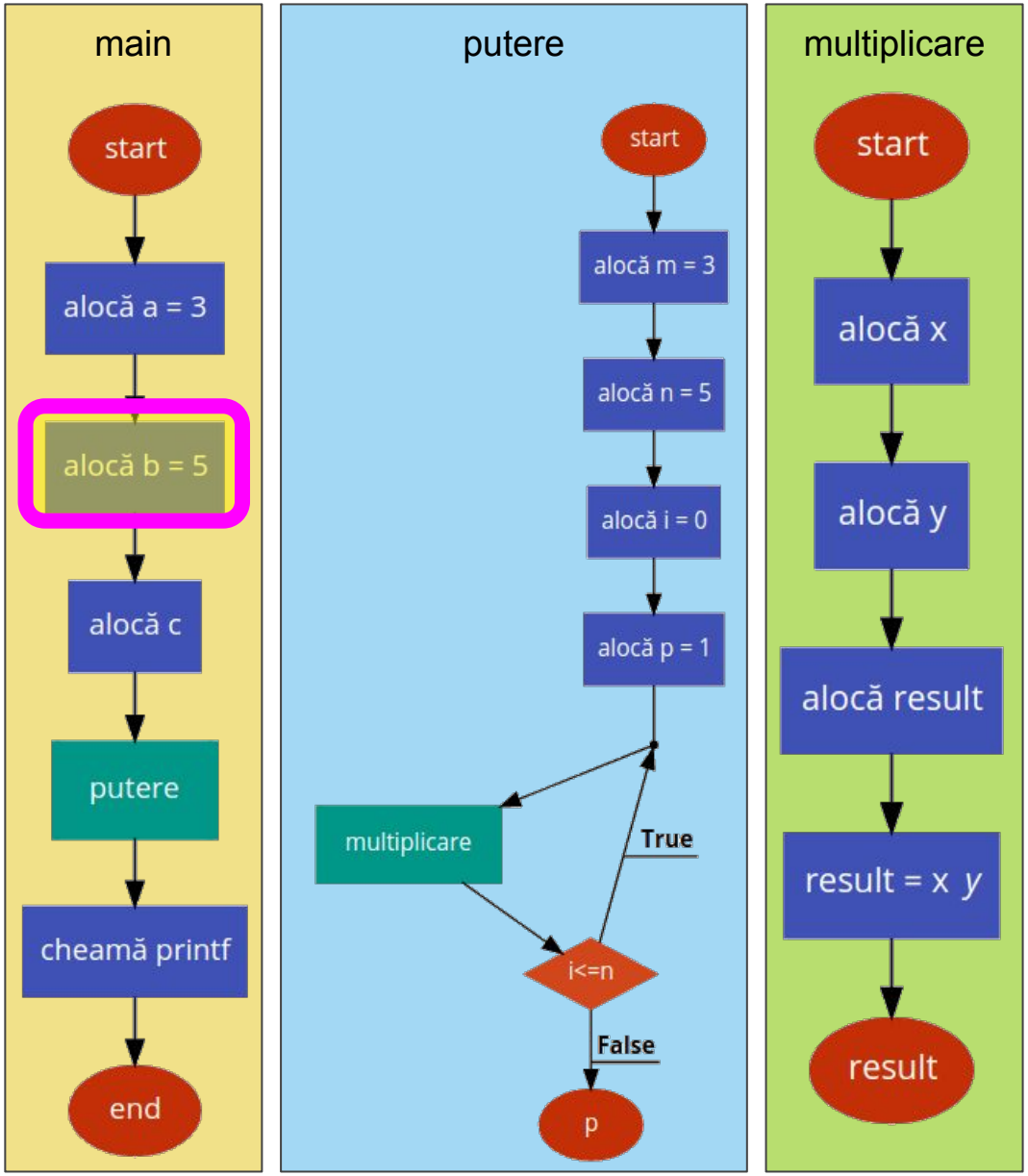


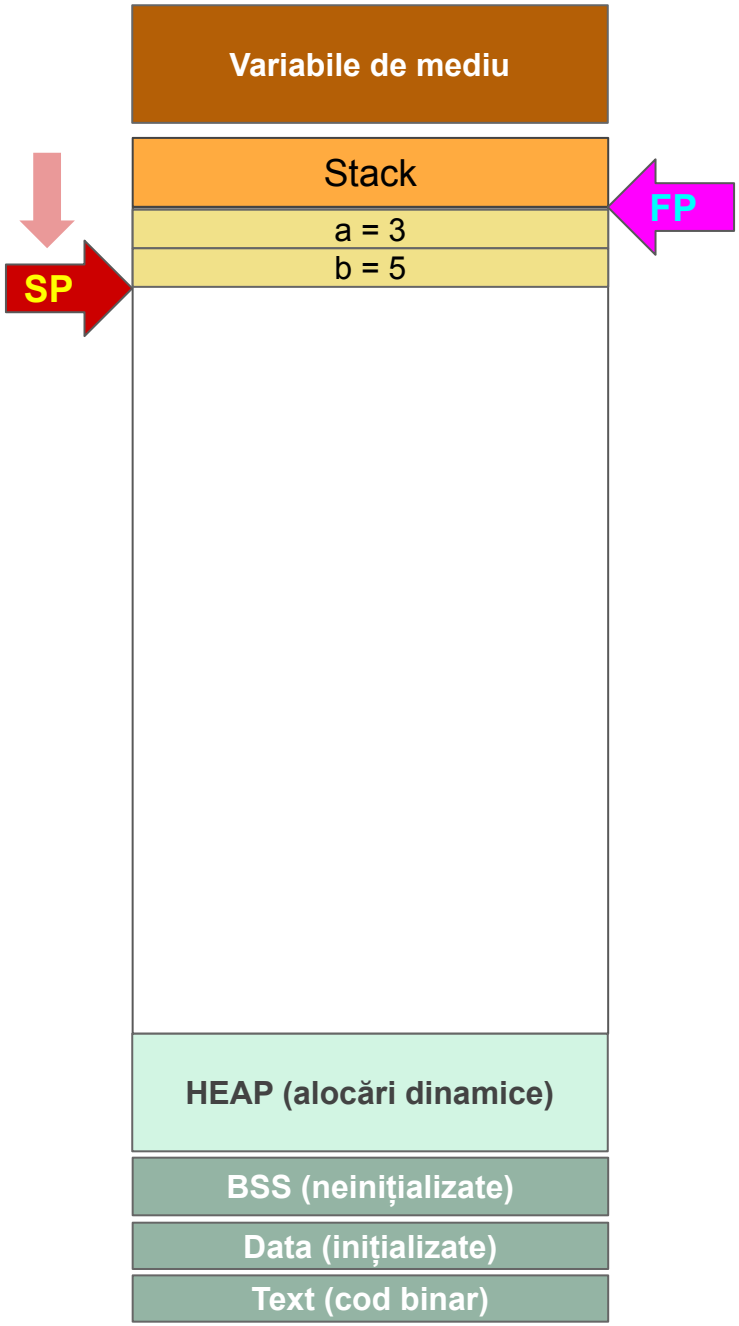




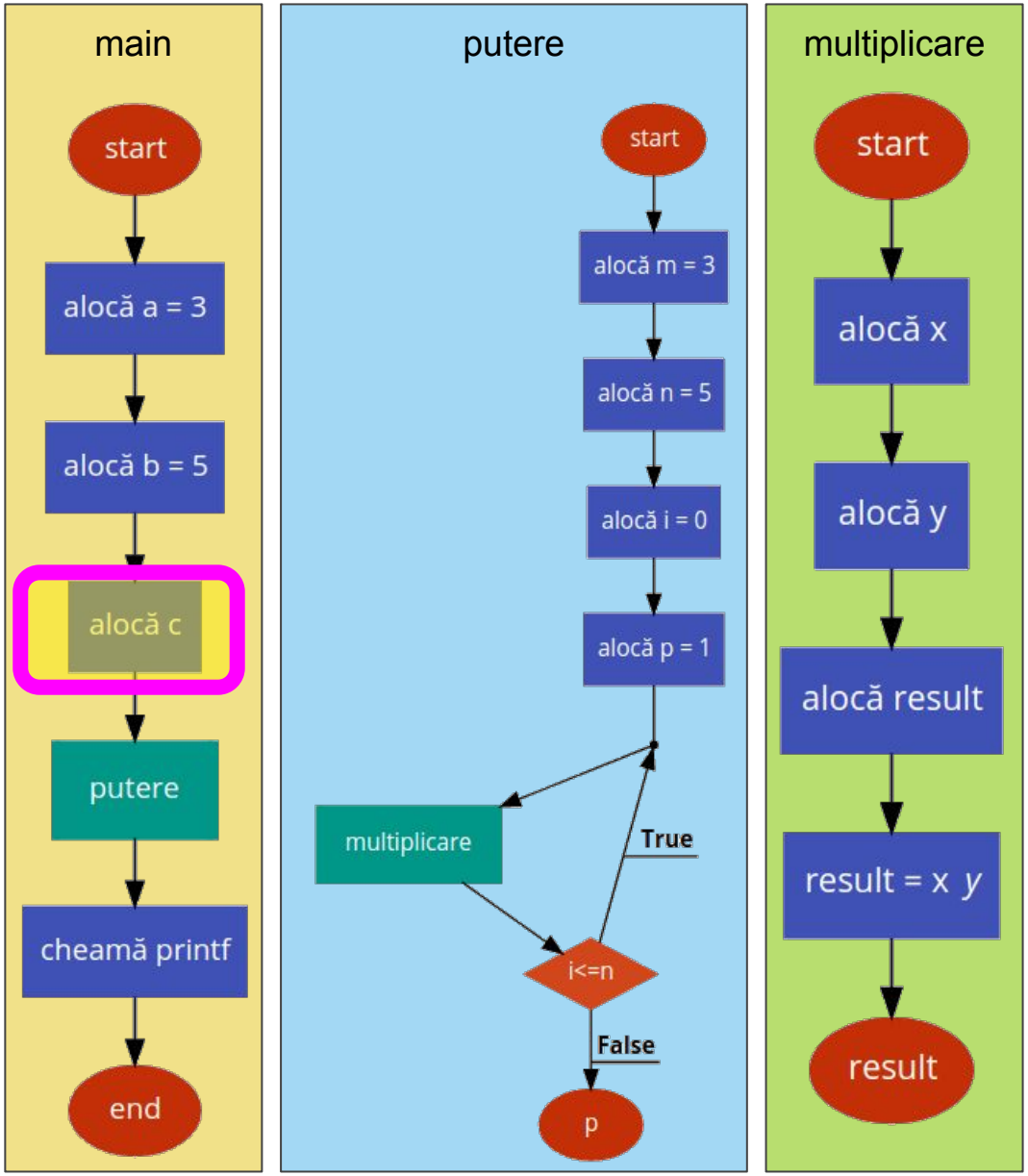


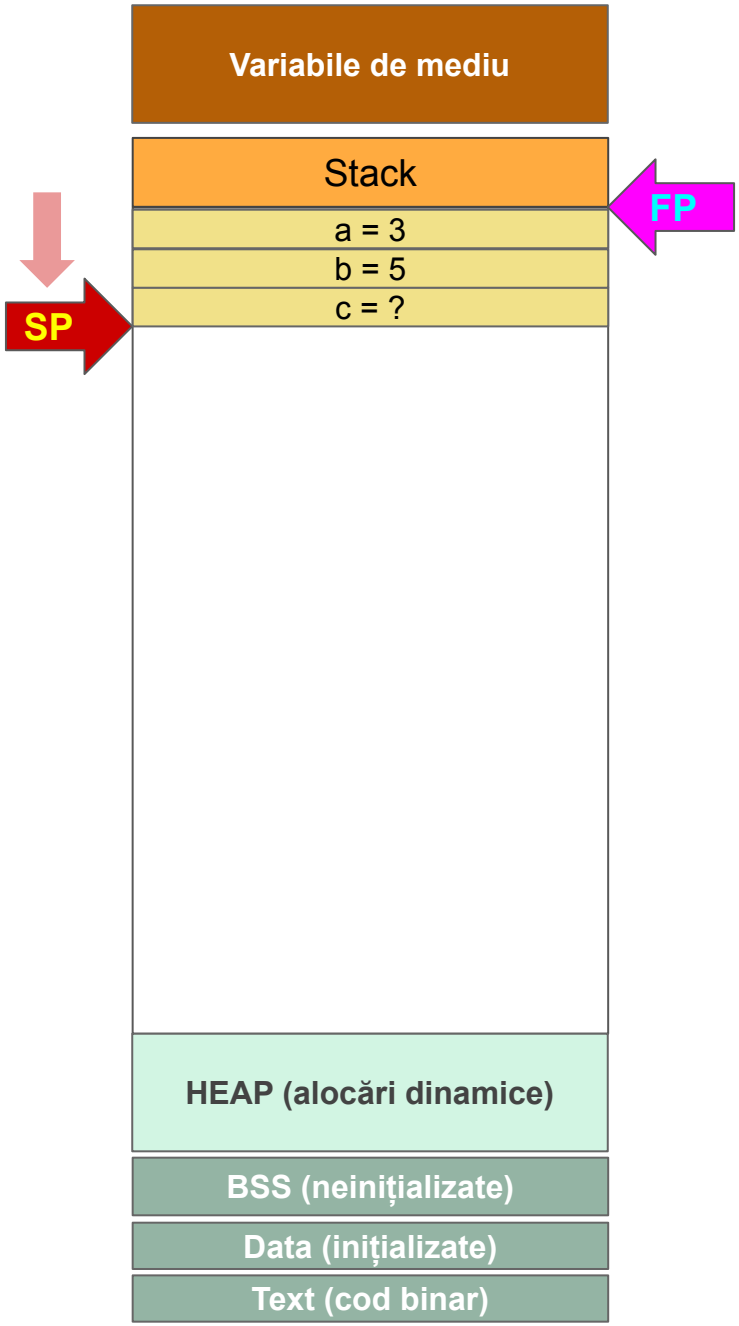
Viața unui program



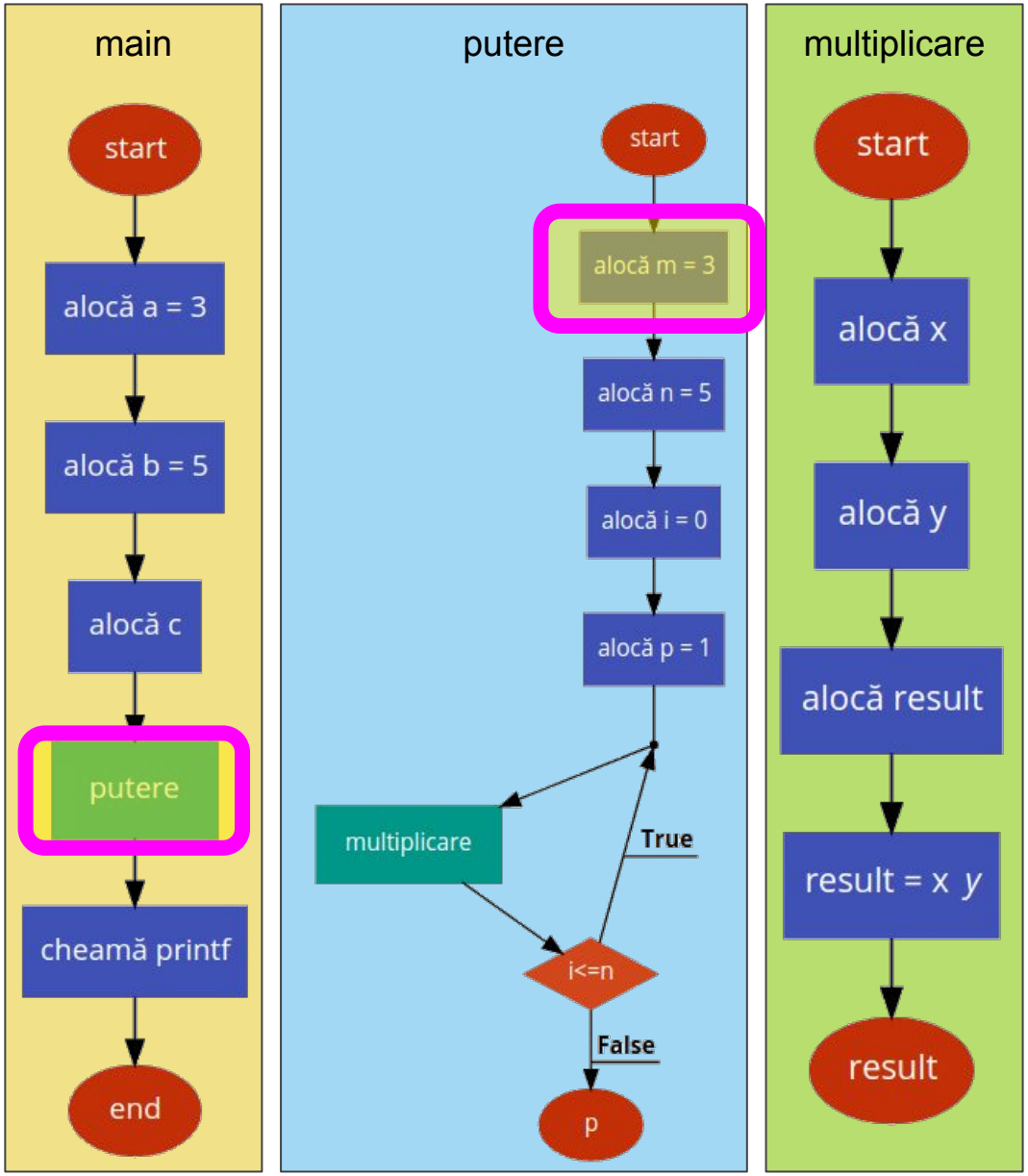


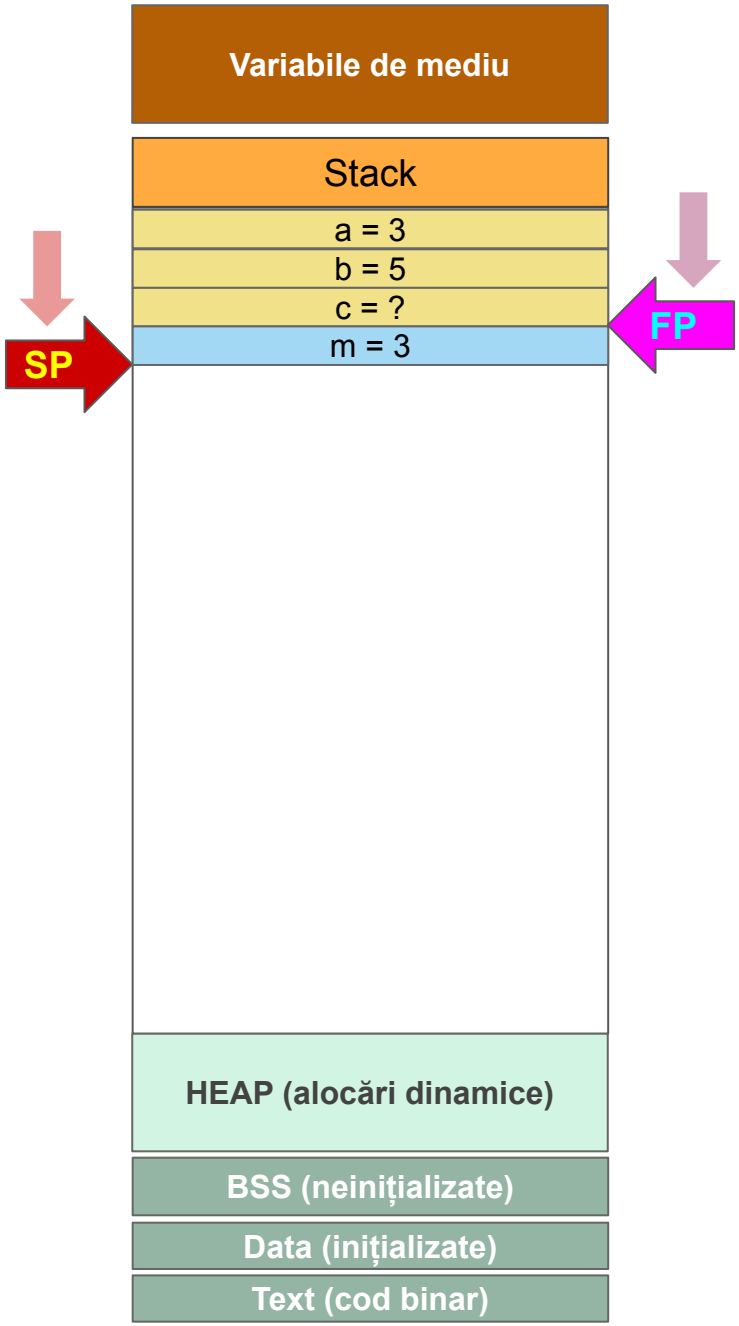
Viața unui program



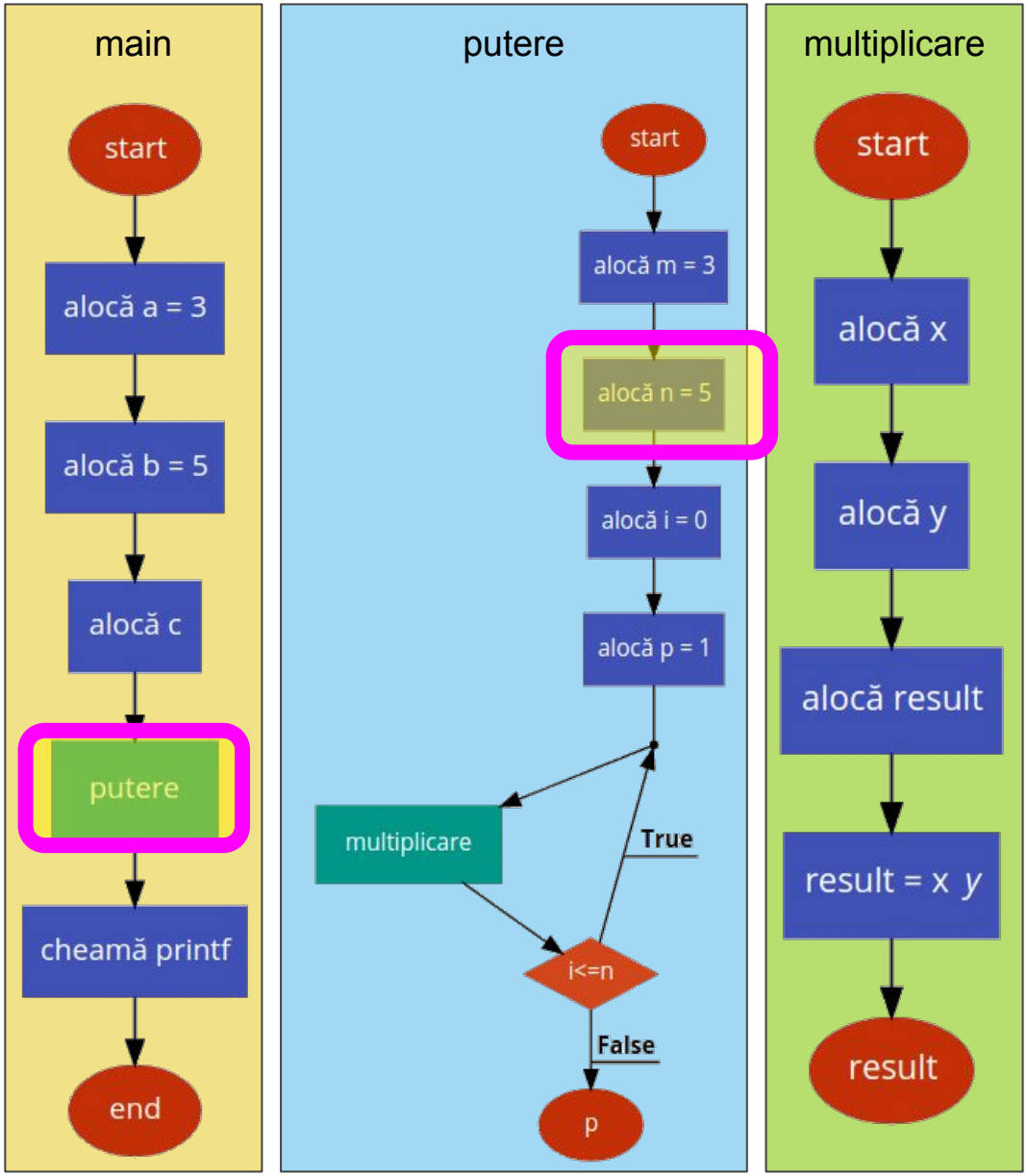


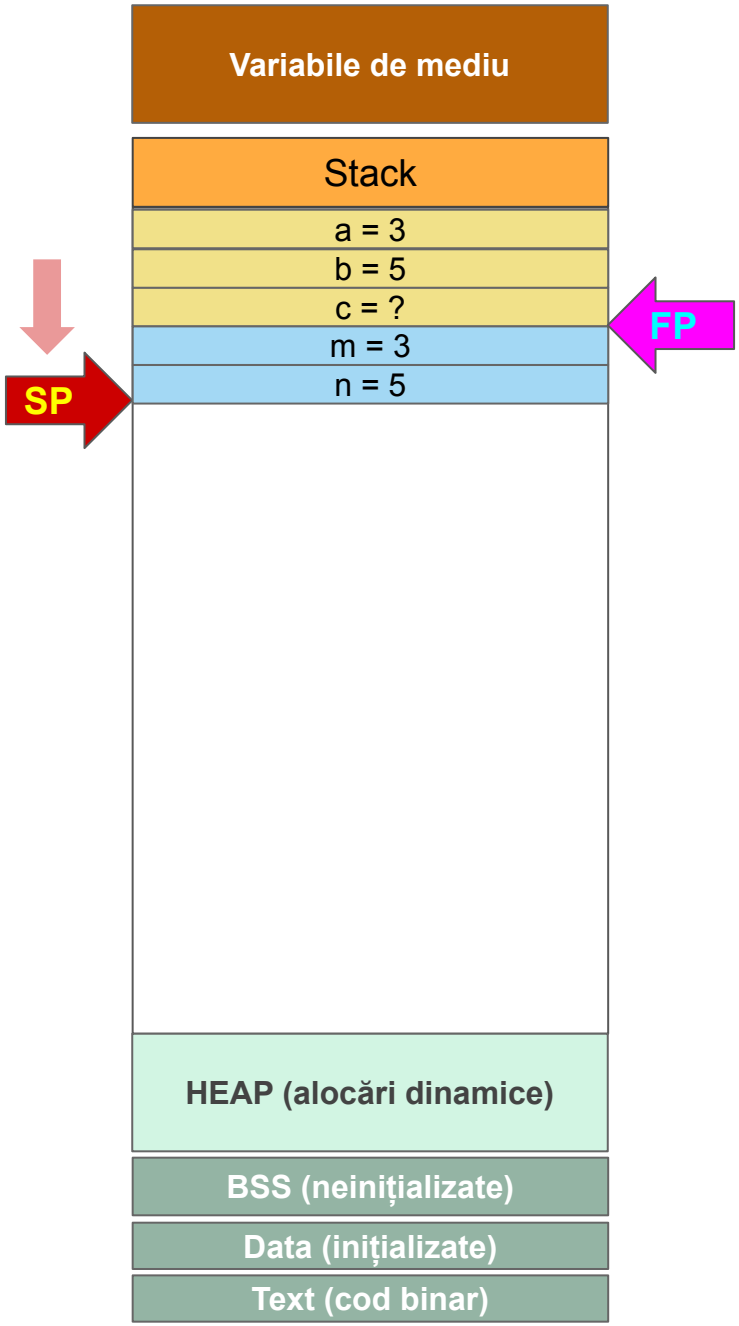
Viața unui program



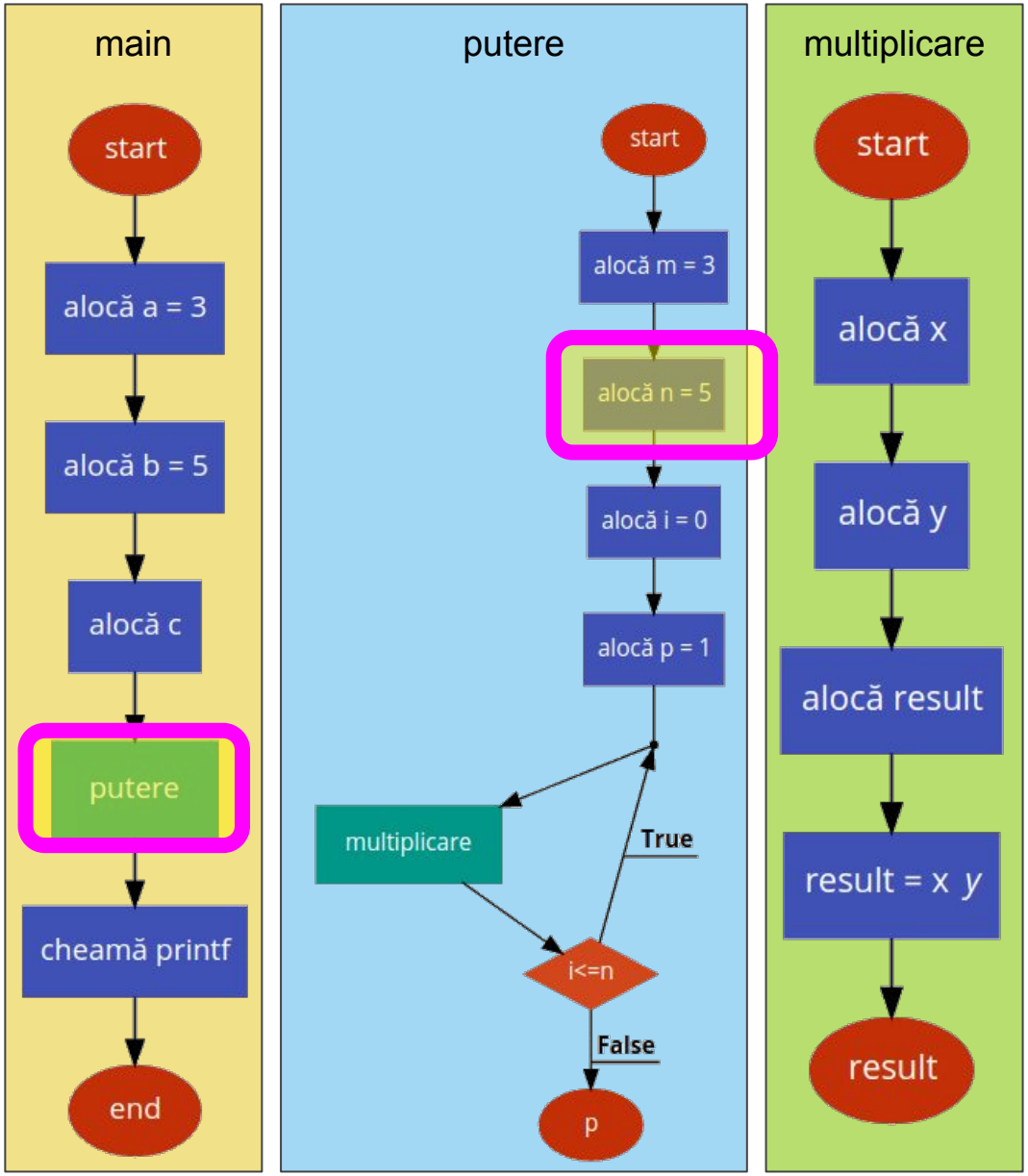


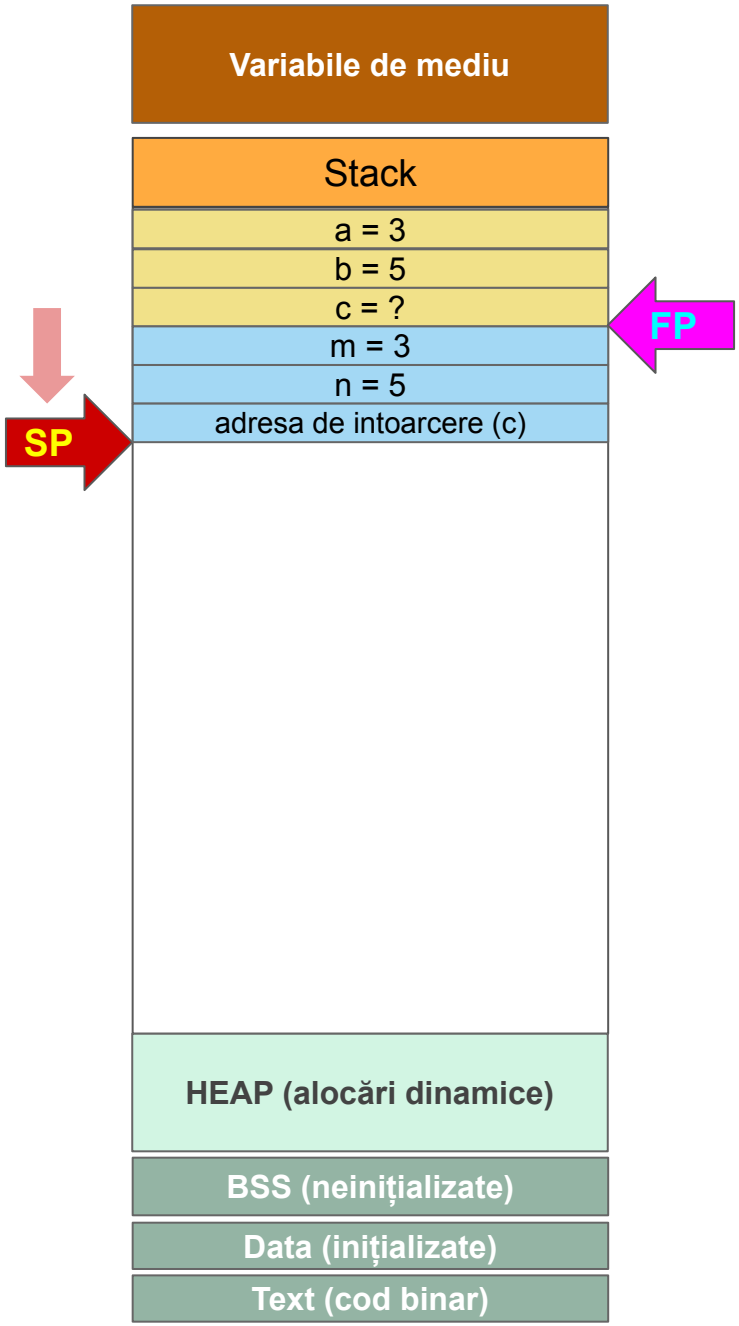
Viața unui program



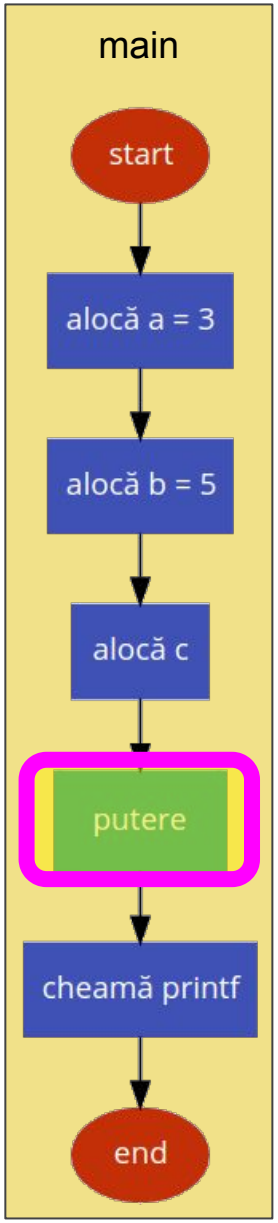


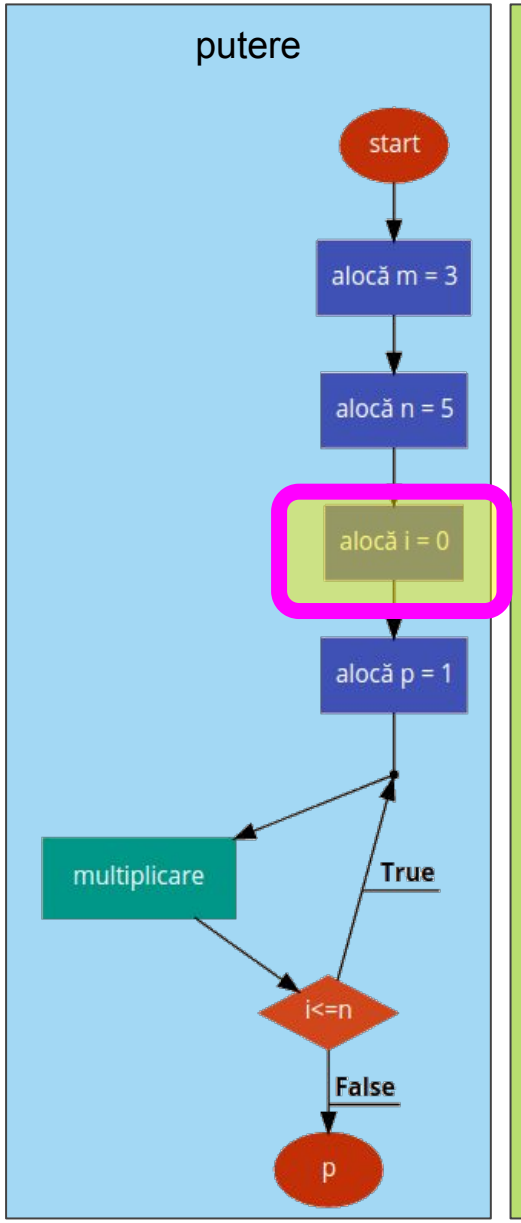
Viața unui program

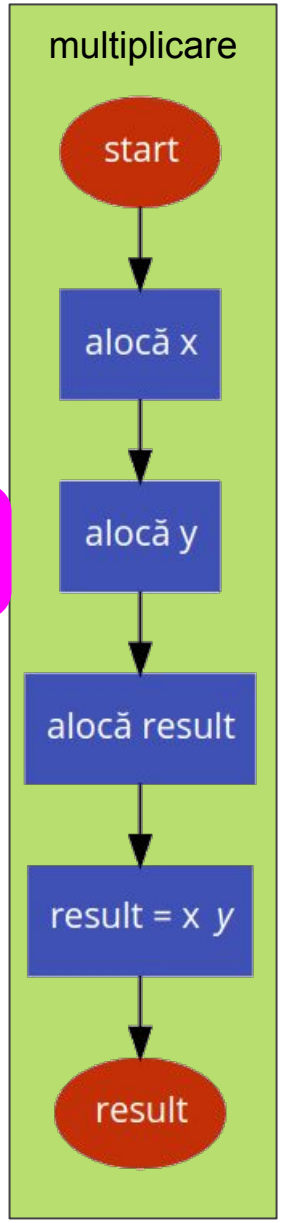


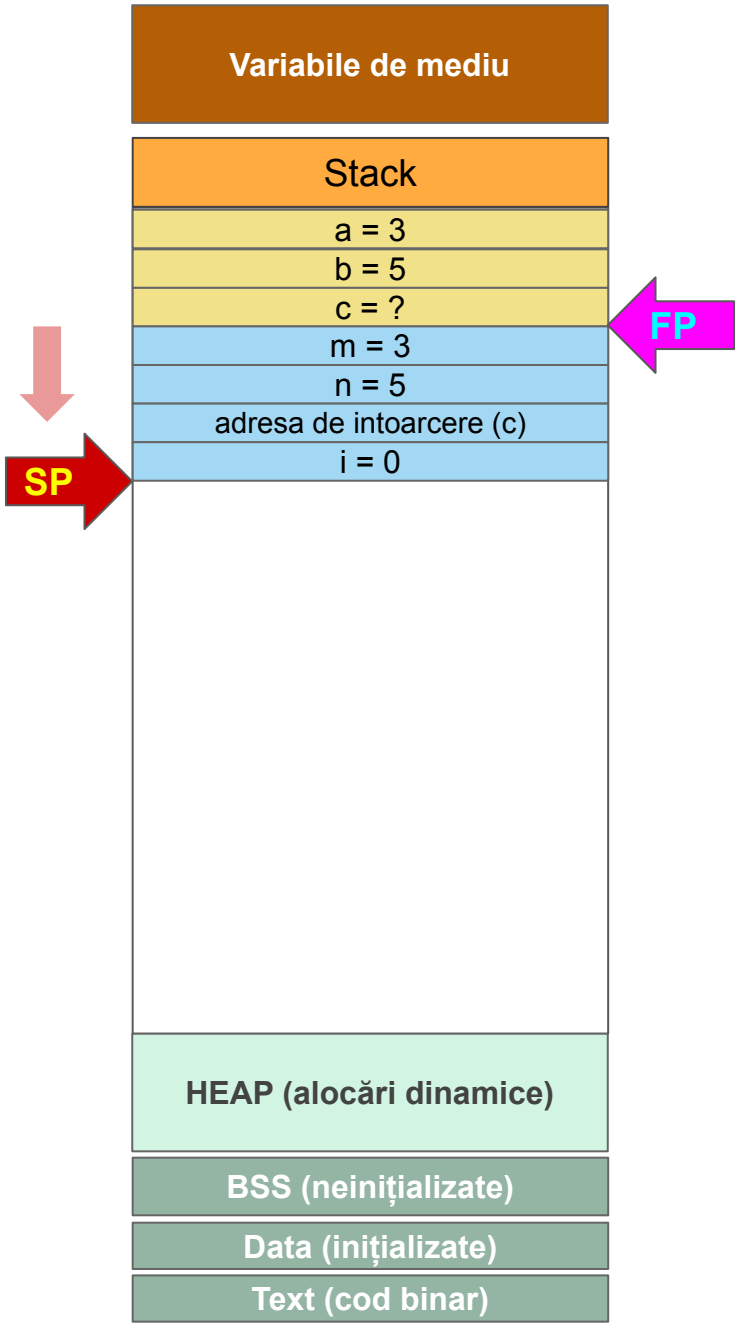


Viața unui program

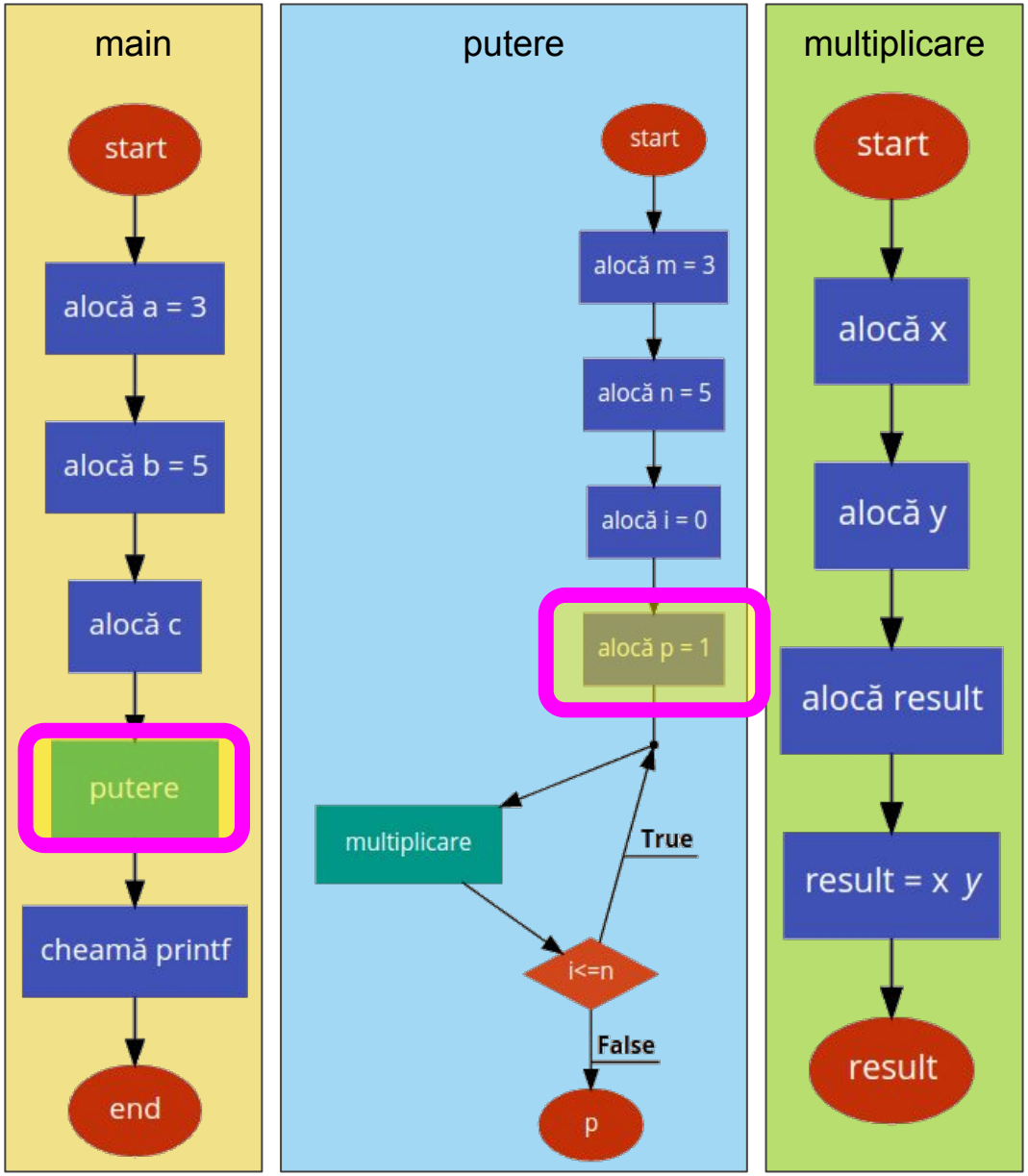


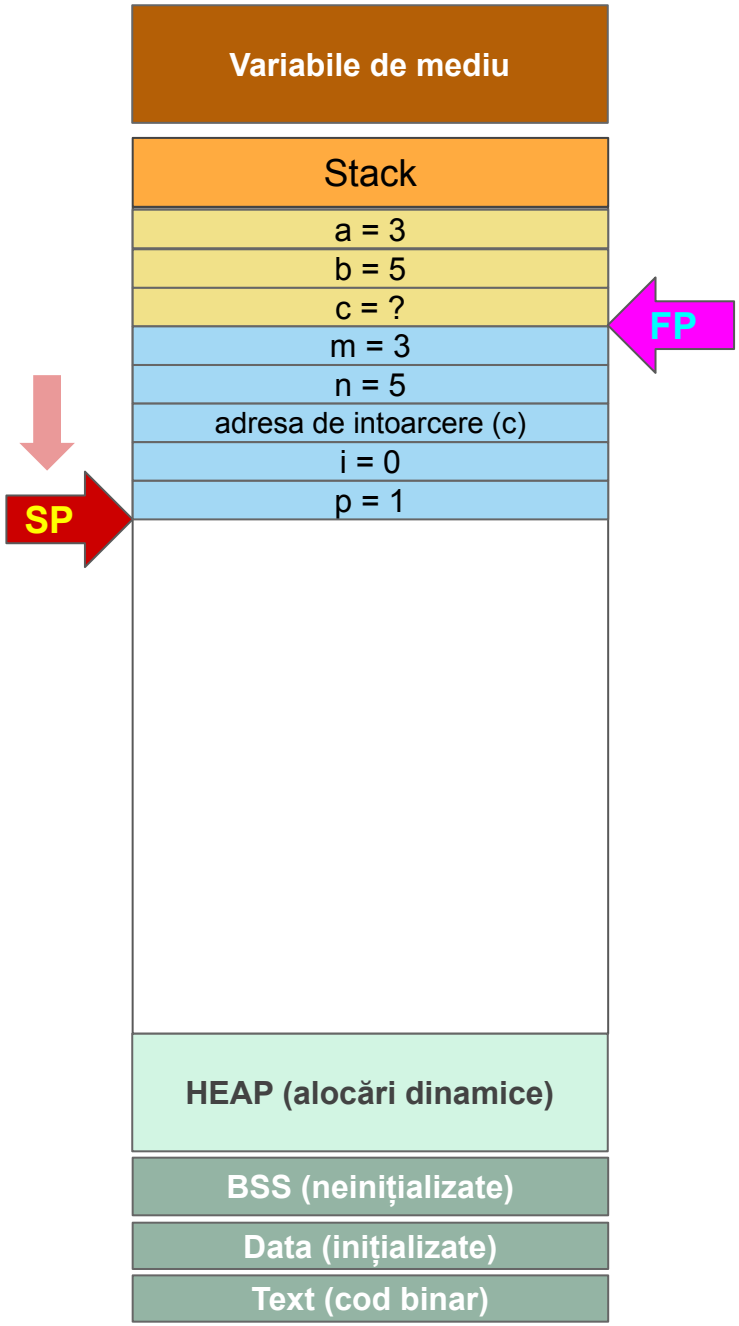




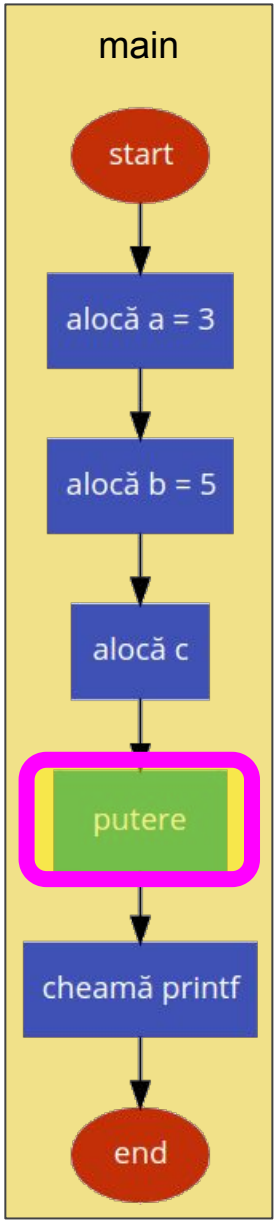


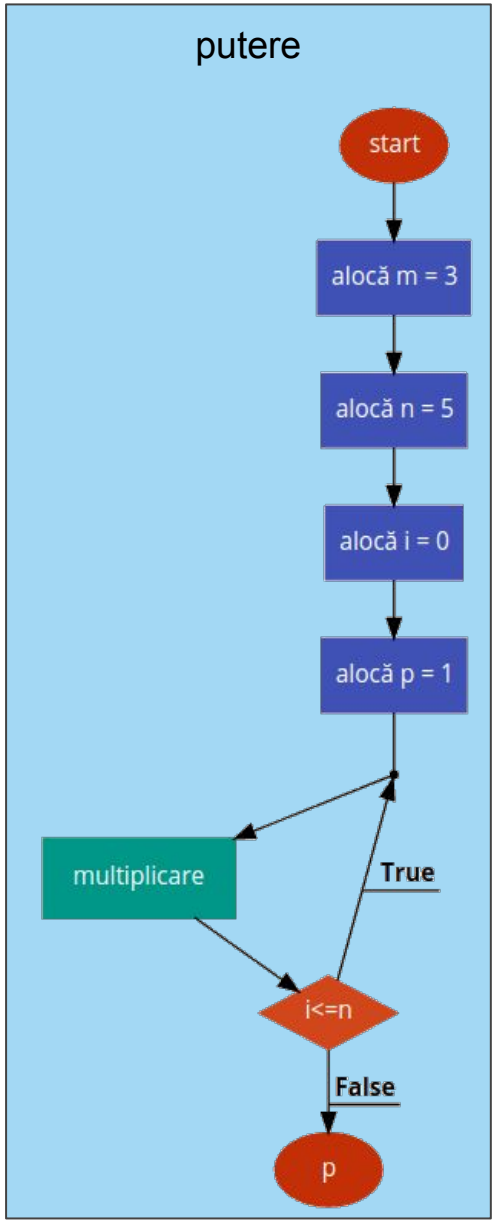
Viața unui program

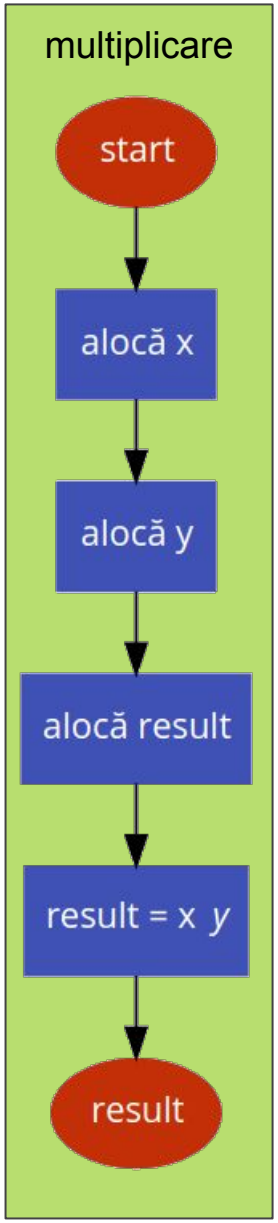


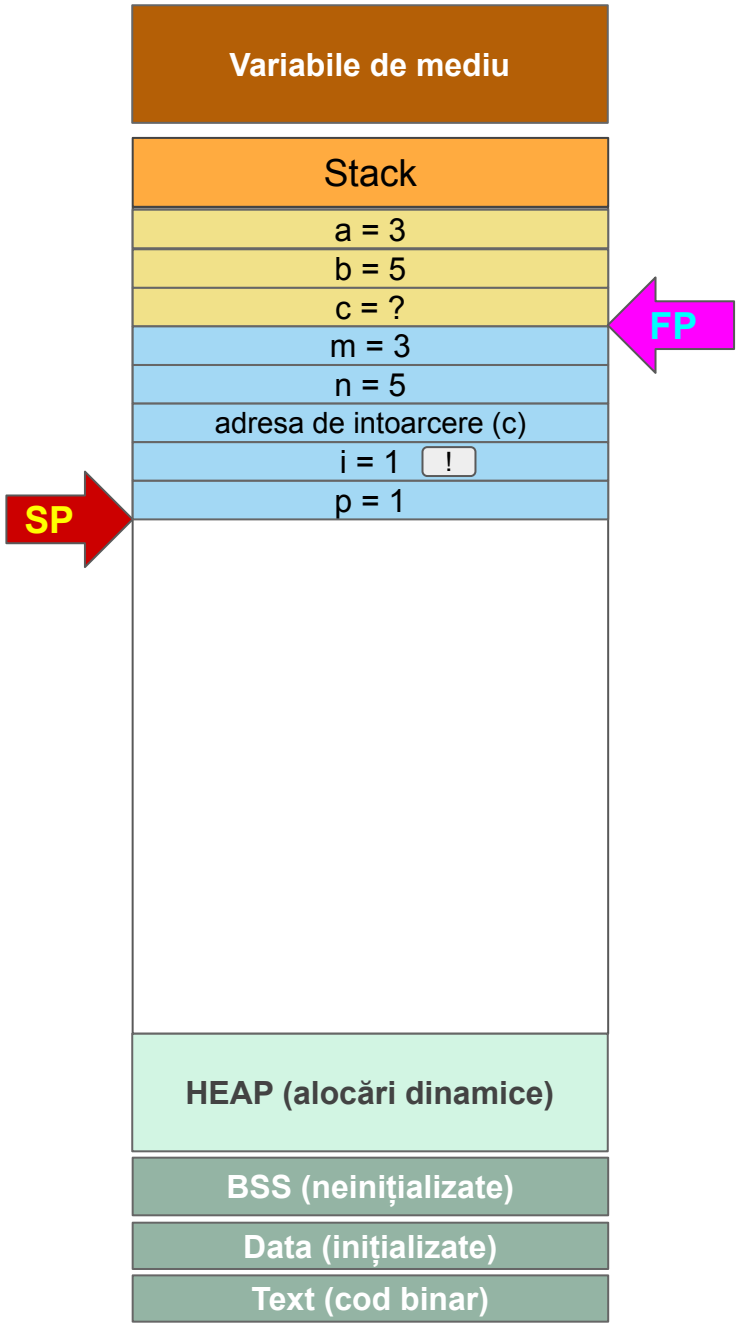


Viața unui program

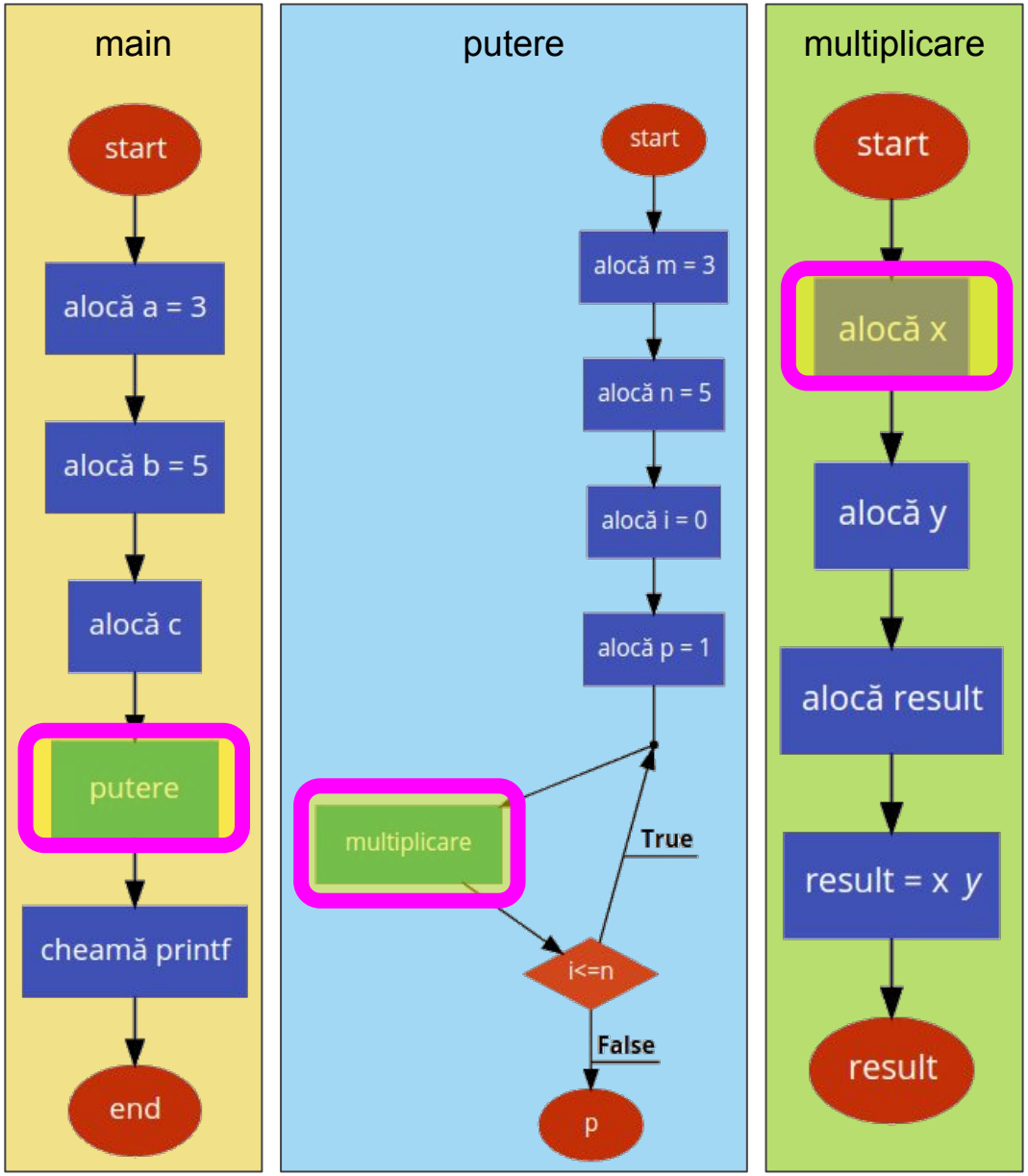


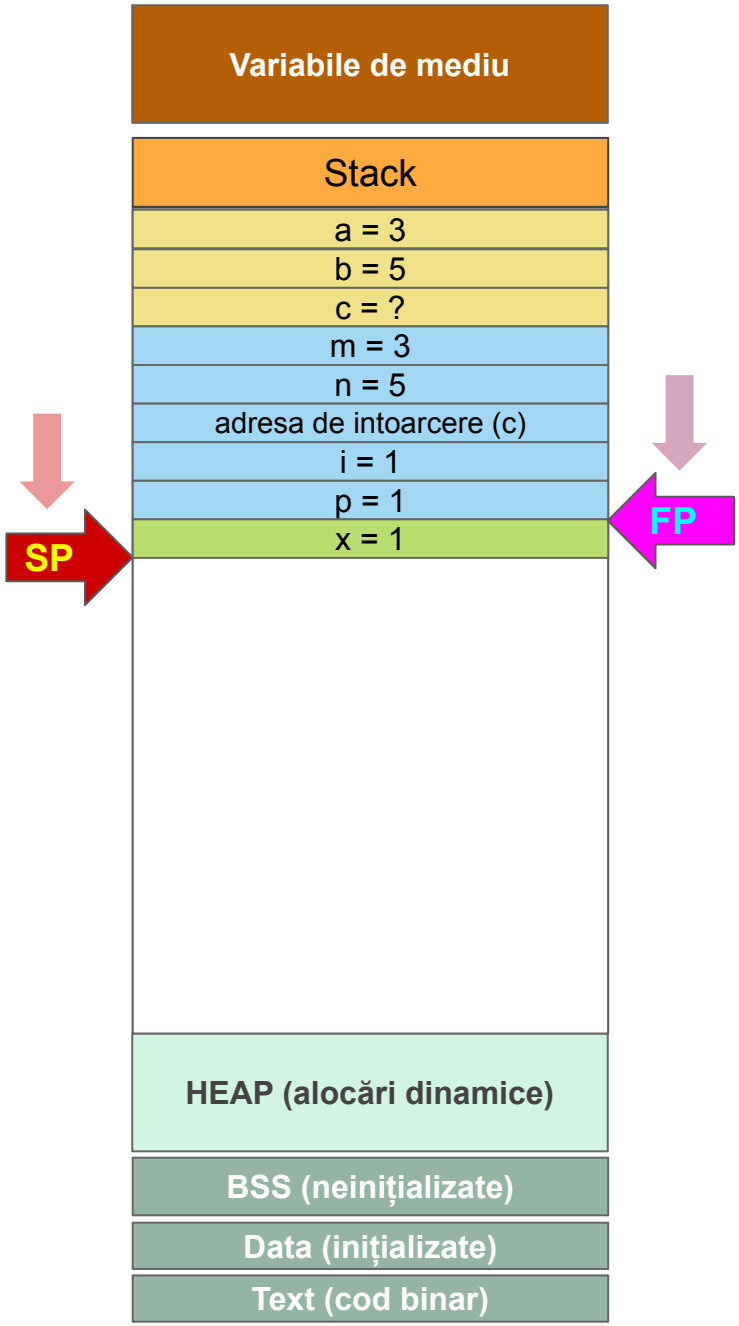




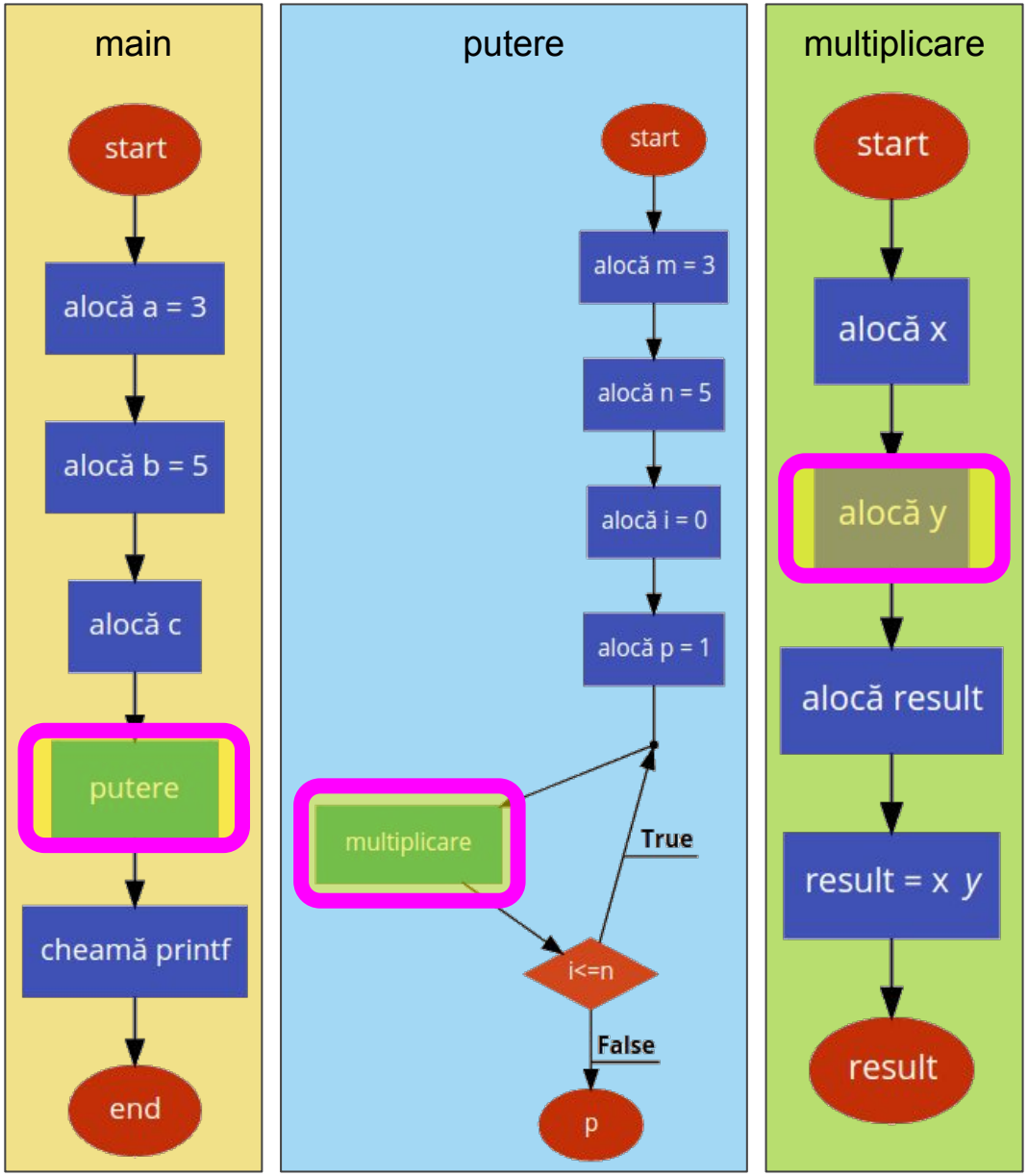


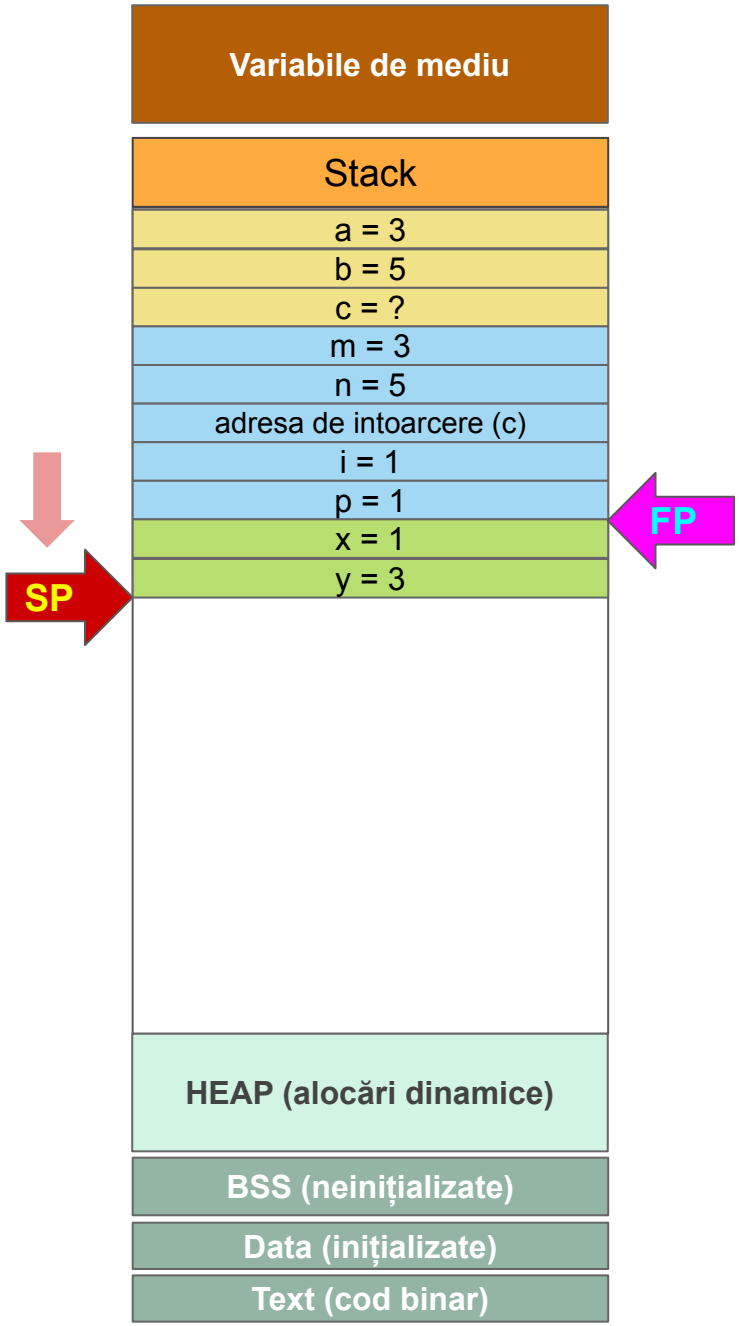
Viața unui program



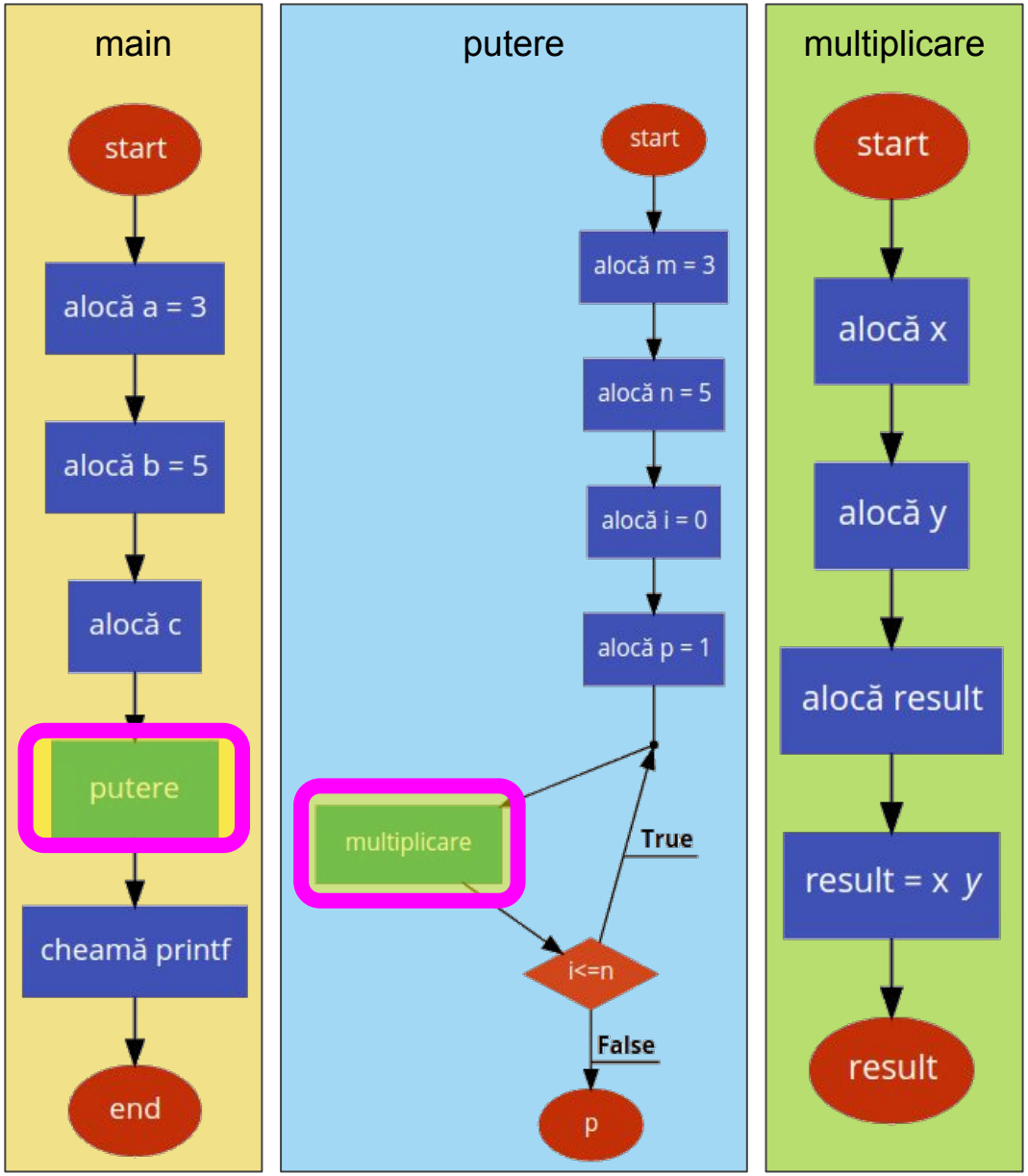


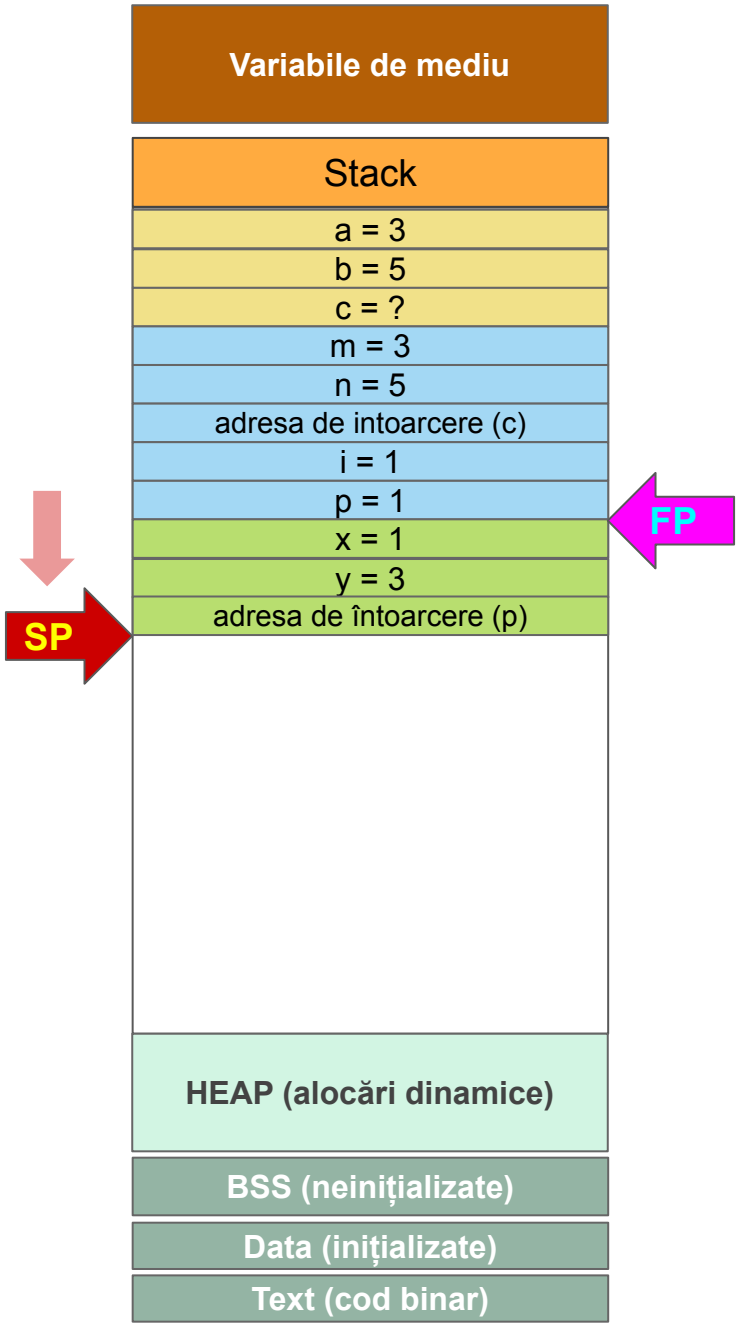
Viața unui program



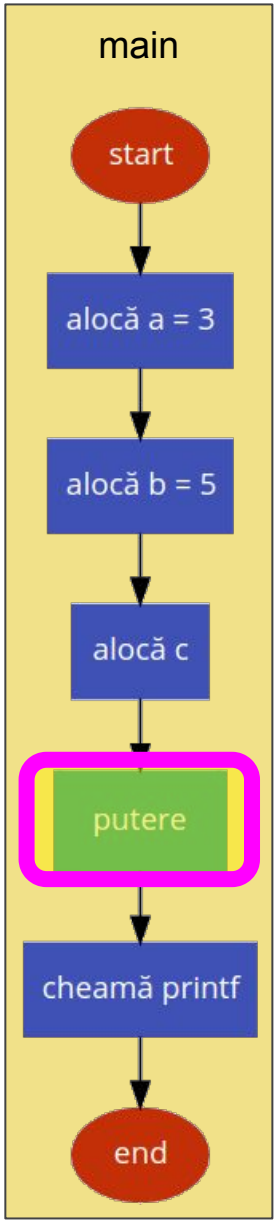


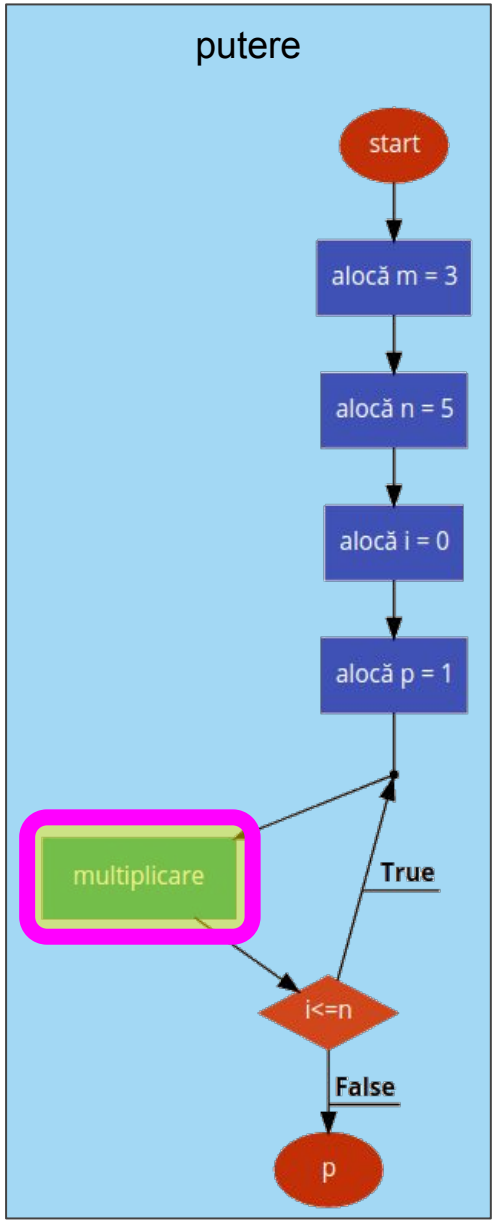
Viața unui program

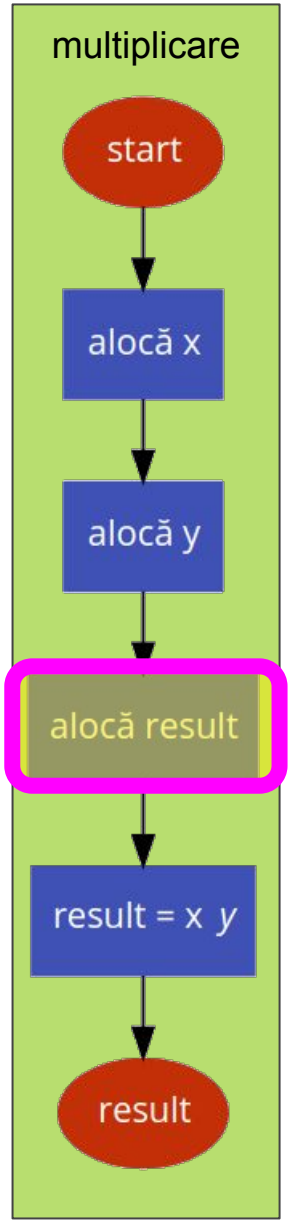


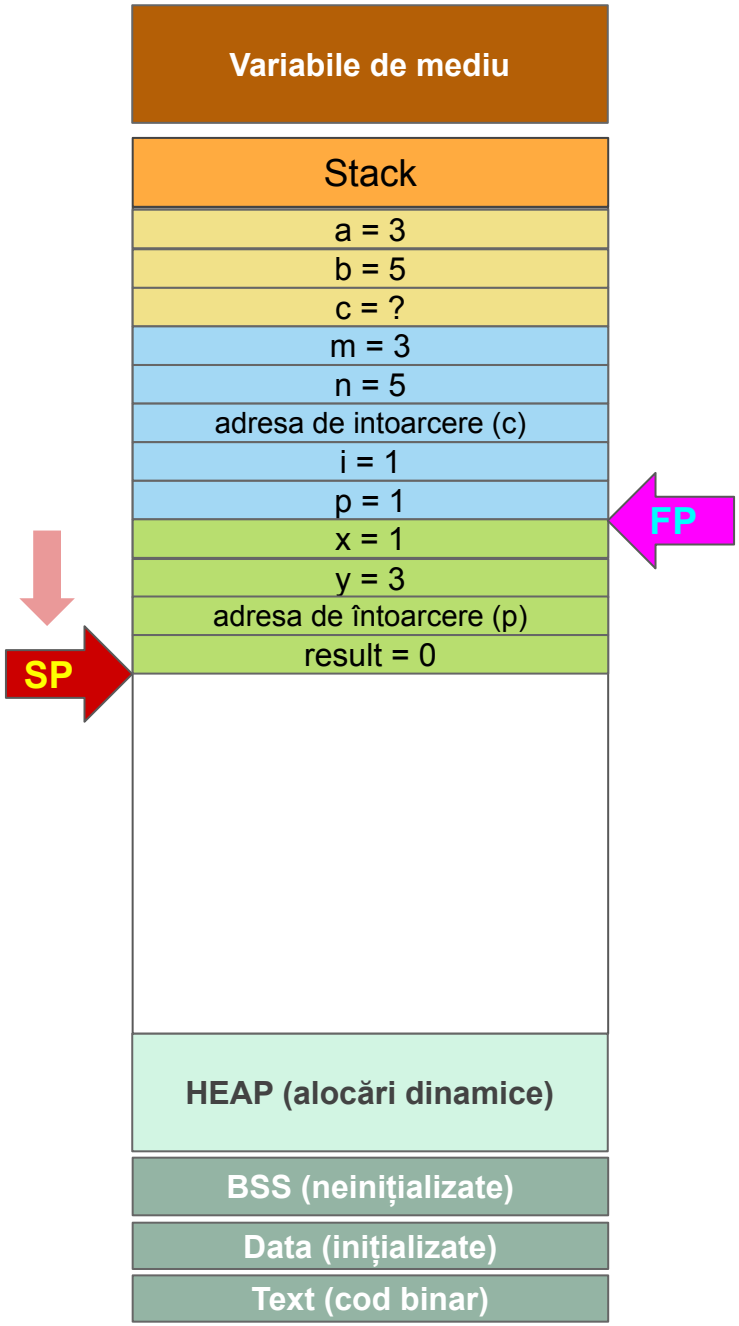


Viața unui program

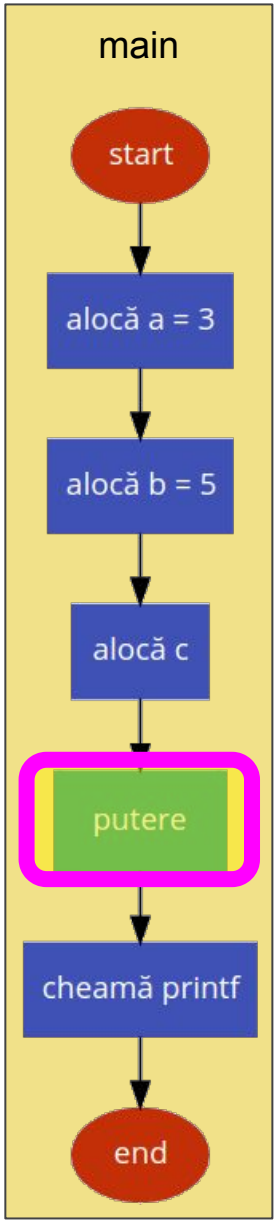


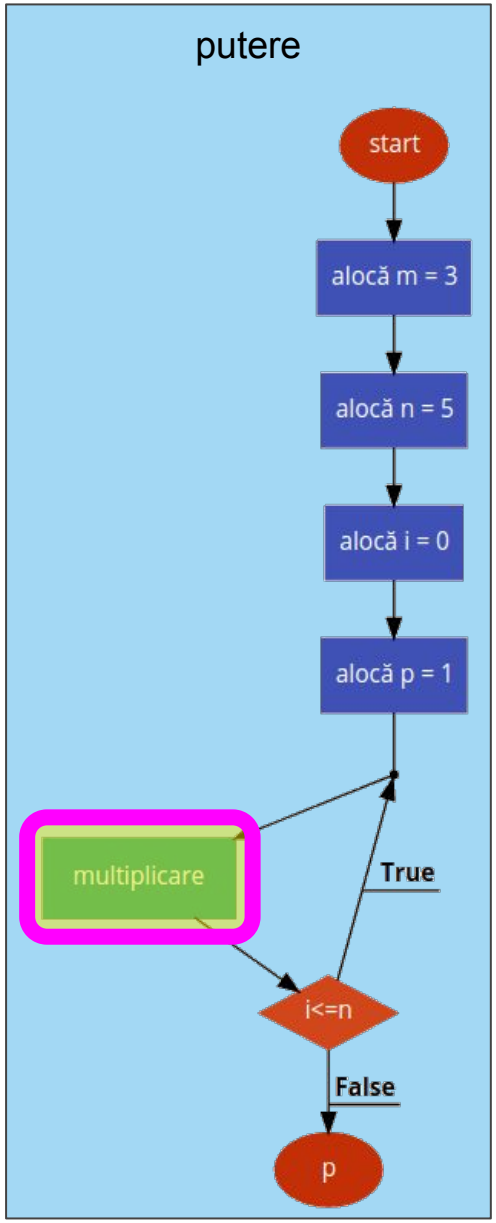


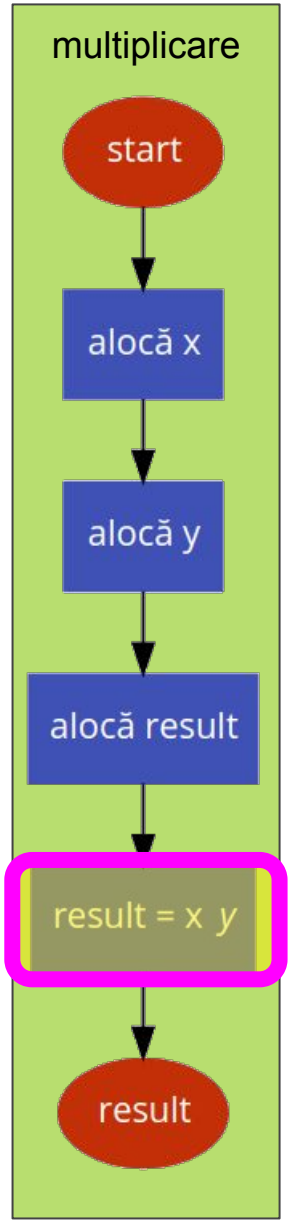


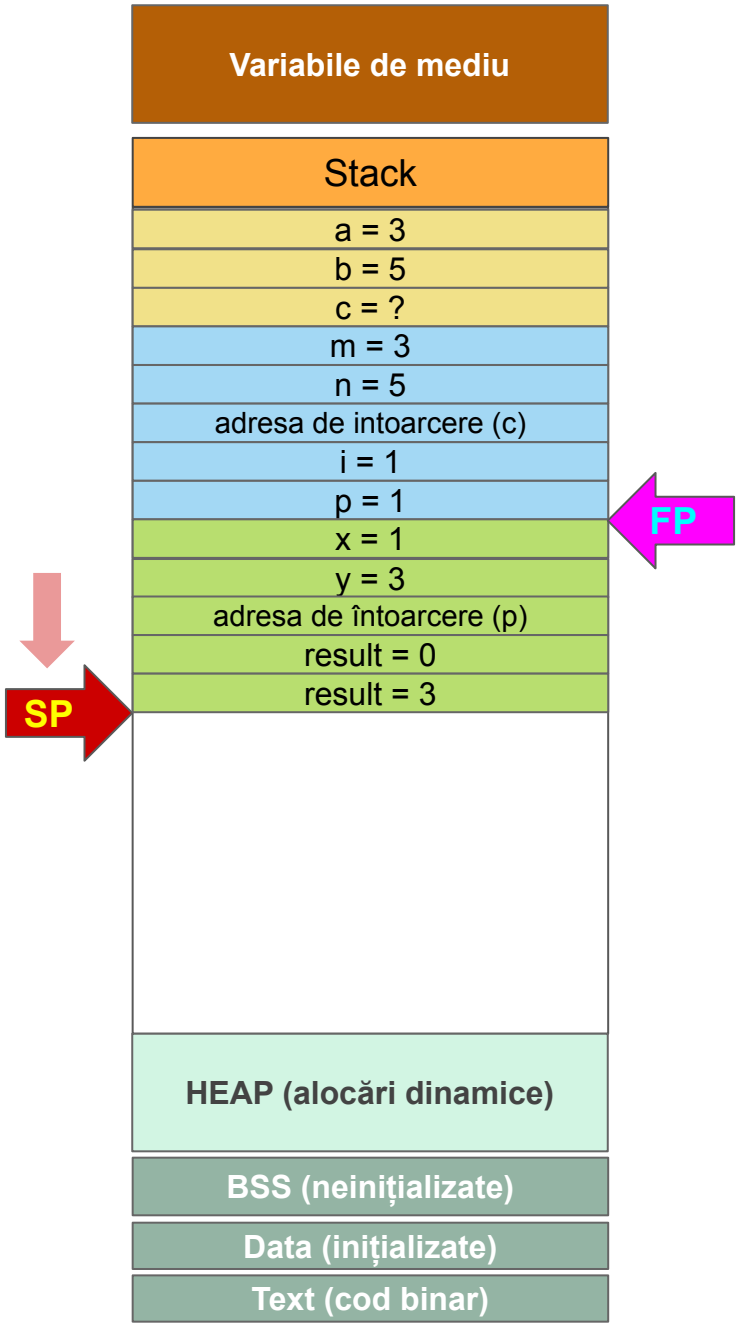


Viața unui program

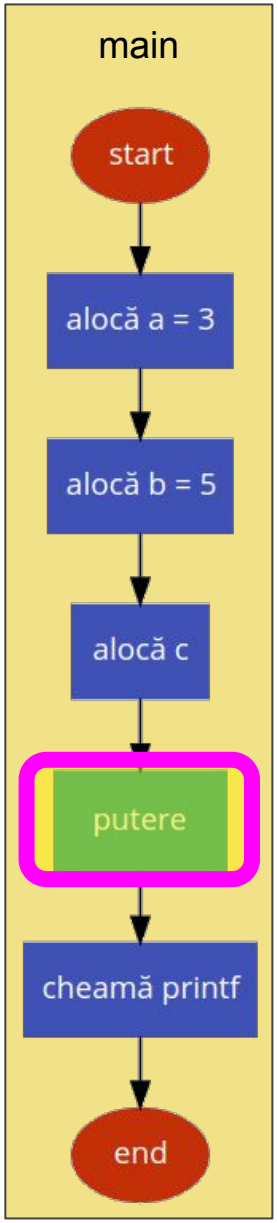


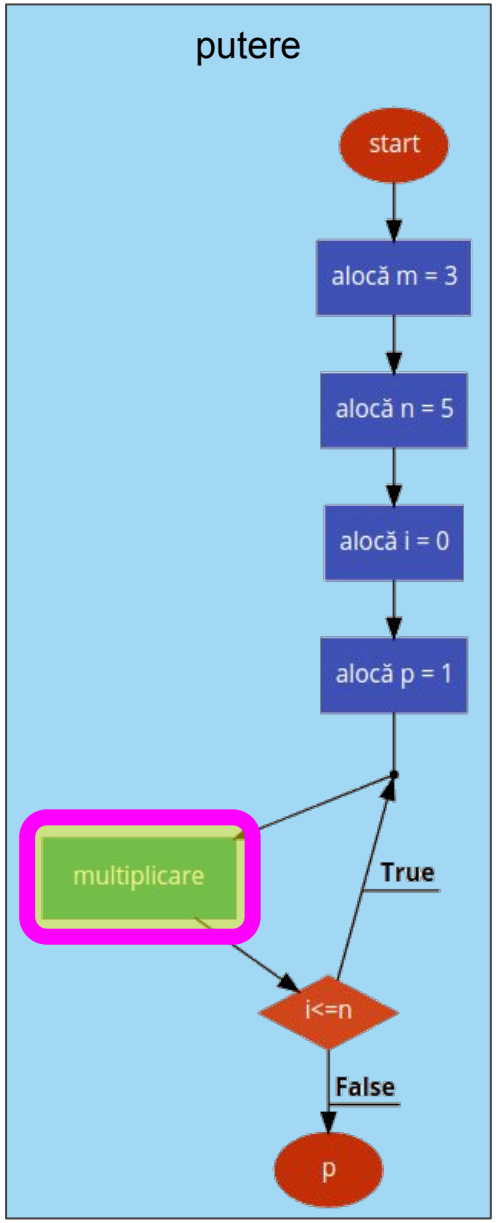


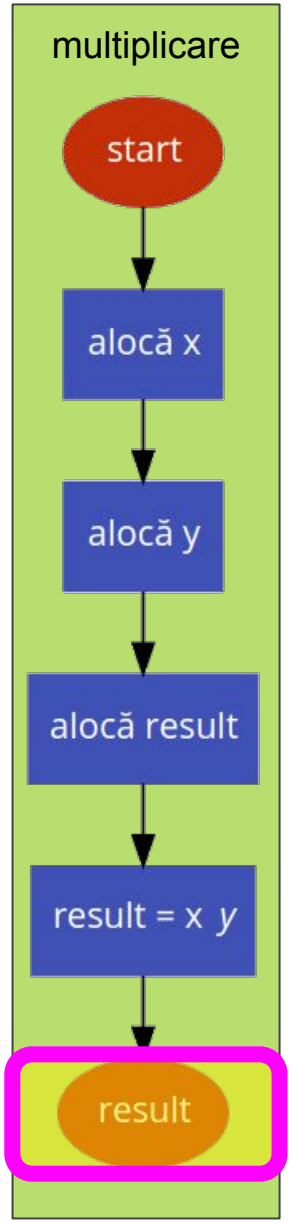


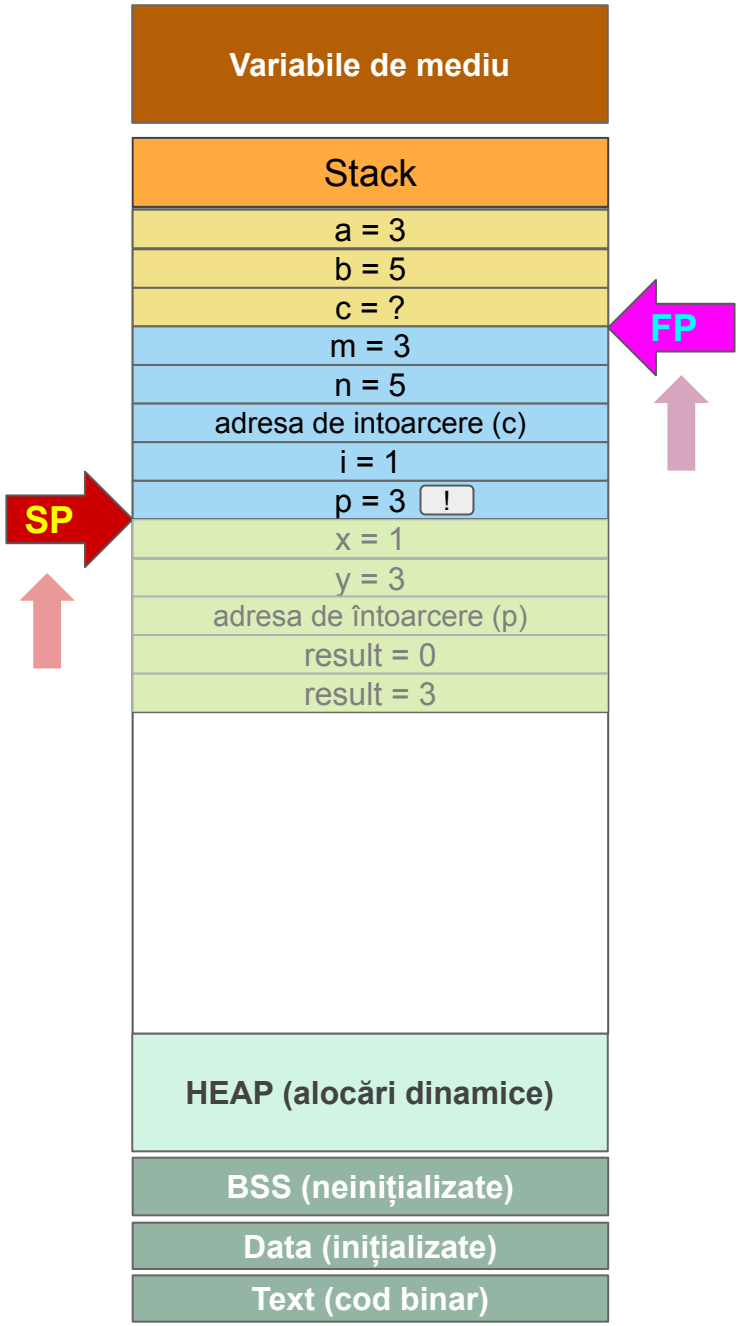


Viața unui program

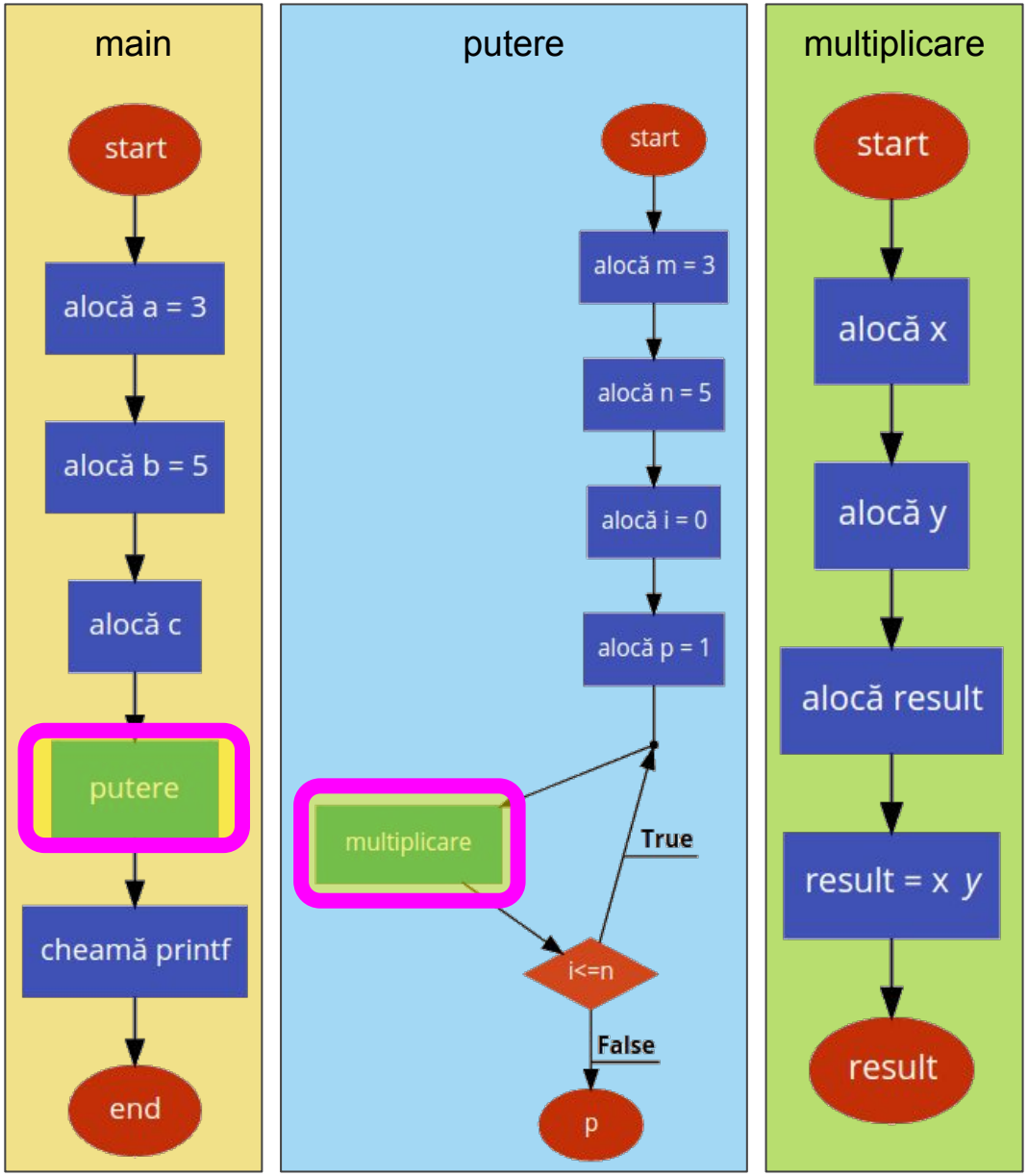


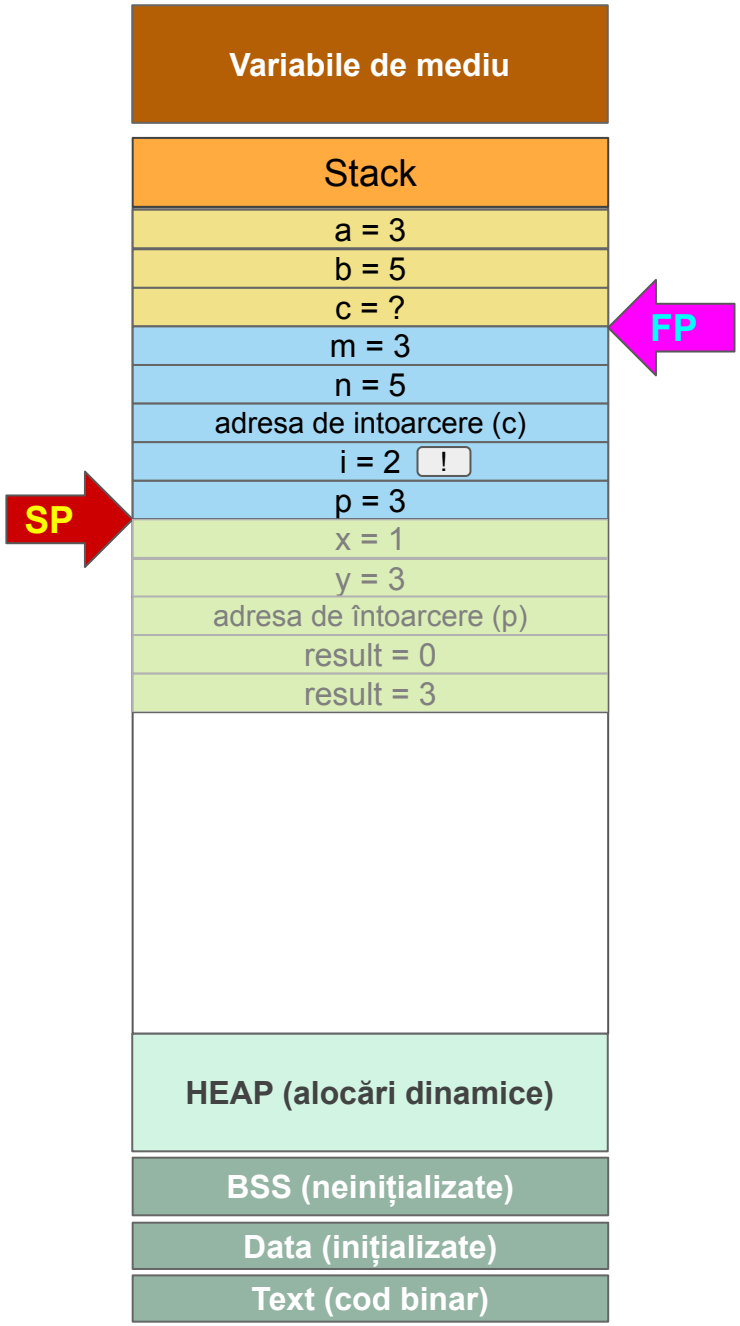




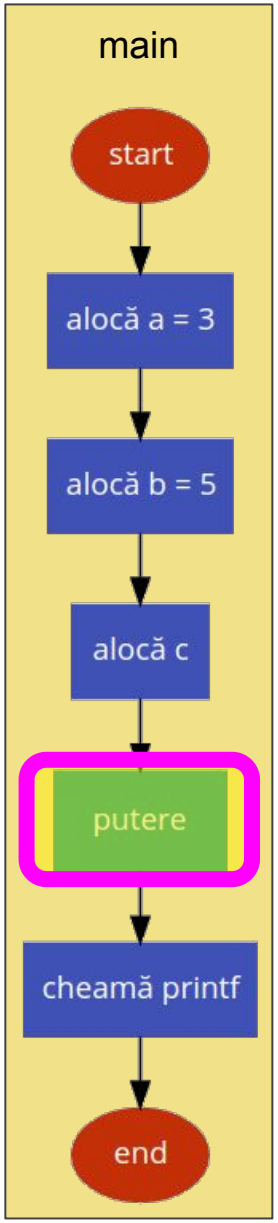


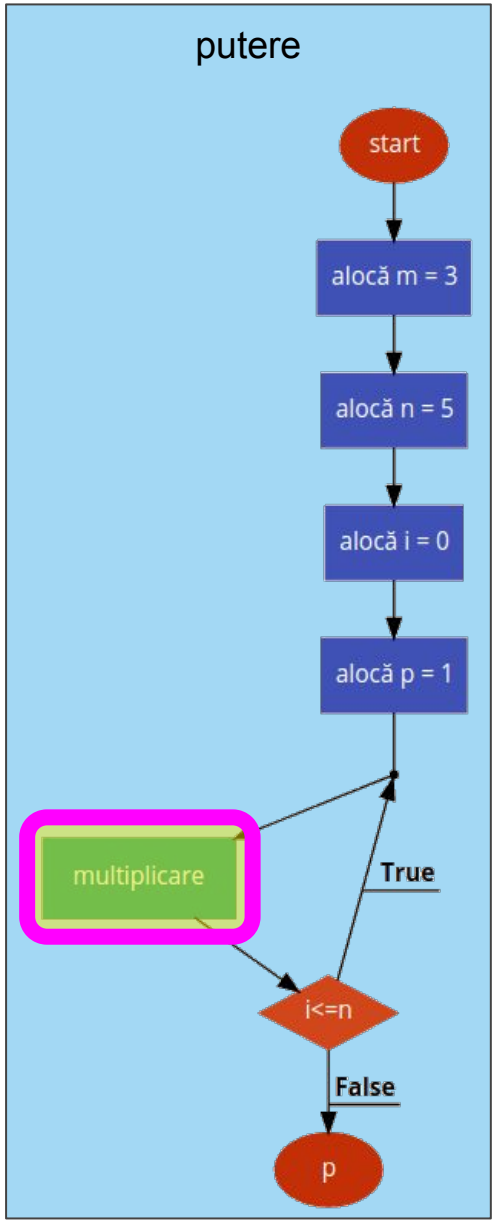
Viața unui program

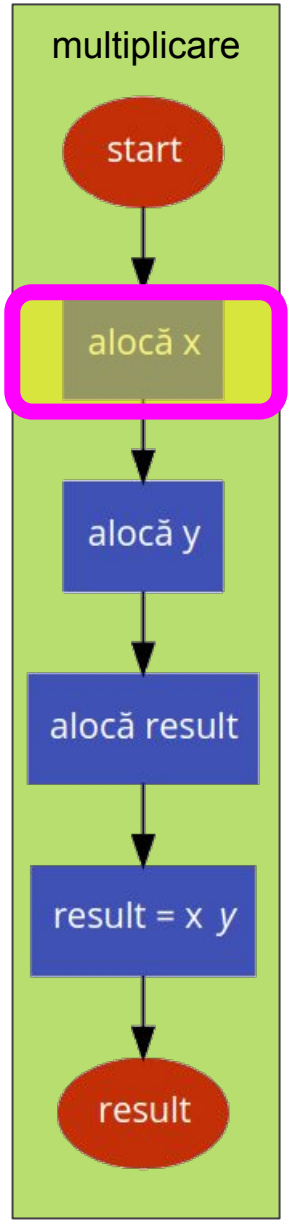


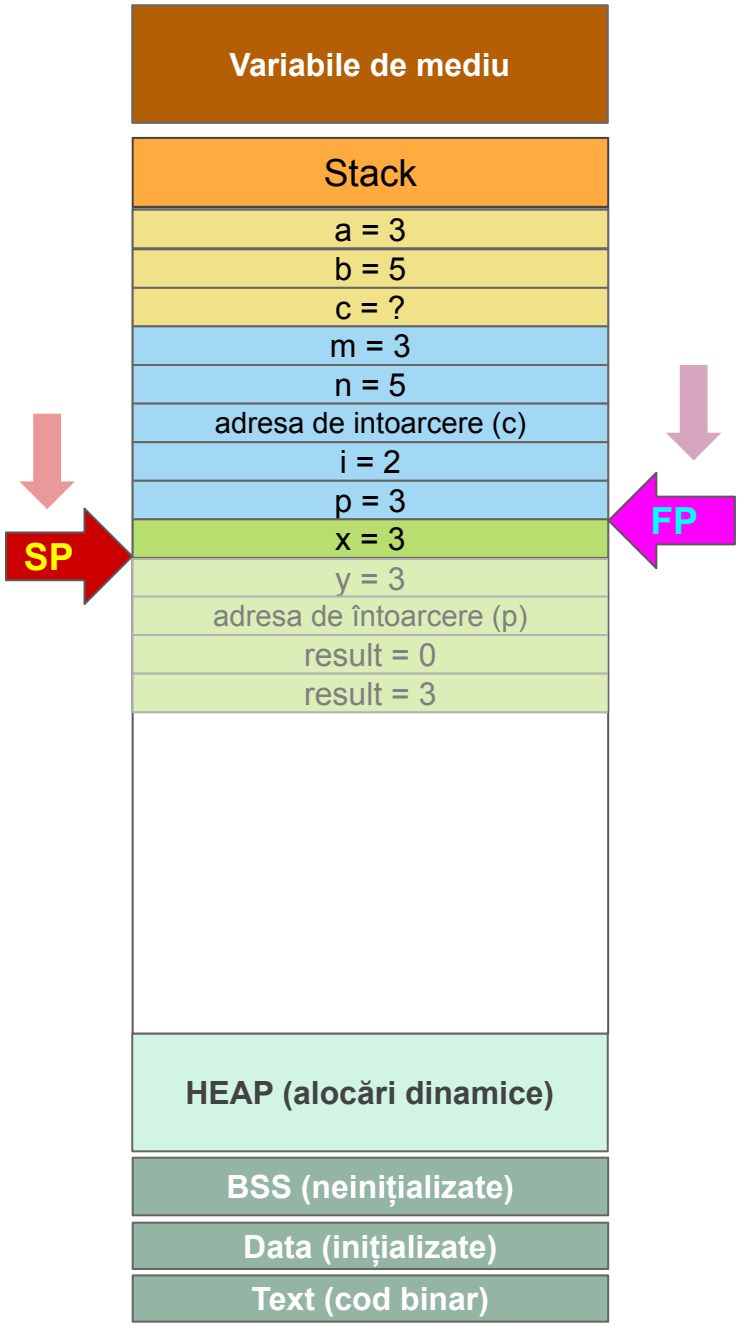


Viața unui program

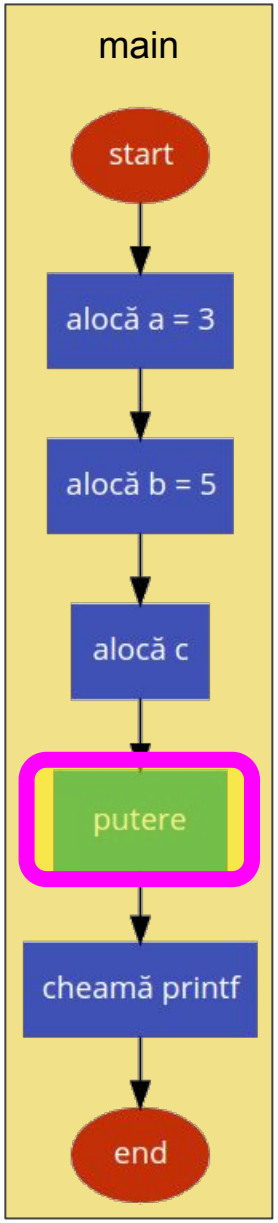


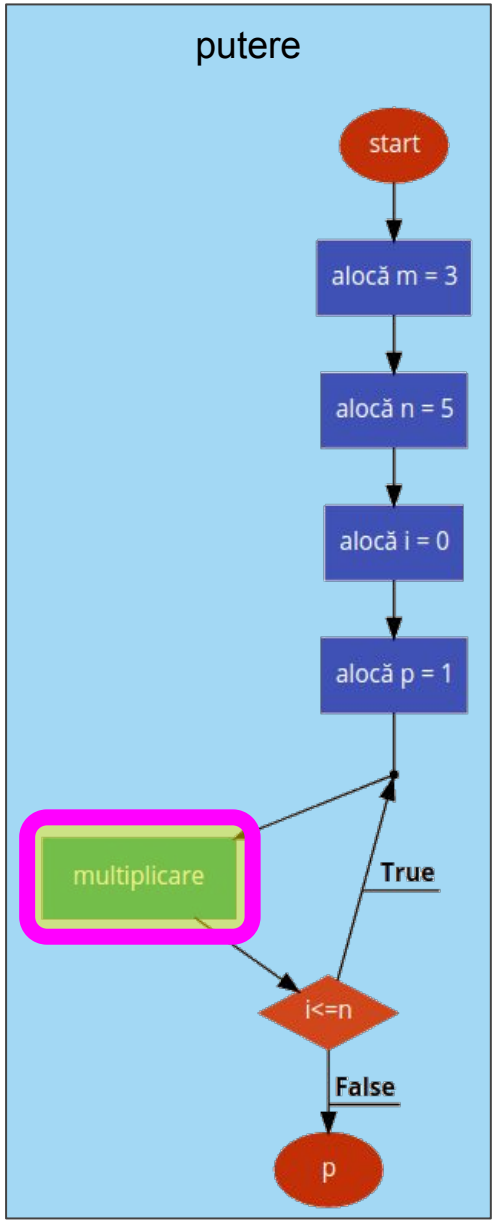


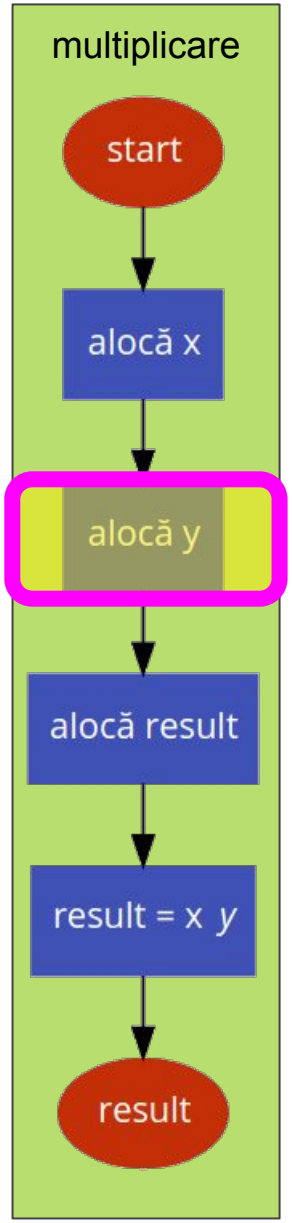


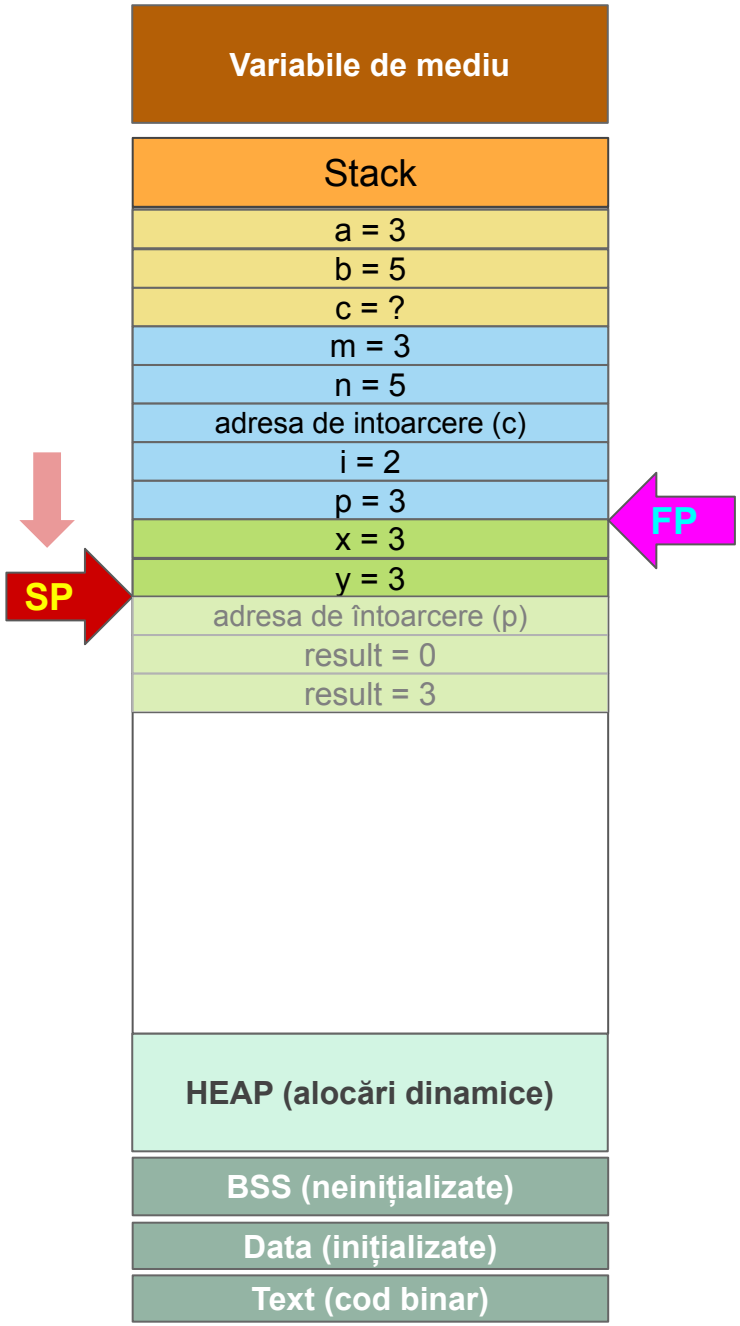


Viața unui program

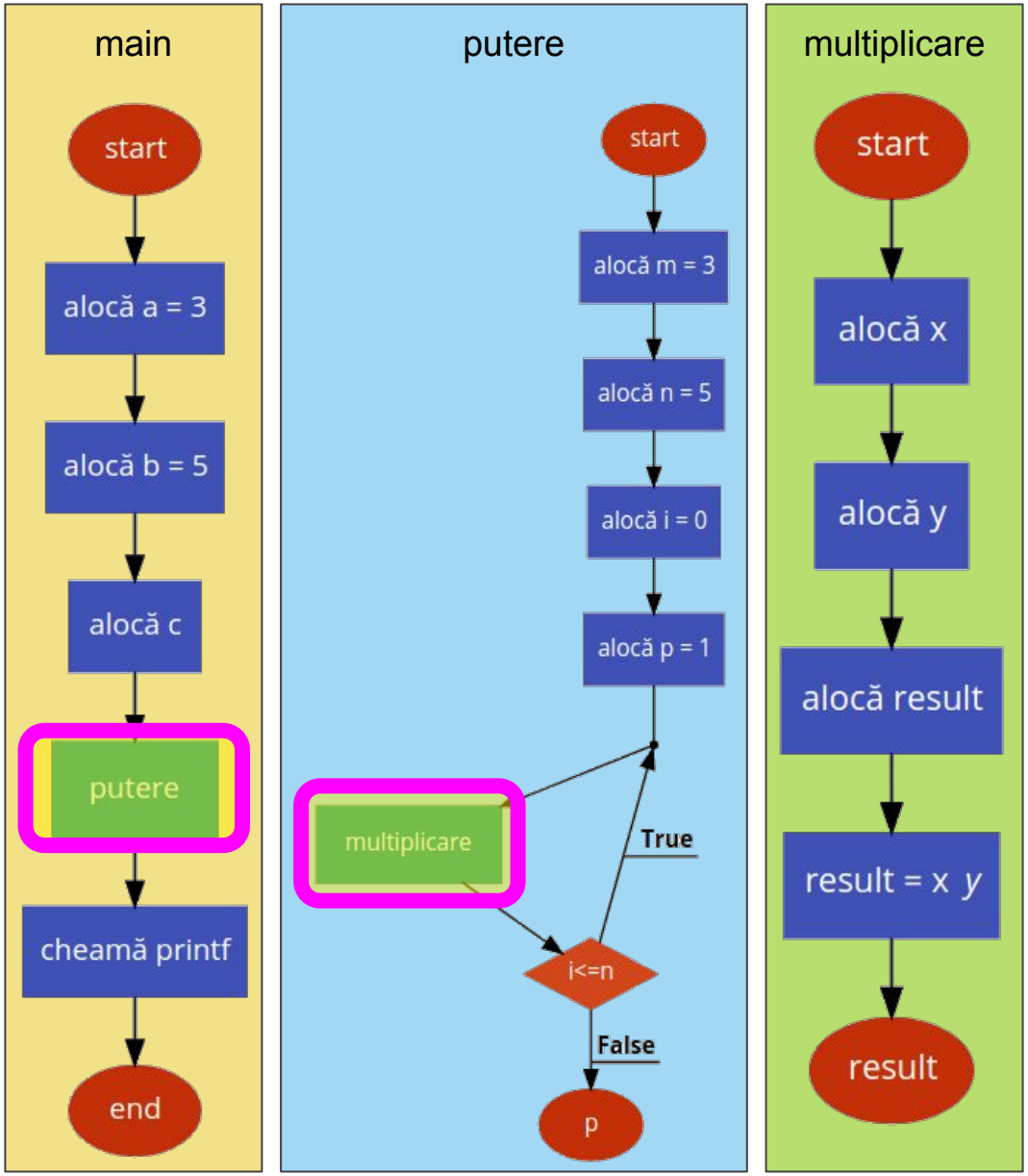


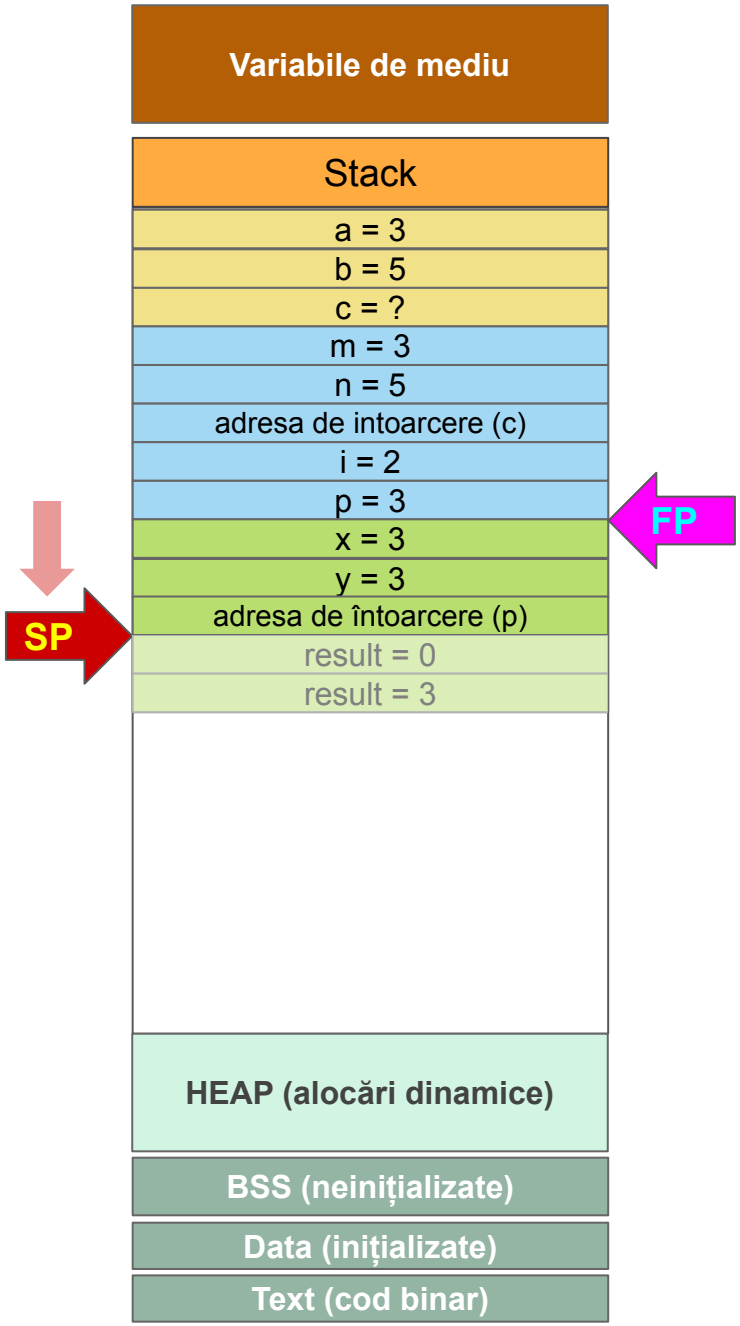




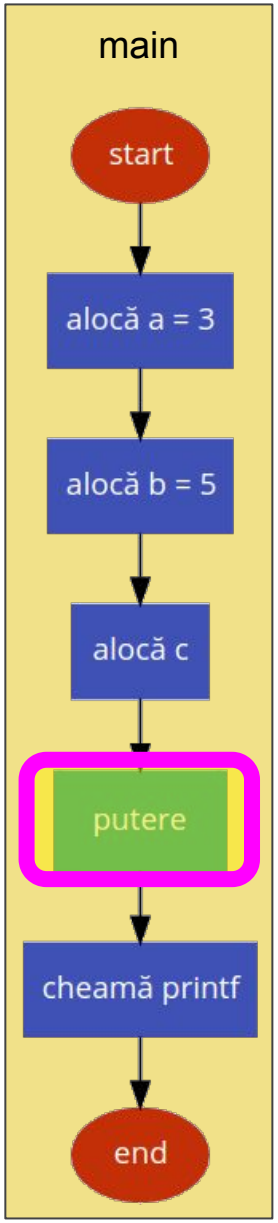


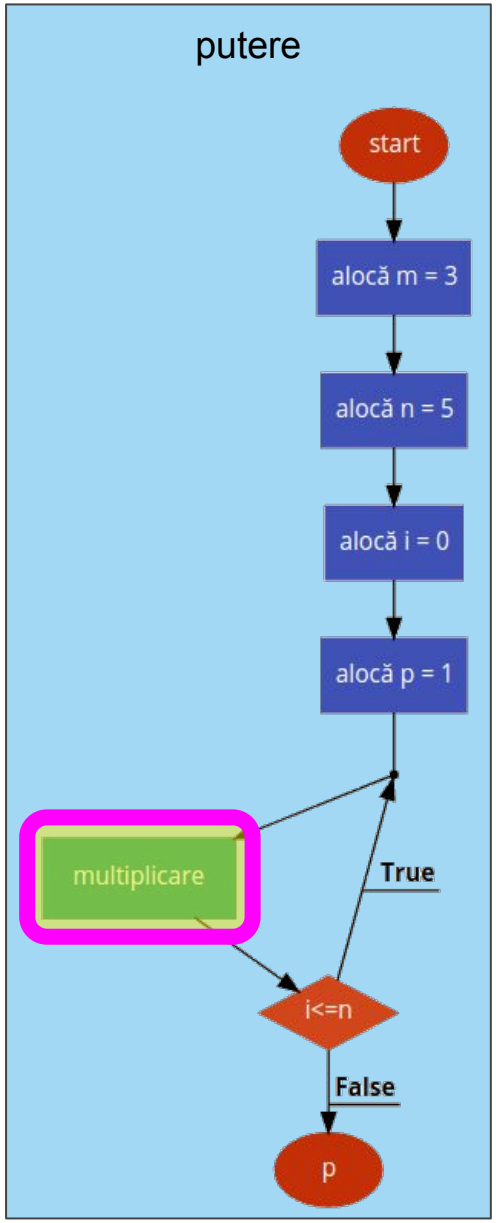
Viața unui program

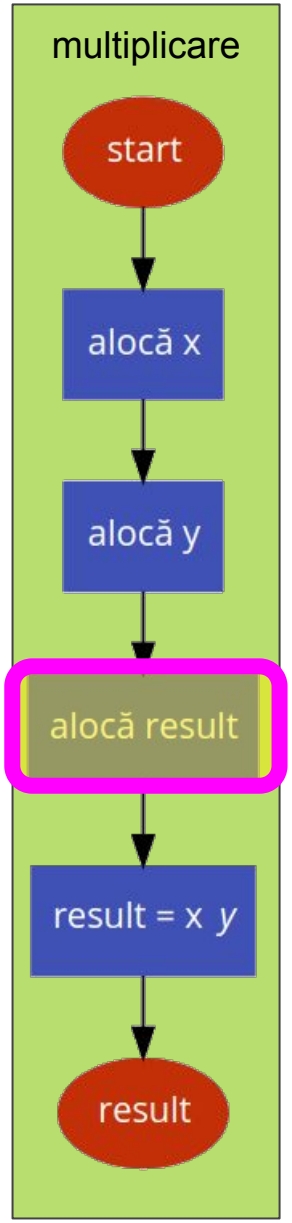


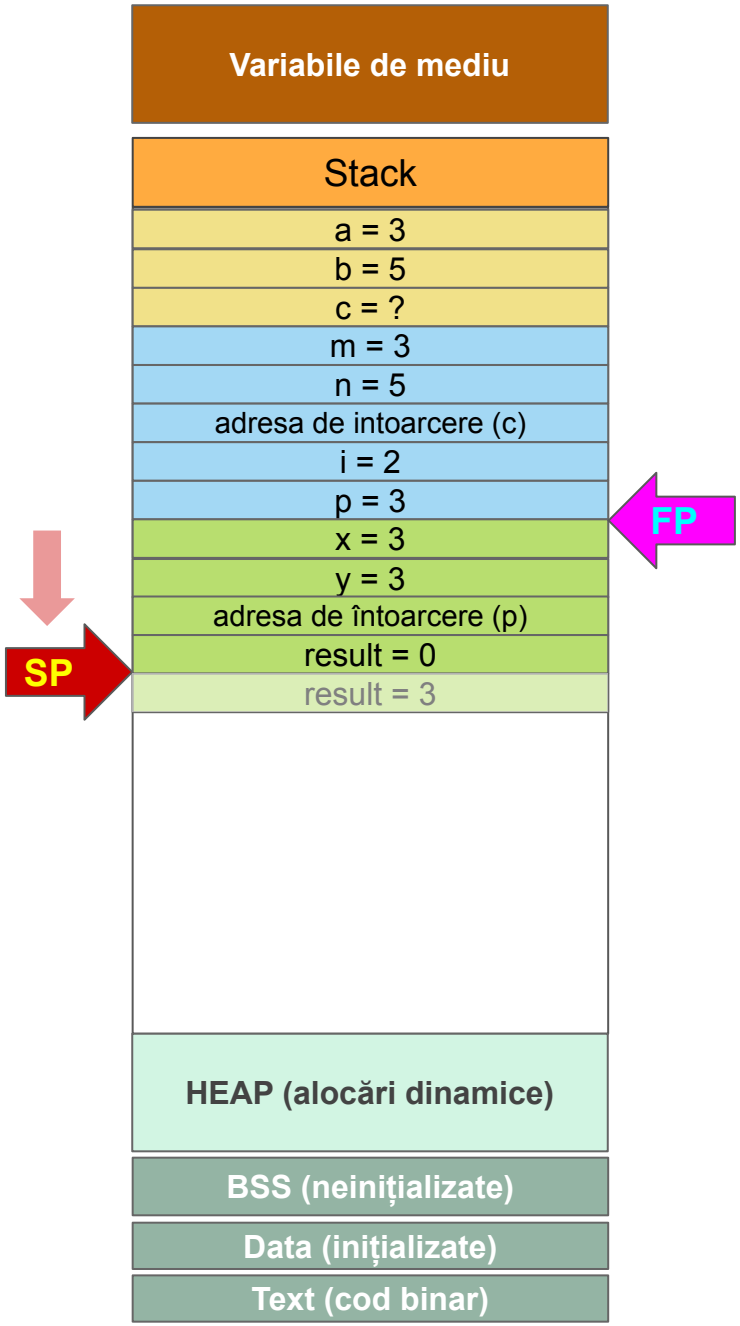


Viața unui program

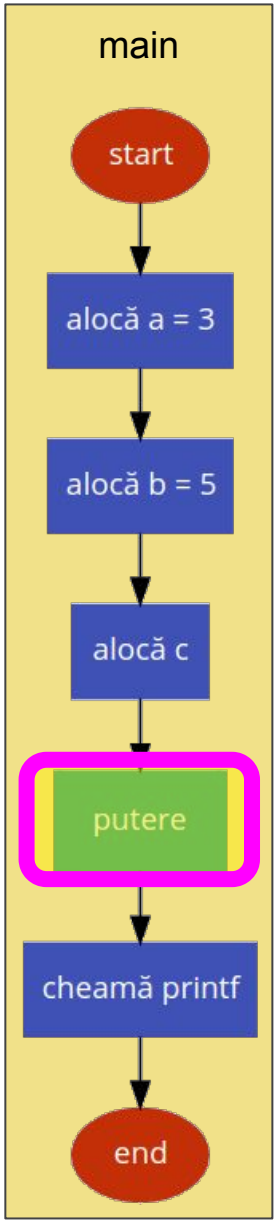


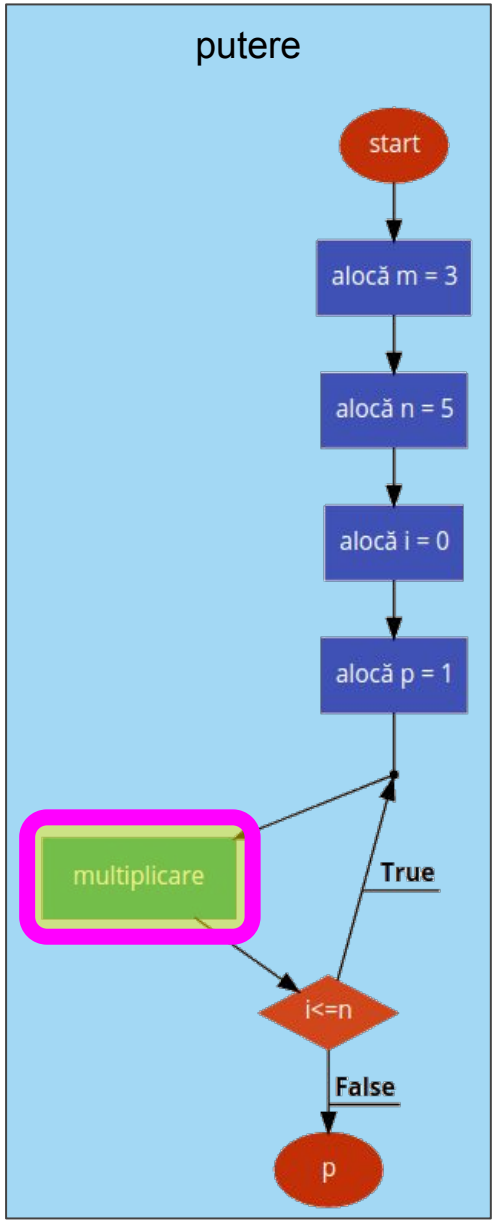


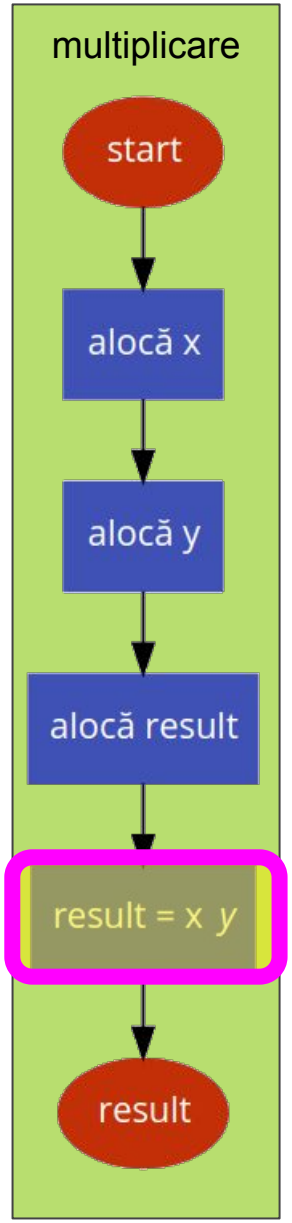


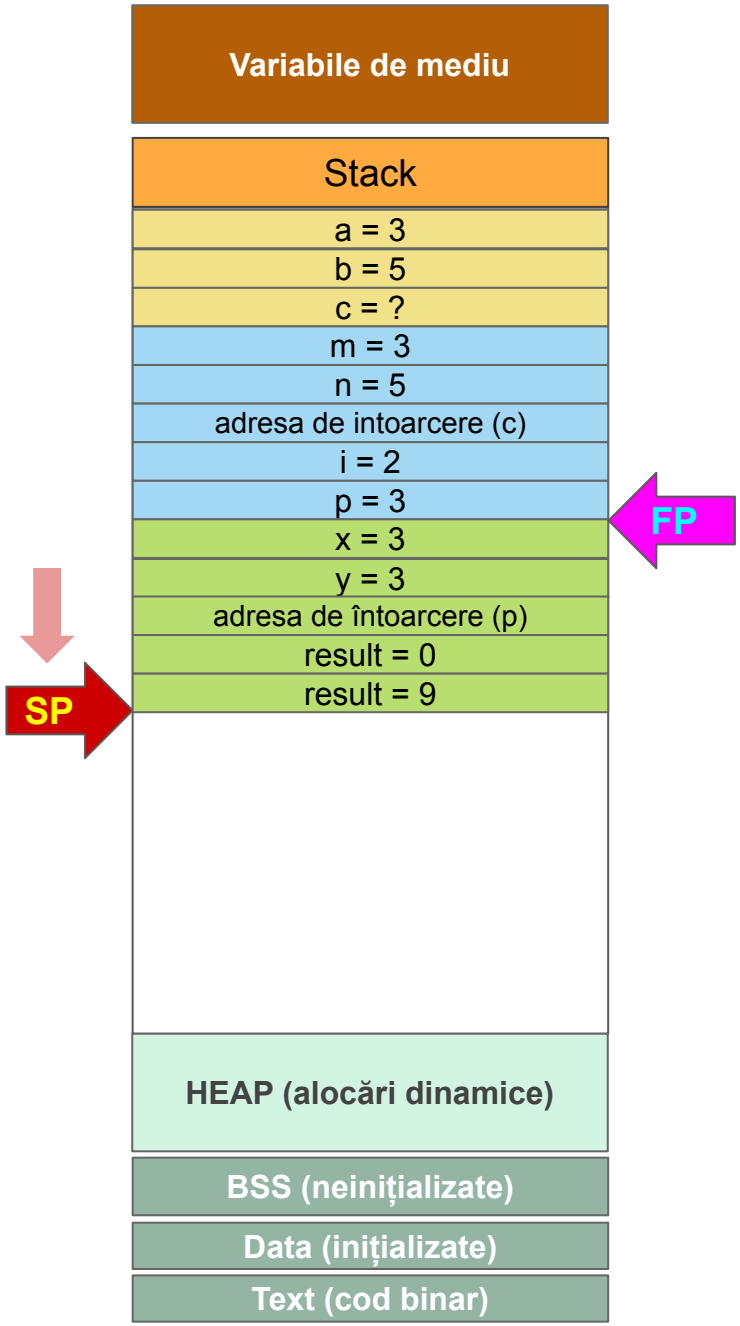


Viața unui program

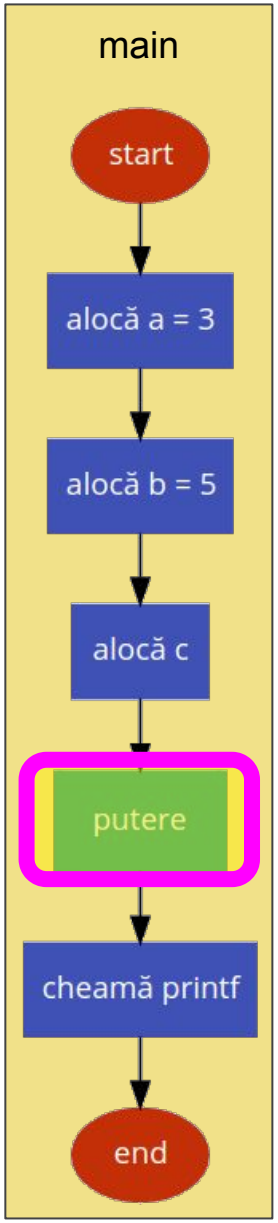


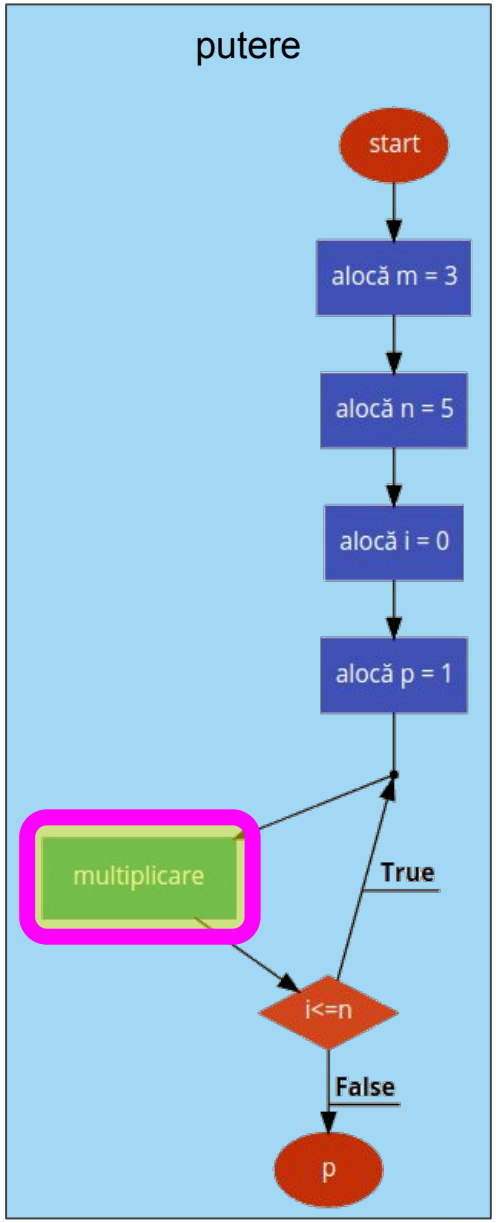


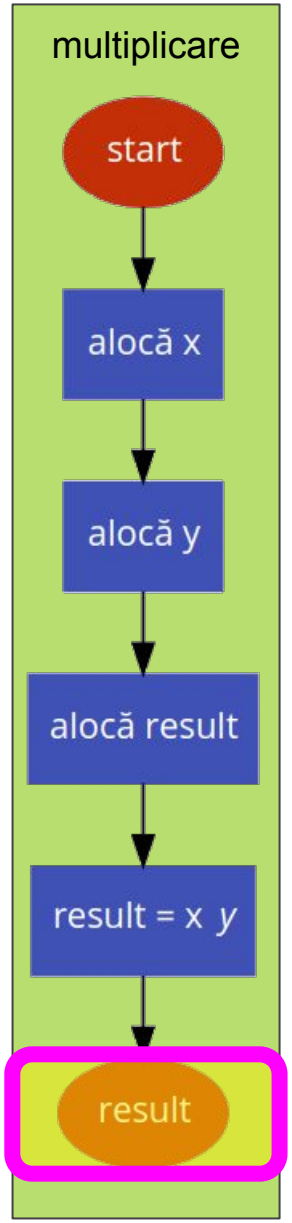


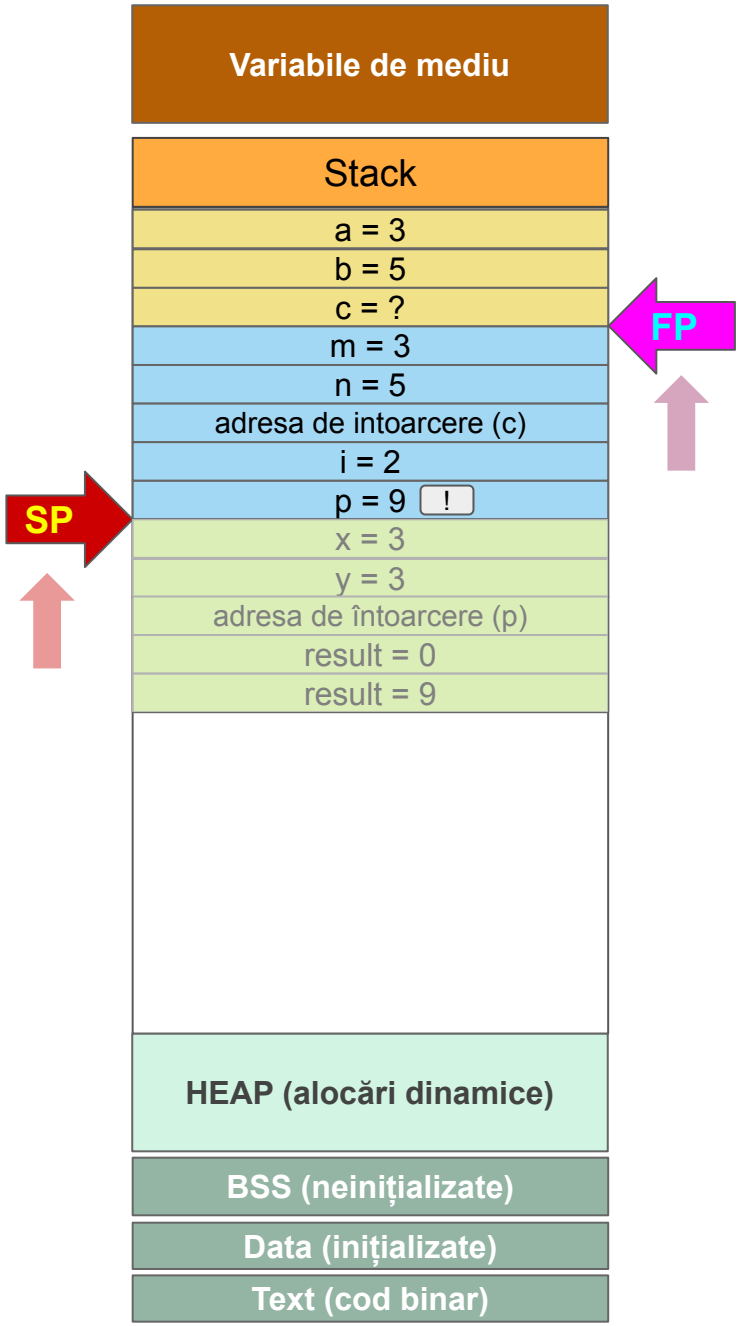


Viața unui program

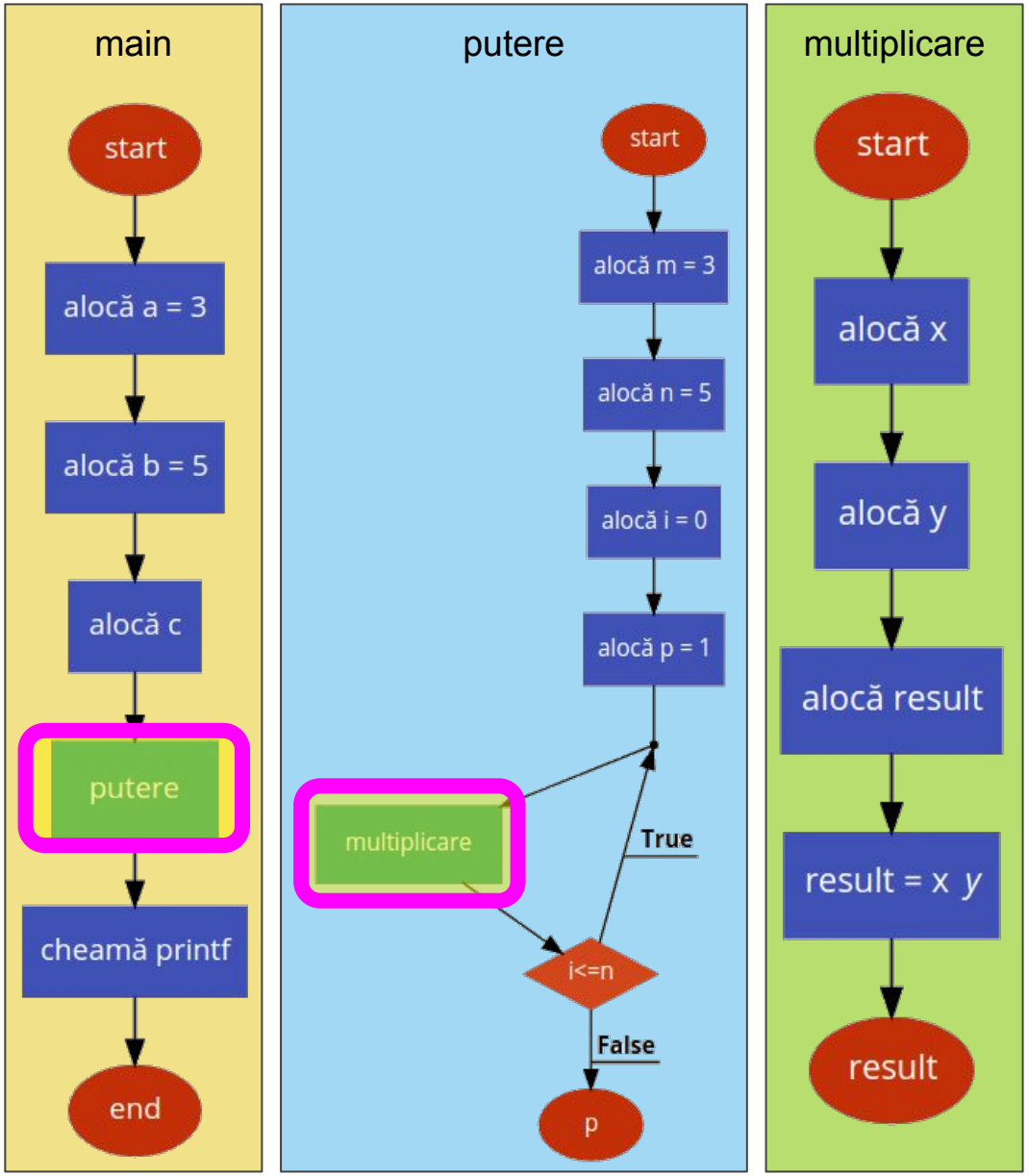


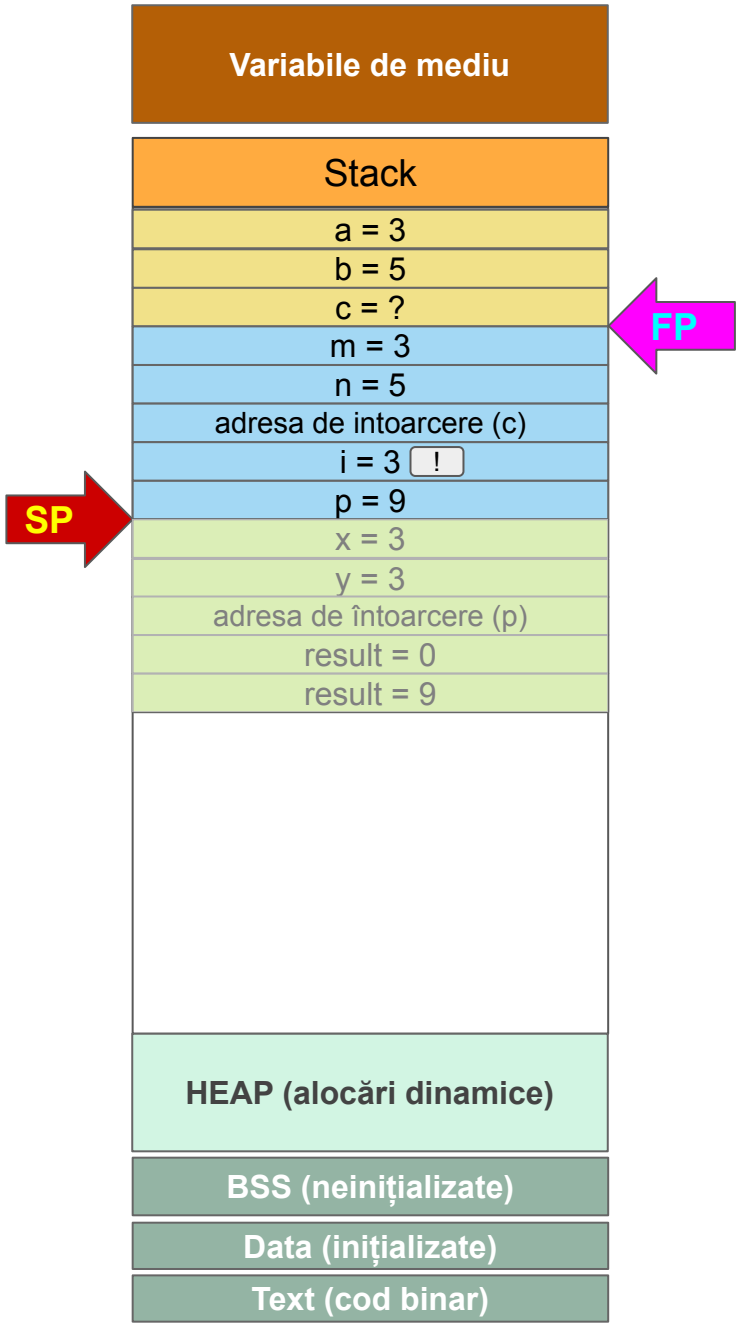






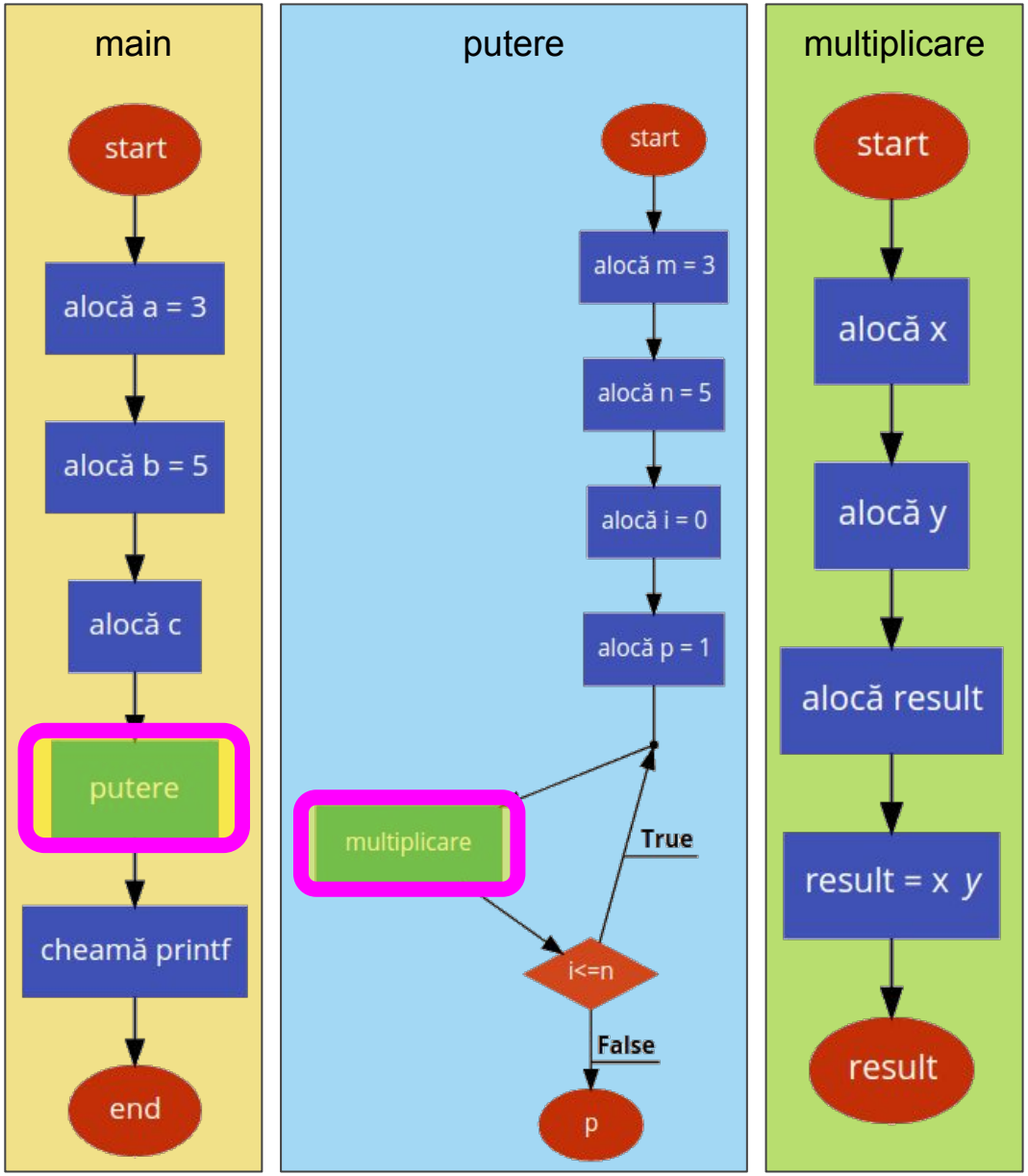
Viața unui program

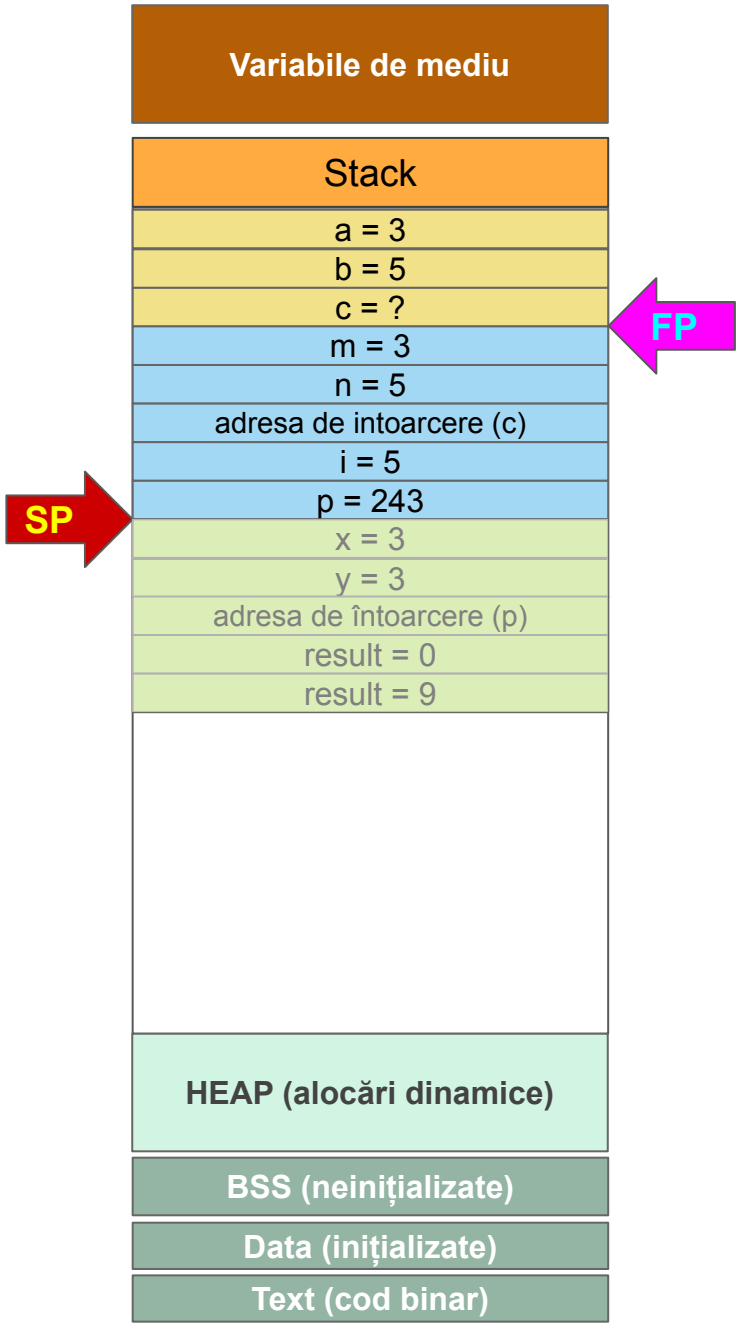




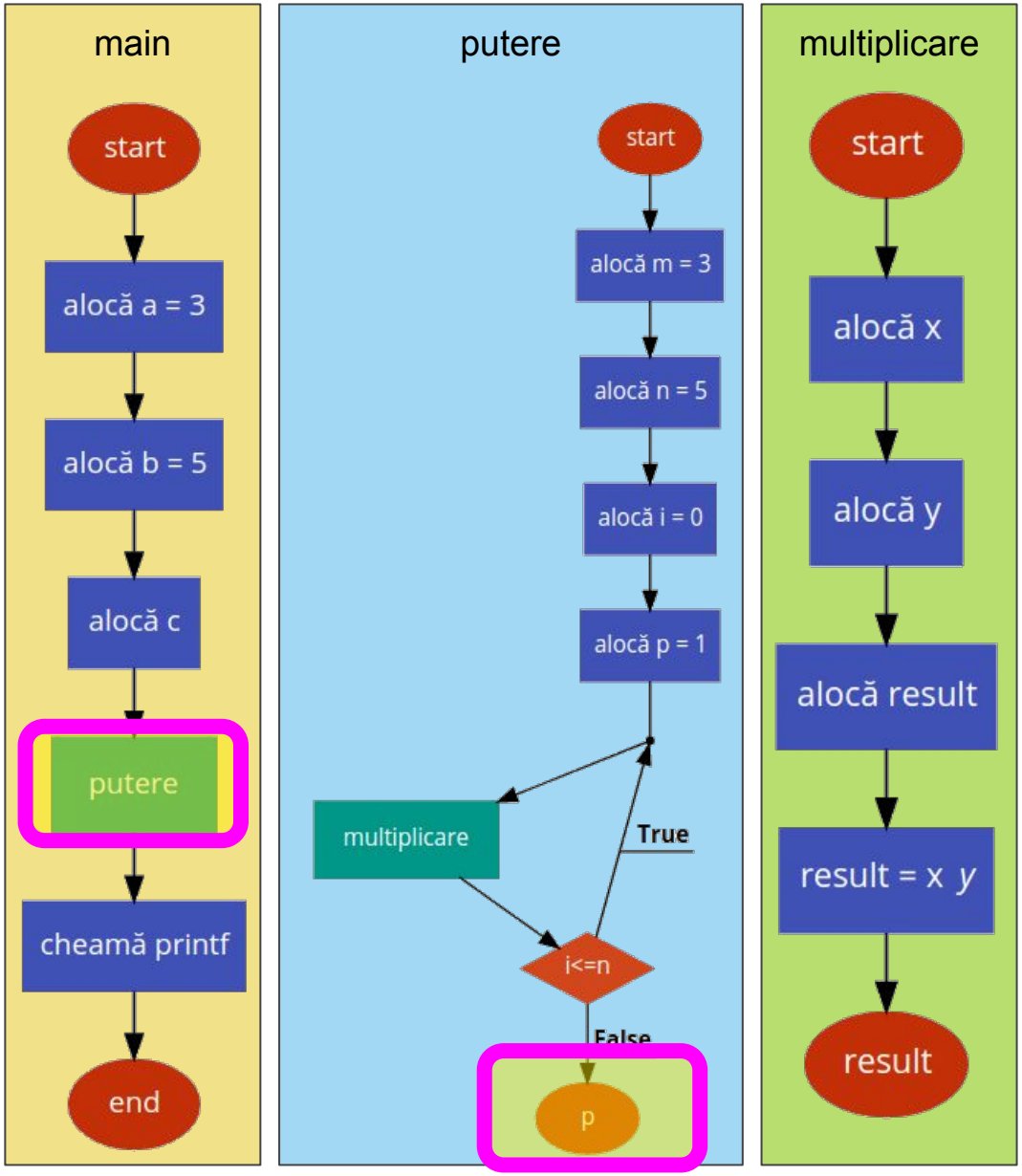
Programul continuă să ruleze până
când i devine egal cu n

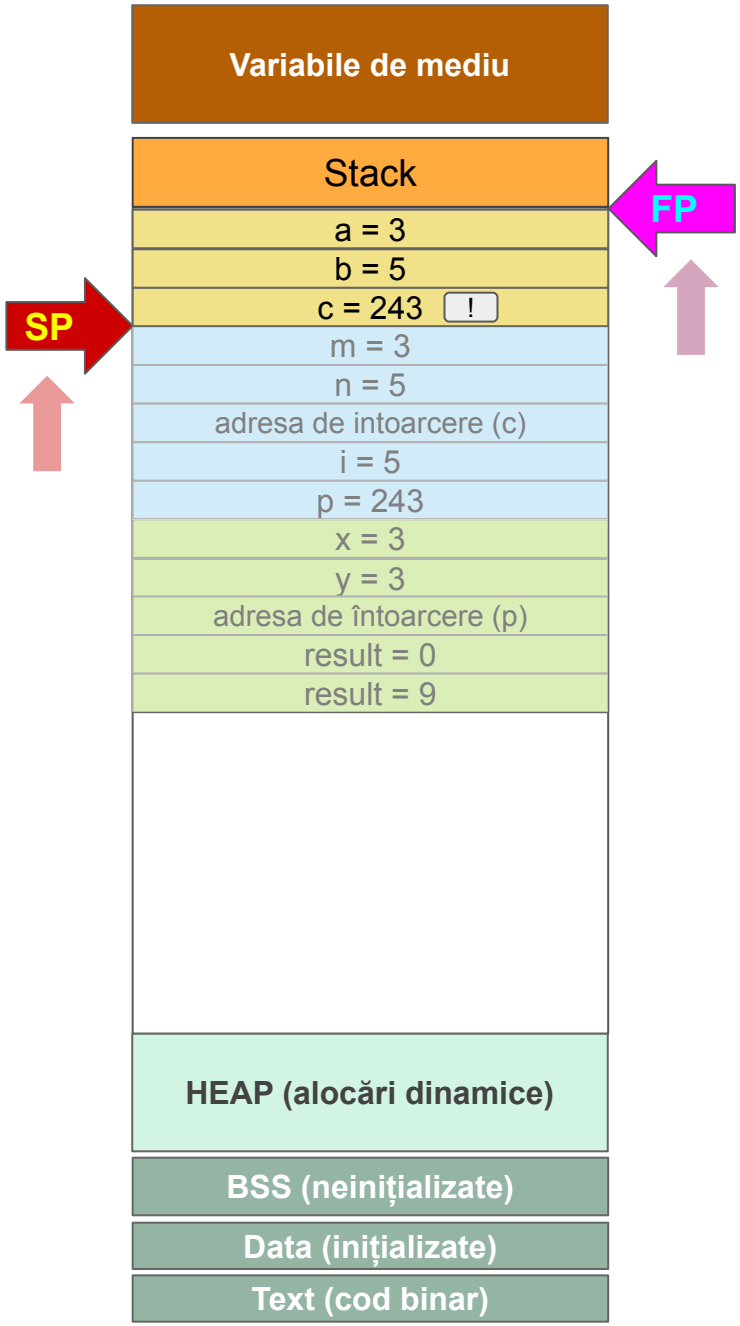
Viața unui program



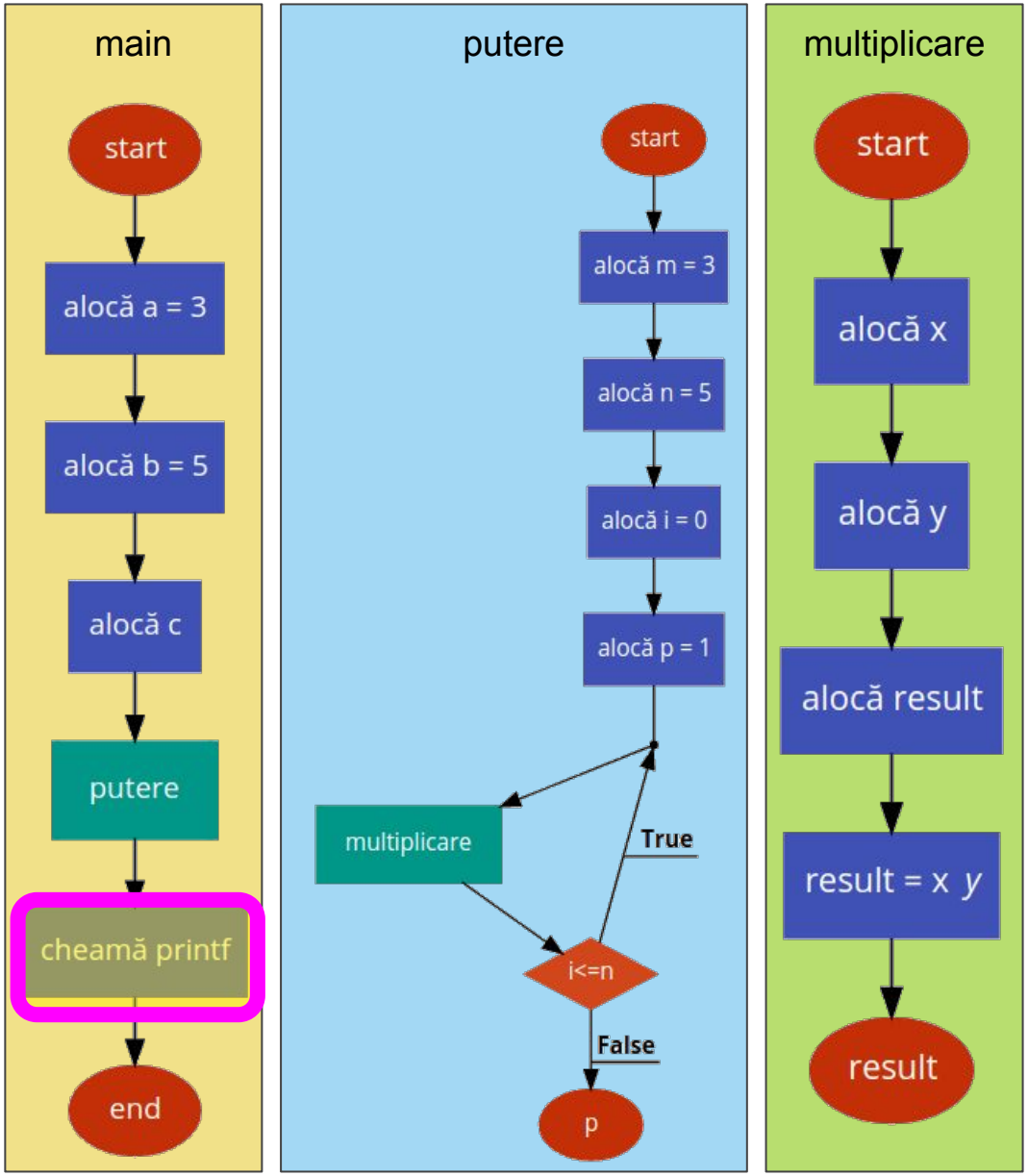


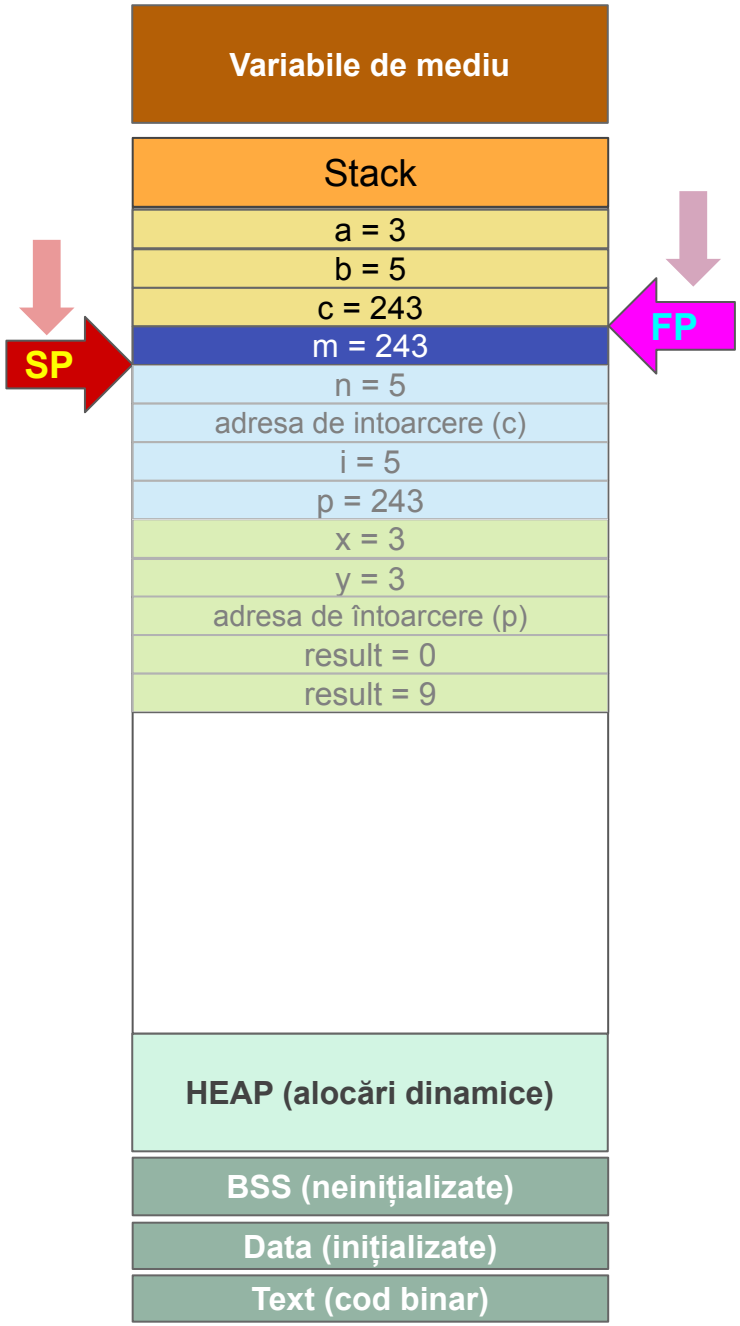
Viața unui program



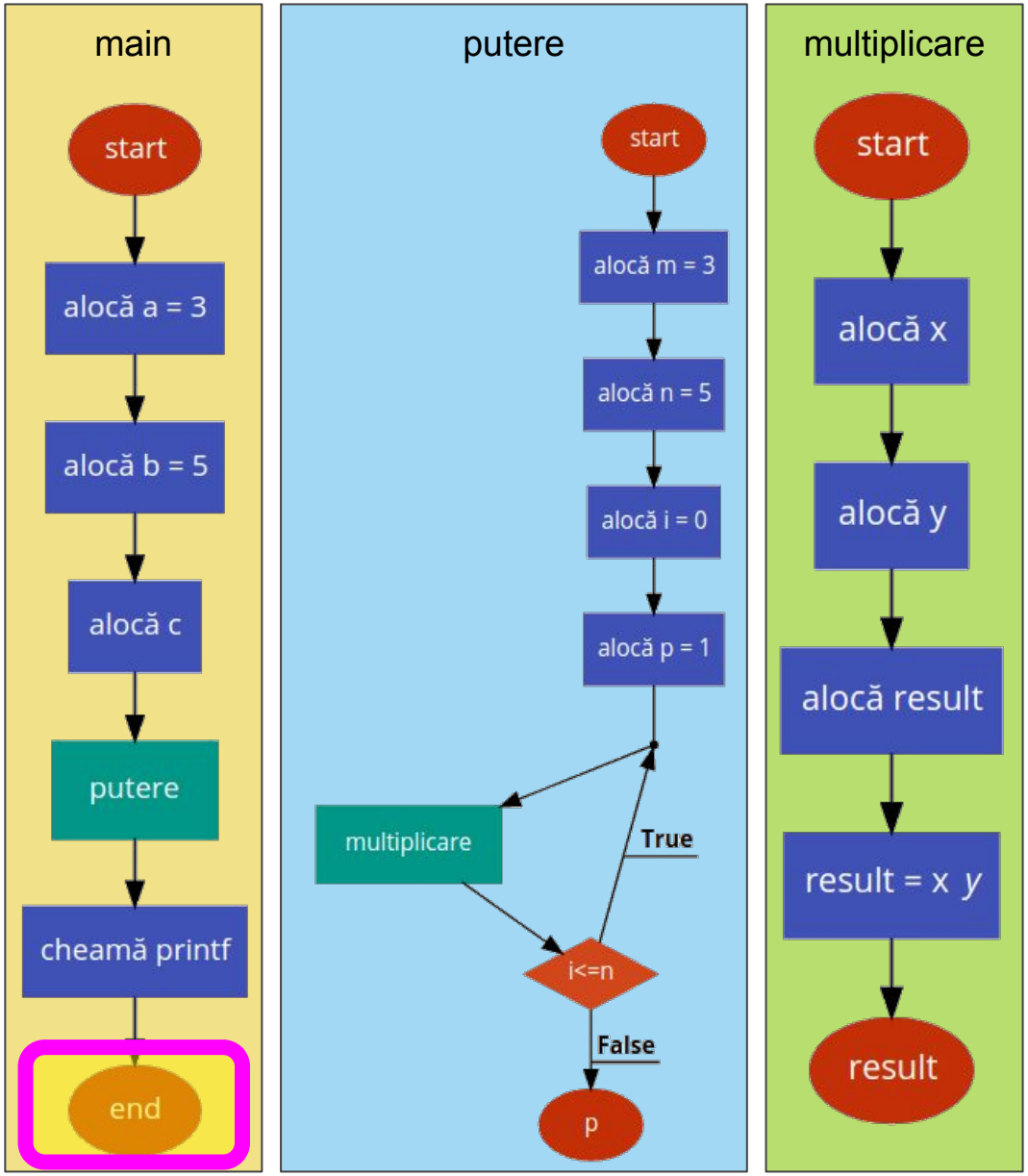


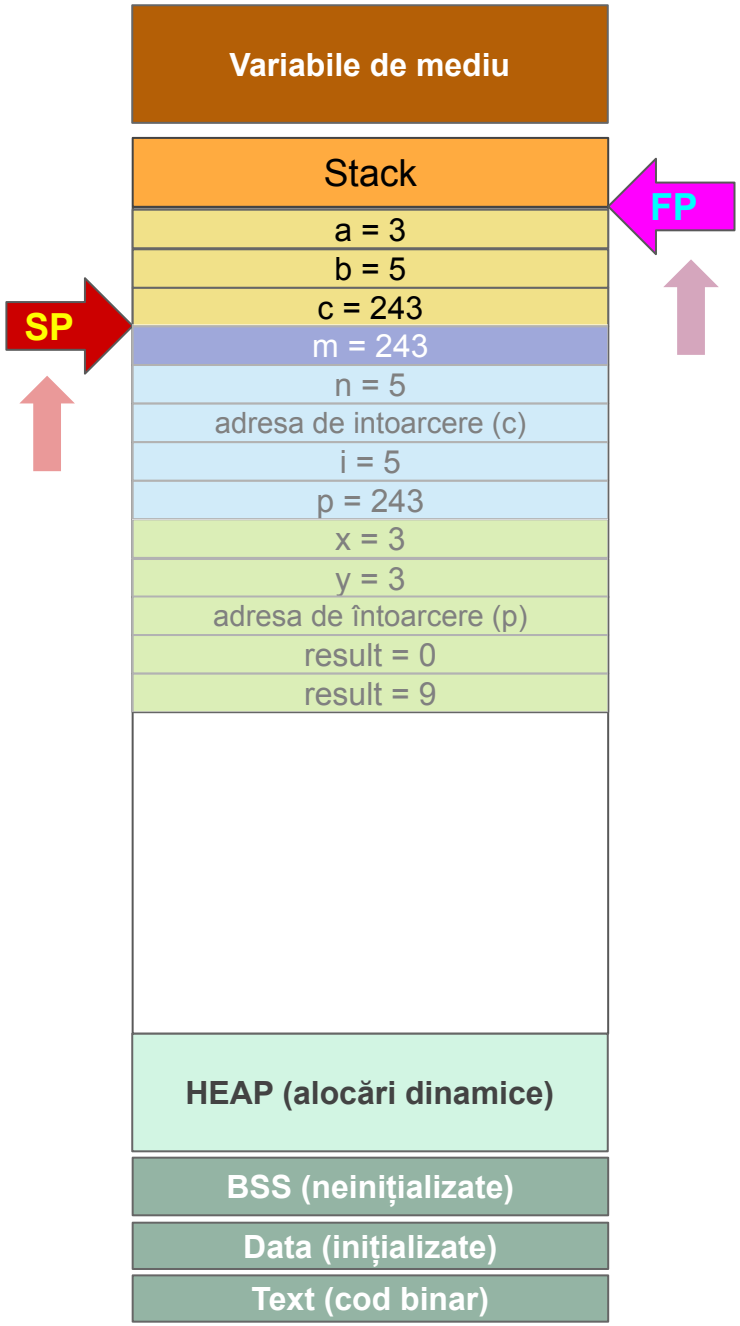
Viața unui program



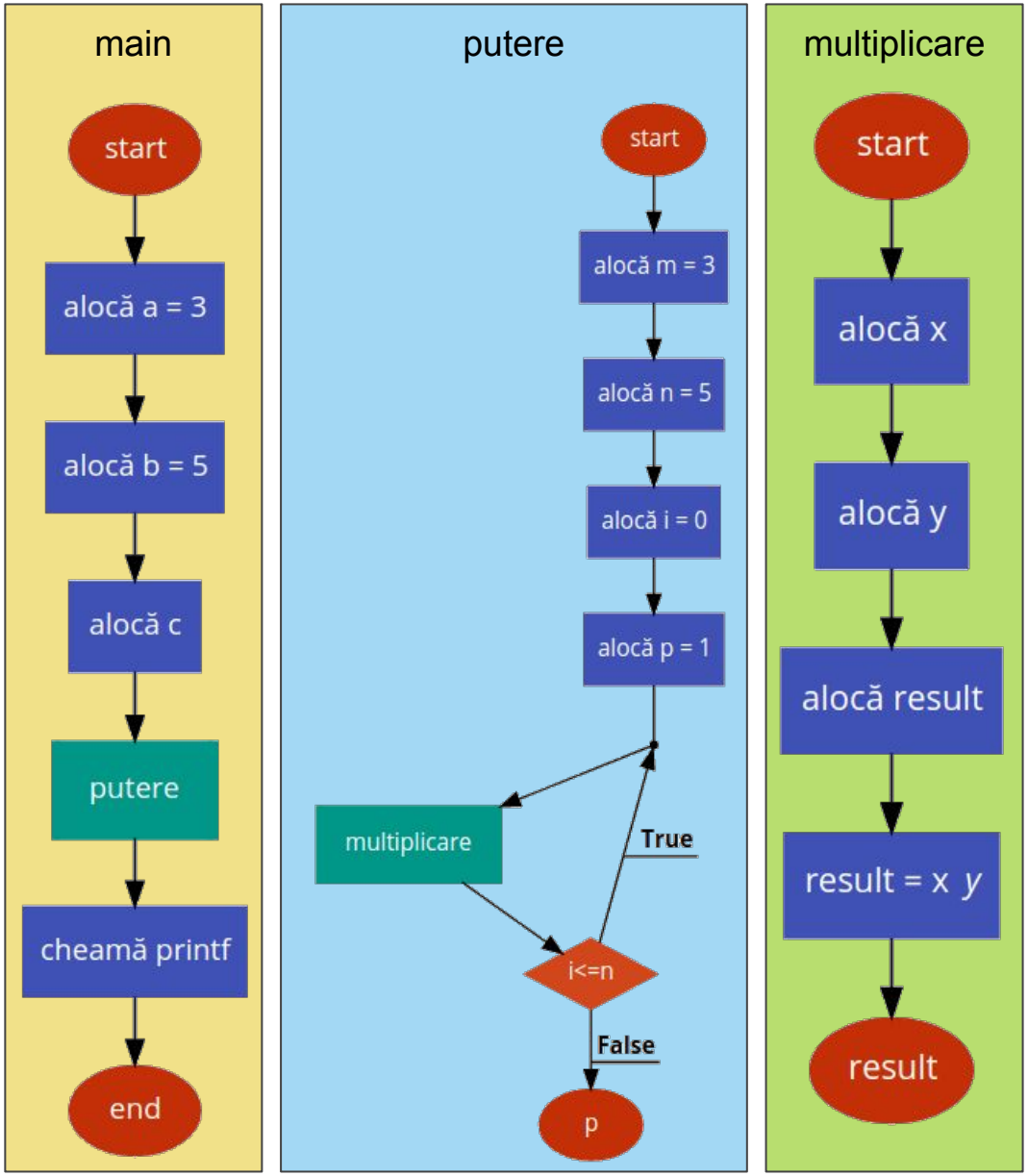


Viața unui program





Viața unui program



Ca de fiecare dată, compilatoarele moderne sunt mult mai complicate. Anumite subrutine pot să nu aibă un frame, variabilele vor fi păstrate în registrele procesorului - decizia este a compilatorului în funcție de numărul variabilelor neceare.

Frame pointerul poate fi de asemenea omis de către compilator (compilatoarele au opțiunea -fomit-frame-pointer, care face un cod cu foarte multe apeluri de subrutine puțin mai rapid. Citirea variabilelor se face în funcție de stack pointer.

De reținut că fomit-frame-pointer poate face debuggingul imposibil pe anumite platforme - debuggerele se bazează pe acesta pentru identificarea corectă a variabilelor in memorie.

Programarea structurată are suport hardware.

Programarea structurată are suport hardware. Altfel spus, setul de instrucțiuni x86 (si multe altele) constituie un procesor structural.

Unitatea de procesare este o instructiune (parte dintr-o secventă algoritmică), modul segmentat de adresare al memoriei este potrivit pentru acest gen de structură.

Procesoare “Orientate Obiect”

- Sunt compuse din mai multe unități (mai multe microprocesoare, fiecare cu scopul său)
- Permit adresarea memoriei sub forma unor pachete, Unitatea de management al memoriei este integrată în construcția procesorului.
- Au instrucțiuni specifice pentru accesarea segmentelor variabile de memorie
- Majoritatea implementărilor au fost eșecuri comerciale.

https://en.wikipedia.org/wiki/Intel_iAPX_432
https://en.wikipedia.org/wiki/Rekursiv
http://merlintec.com/lsi/tachyon/
https://www.researchgate.net/publication/224163056_An_object-oriented_processor_with_Just-In-Time_compilation_support/link/546d3e0f0cf26e95bc3cafd6/download

Tablouri de date

Șiruri in C

- Un șir în C este o colecție de date de același tip care sunt stocate în poziții de memorie succesive.
- Un șir poate conține **un singur tip** de variabile
- Un șir poate conține orice tip de variabilă, atâta timp cât t**oți membrii săi sunt la fel**.
- Dimensiunea șirului (numărul de variabile pe care le conține) trebuie **să fie cunoscută** la momentul definirii acestuia.
- Elementele din șir se pot accesa cu ajutorul indexului. sir[0] este primul element, sir[1] al doilea etc.
- Adresa șirului este adresa primului element al acestuia (elementul cu index 0).
- Șirurile pot fi traversate prin folosirea indexului.

```
for ( i = 0; i < N; i++ ) {
  printf("Valoarea lui %i: %d\n",i, val[i]);
}
```

Localizarea în memorie a membrilor unui șir

array_memory_char.c

```c
int main()
{
    int N = 10;
    char val[N];
    int i = 0;
    for ( i = 0; i < N; i++ ) {
      printf("Adresa %i: %p\n",i, &val[i]);
    }
    return 0;
}
```

https://onlinegdb.com/H1bNdTSa7

```
Adresa 0: 0x7ffdfd5e8610
Adresa 1: 0x7ffdfd5e8611
Adresa 2: 0x7ffdfd5e8612
Adresa 3: 0x7ffdfd5e8613
Adresa 4: 0x7ffdfd5e8614
Adresa 5: 0x7ffdfd5e8615
Adresa 6: 0x7ffdfd5e8616
Adresa 7: 0x7ffdfd5e8617
Adresa 8: 0x7ffdfd5e8618
Adresa 9: 0x7ffdfd5e8619
```

Diferența dintre două valori succesive: 1 octet

array_memory_short.c

```c
int main()
{
    int N = 10;
    short val[N];
    int i = 0;
    for ( i = 0; i < N; i++ ) {
      printf("Adresa %i: %p\n",i, &val[i]);
    }
    return 0;
}
```

https://onlinegdb.com/Syxia_TrTX

```
Adresa 0: 0x7ffecf392a50
Adresa 1: 0x7ffecf392a52
Adresa 2: 0x7ffecf392a54
Adresa 3: 0x7ffecf392a56
Adresa 4: 0x7ffecf392a58
Adresa 5: 0x7ffecf392a5a
Adresa 6: 0x7ffecf392a5c
Adresa 7: 0x7ffecf392a5e
Adresa 8: 0x7ffecf392a60
Adresa 9: 0x7ffecf392a62
```

Diferența dintre două valori succesive: 2 octeți

În funcție de dimensiunea membrilor șirului, locația de memorie a fiecărui membru arată continuitatea lor în memorie.

Localizarea în memorie a membrilor unui șir

array_memory_int.c

```c
int main()
{
    int N = 10;
    int val[N];
    int i = 0;
    for ( i = 0; i < N; i++ ) {
     printf("Adresa %i: %p\n",i, &val[i]);
    }
    return 0;
}
```

https://onlinegdb.com/rymOP6rTQ

```
Adresa 0: 0x7ffc458a2a70
Adresa 1: 0x7ffc458a2a74
Adresa 2: 0x7ffc458a2a78
Adresa 3: 0x7ffc458a2a7c
Adresa 4: 0x7ffc458a2a80
Adresa 5: 0x7ffc458a2a84
Adresa 6: 0x7ffc458a2a88
Adresa 7: 0x7ffc458a2a8c
Adresa 8: 0x7ffc458a2a90
Adresa 9: 0x7ffc458a2a94
```

Diferența dintre două valori succesive: 4 octeți

array_memory_long.c

```c
int main()
{
    int N = 10;
    long val[N];
    int i = 0;
    for ( i = 0; i < N; i++ ) {
     printf("Adresa %i: %p\n",i, &val[i]);
    }
    return 0;
}
```

https://onlinegdb.com/SJrQP6BaQ

```
Adresa 0: 0x7fff5161c040
Adresa 1: 0x7fff5161c048
Adresa 2: 0x7fff5161c050
Adresa 3: 0x7fff5161c058
Adresa 4: 0x7fff5161c060
Adresa 5: 0x7fff5161c068
Adresa 6: 0x7fff5161c070
Adresa 7: 0x7fff5161c078
Adresa 8: 0x7fff5161c080
Adresa 9: 0x7fff5161c088
```

Diferența dintre două valori succesive: 8 octeți

În funcție de dimensiunea membrilor șirului, locația de memorie a fiecărui membru arată continuitatea lor în memorie.

Inițializarea șirurilor

array_init.c

```c
int main()
{
    int N = 10;
    int a[] = { 0 };
    int b[5] = { 0 };
    int c[] = {};
    int d[5] = {};
    int e[] = { 5, 5, 5, 5, 5 };
    int f[5] = { 5, 5, 5};
    int g[N];

    printf ("a: %ld\n", sizeof(a)/sizeof(a[0]));
    printf ("b: %ld\n", sizeof(b)/sizeof(b[0]));
    printf ("c: %ld\n", sizeof(c)/sizeof(c[0]));
    printf ("d: %ld\n", sizeof(d)/sizeof(d[0]));
    printf ("e: %ld\n", sizeof(e)/sizeof(e[0]));
    printf ("f: %ld\n", sizeof(f)/sizeof(f[0]));
    printf ("g: %ld\n", sizeof(g)/sizeof(g[0]));
    return 0;
}
```

https://onlinegdb.com/Hk_Aw8DaQ

```
a: 1
b: 5
c: 0
d: 5
e: 5
f: 5
g: 10
```

Un sir cu un singur membru, care este 0.

Declararea unui șir cu 5 elemente inițializate la 0.

Declarare fără dimensiune și fără inițializare. Nu există nici o sursă de informații despre dimensiunea șirului, va fi construit un șir cu 0 elemente

Declarare cu inițializare completă, fără dimensiune.

Declarare cu inițializare parțială (celelalte elemente vor fi 0).

Declarare șir cu dimensiune variabilă fără inițializare.

Inițializarea șirurilor

Compilatorul nu poate inițializa șiruri care au dimensiuni variabile.

```
#include <stdio.h>

int main()
{
   int N = 10;
   int g[N] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
   printf ("g: %ld\n", sizeof(g)/sizeof(g[0]));
   return 0;
}
```

```
array_init_wr.c: In function 'main':
array_init_wr.c:8:5: error: variable-sized object may not be initialized
    int g[N] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
    ^
```

```
#include <stdio.h>
#define N 10

int main()
{
   int g[N] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
   printf ("g: %ld\n", sizeof(g)/sizeof(g[0]));
   return 0;
}
```

```
a: 1
b: 5
c: 0
d: 5
e: 5
f: 5
g: 10
```

Ce se întâmplă dacă nu știm dimensiunea?

- Definim șirul suficient de mare pentru a fi acoperitor în toate cazurile.
- Utilizăm alocarea dinamică a memoriei și pointeri.

array_large.c

```c
#include<stdio.h>

#define NR_S 200 /*dimensiune suficient de mare a sirului*/

int main()
{
   int note[NR_S],max,i,studenti,suma=0;
   printf("Numarul de studenti (max %d): ", NR_S);
   scanf("%d", &studenti);
   printf("Introduceti notele\n");
   for(i=0; i < studenti; i++)
   {
       scanf("%d", &note[i]);
   }
   max = 0;
   for(i=0; i< studenti; i++)
   {
      suma = suma + note[i];
      if (note[i] > max)
      {
         max = note[i];
      }
   }
   printf("Cea mai mare nota este %d\n", max);
   printf("Suma este %d\n", suma);
   printf("Media este %f\n", (float)suma/studenti);
   return 0;
}
```

https://onlinegdb.com/HyhlrTBam

```
Numarul de studenti (max 200): 10
Introduceti notele
8
7
6
9
10
10
7
8
9
10
Cea mai mare nota este 10
Suma este 84
Media este 8.400000
```

Trimiterea șirurilor ca parametri ai funcțiilor

array_function_declare_empty.c

```c
#include <stdio.h>


void printint(int st[])
{
   printf ("Lungime in functie : %ld\n", sizeof(st)/sizeof(st[0]));
}

int main()
{
   int c[14] = {};
   printf ("Lungime in main: %ld\n", sizeof(c)/sizeof(c[0]));
   printint(c);
   return 0;
}
```

https://onlinegdb.com/0g0Nfa04a

```
Lungime in main: 14
Lungime in functie : 2
```

?

În sistemele pe 64 de biți un pointer are 8 octeți iar un int are 4. De aceea în funcția printint valoarea rezultată este 2

Chiar dacă pare că trimitem șirul către funcție, în realitate compilatorul înlocuiește șirul cu adresa sa de memorie (pointer). În interiorul funcției obținem dimensiunea pointerului, nu a șirului. De aceea, la transmiterea șirurilor către funcții trebuie trimisă și lungimea acestora.

Trimiterea șirurilor ca parametri ai funcțiilor

array_function_declare_size.c

```
void printint(int st[14])
{
   printf ("Lungime in functie : %ld\n", sizeof(st)/sizeof(st[0]));
}

int main()
{
   int c[14] = {};
   printf ("Lungime in main: %ld\n", sizeof(c)/sizeof(c[0]));
   printint(c);
   return 0;
}
```

https://onlinegdb.com/r1XrNarpm

Parametrul se poate declara cu dimensiunea predefinita, rezultatul e același.

```
Lungime in main: 14
Lungime in functie : 2
```

array_function_declare_pointer.c

```
void printint(int *st)
{
   printf ("Lungime in functie : %ld\n", sizeof(st)/sizeof(st[0]));
}

int main()
{
   int c[14] = {};
   printf ("Lungime in main: %ld\n", sizeof(c)/sizeof(c[0]));
   printint(c);
   return 0;
}
```

https://onlinegdb.com/BJcjEaB6m

Parametrul se poate declara ca pointer, rezultatul e același.

```
Lungime in main: 14
Lungime in functie : 2
```

Trimiterea șirurilor ca parametri ai funcțiilor

array_function_pass_size.c

```c
#include <stdio.h>

void printint(int st[], int size)
{
  int i;
  printf ("Lungime in functie : %ld\n", sizeof(st)/sizeof(st[0]));
  for (i = 0; i < size; i++ ) {
      printf("nr: \%i este %i\n",i, st[i]);
  }
}

int main()
{
  int c[14] = {1,2,3,4,5,6};
  printint(c,10);
  return 0;
}
```

https://onlinegdb.com/S18CQ6rpQ

```
Lungime in functie : 2
nr: 0 este 1
nr: 1 este 2
nr: 2 este 3
nr: 3 este 4
nr: 4 este 5
nr: 5 este 6
nr: 6 este 7
```

Singura variantă pentru a putea enumera șirul în interiorul unei funcții este să trimitem și dimensiunea sa ca parametru suplimentar.

Șiruri multidimensionale (tabele, matrici)

C permite definirea șirurilor cu mai multe dimensiuni. Numărul maxim de dimensiuni este determinat de mărimea tipului de variabilă, de arhitectura sistemului pe care se compilează programul și de alți factori.

```
type name[size1][size2]...[sizeN];
```

```
char arr[3][5] = {};
```

	Coloană 1	Coloană 2	Coloană 3	Coloană 4	Coloană 5
Rând 1	arr[0][0]	arr[0][1]	arr[0][2]	arr[0][3]	arr[0][4]
Rând 2	arr[1][0]	arr[1][1]	arr[1][2]	arr[1][3]	arr[1][4]
Rând 3	arr[2][0]	arr[2][1]	arr[2][2]	arr[2][3]	arr[2][4]

Șiruri multidimensionale (tabele, matrici)

multi.c

```c
#include<stdio.h>

int main()
{

   char arr[7][20] = {};
   int i,j;

   arr[0][15] = 94;
   arr[0][16] = 94;
   arr[1][13] = 94;
   . . . . .
   arr[6][18] = 126;
   arr[6][19] = 94;

   for(i=0; i<=6;i++) {
       for (j=0; j<20; j++) {
            if (arr[i][j] > 0) {
                printf("%c",arr[i][j]);
            } else {
                printf(" ");
            }
       }
       printf("\n");
   }

return 0;
}
```

https://onlinegdb.com/BJjlv8Dpm

```
                  ^^
                ^^
               ^^
   _______/LLLL\__
   \_____________"|
^~^~^~^~^~^~^~^~^~^~^~^~
~^~^~^~^~^~^~^~^~^~^~^~^
```

“Vizitarea” tuturor membrilor unei matrici se numește și traversarea matricii.

Traversarea unei matrici se face prin utilizarea buclelor repetitive.

În interiorul buclelor se plasează de obicei structuri decizionale.

Stocarea în memorie a matricilor

```
int arr[4][3] = {{2,3,5},{6,7,4},{3,9,11},{0,17,10}};
```

arr	0	1	2
0	2	3	5
1	6	7	4
2	3	9	11
3	0	17	10

```
arr[0][0] = 2
arr[0][1] = 3
arr[0][2] = 5
arr[1][0] = 6
arr[1][1] = 7
arr[1][2] = 4
arr[2][0] = 3
arr[2][1] = 9
arr[2][2] = 11
arr[3][0] = 0
arr[3][1] = 17
arr[3][2] = 10
```

```
arr[0][0] = 2
arr[0][1] = 3
arr[0][2] = 5
arr[0][3] = 6
arr[0][4] = 7
arr[0][5] = 4
arr[0][6] = 3
arr[0][7] = 9
arr[0][8] = 11
arr[0][9] = 0
arr[0][10] = 17
arr[0][11] = 10
```

Surprinzător, dacă tratăm matricea ca pe un șir unidimensional, rezultatele sunt aceleași. În realitate, matricile multidimensionale sunt stocate în memorie tot ca șiruri unidimensionale, cu rândurile stocate unul după celălalt.

	0	1	2	3	4	5	6	7	8	9	10	11
0	2	3	5	6	7	4	3	9	11	0	17	10

De aceea, la inițializarea matricilor nu e nevoie de specificarea primei dimensiuni. Declarația de mai sus este echivalentă cu:

```
int arr[][3] = {{2,3,5},{6,7,4},{3,9,11},{0,17,10}};
```

Stocarea în memorie a matricilor

multi_access.c

```c
#include <stdio.h>

int main()
{
   int arr[4][3] =
{{2,3,5},{6,7,4},{3,9,11},{0,17,10}};
   int i;
   int j;

   printf("---Acces multidimensional----\n");

   for(i=0; i<4;i++) {
       for (j=0; j<3; j++) {
         printf("arr[%i][%i] = %d\n",i,j,arr[i][j]);
       }
   }

   printf("\n--Acces unidimensional-----\n");

   for(i=0; i<12;i++) {
        printf("arr[0][%i] = %d\n",i,arr[0][i]);
   }

   return 0;
}
```

https://onlinegdb.com/B1r4118Tm

```
---Acces multidimensional----
arr[0][0] = 2
arr[0][1] = 3
arr[0][2] = 5
arr[1][0] = 6
arr[1][1] = 7
arr[1][2] = 4
arr[2][0] = 3
arr[2][1] = 9
arr[2][2] = 11
arr[3][0] = 0
arr[3][1] = 17
arr[3][2] = 10

---Acces unidimensional---
arr[0][0] = 2
arr[0][1] = 3
arr[0][2] = 5
arr[0][3] = 6
arr[0][4] = 7
arr[0][5] = 4
arr[0][6] = 3
arr[0][7] = 9
arr[0][8] = 11
arr[0][9] = 0
arr[0][10] = 17
arr[0][11] = 10
```

Stocarea în memorie a matricilor

array_multi_uni.c

```
#include <stdio.h>

int main()
{
   int arr[2][4][3] = {
    { {2,3,5}, {6,7,4},{3,9,11},{0,17,10} },
    { {22,23,25}, {26,27,24}, {23,29,211},
{20,217,210} }
   };
   int i,j,k;
   printf("---Acces multidimensional-------\n");
   for(i=0; i<2;i++) {
       for (j=0; j<4; j++) {
           for (k=0; k<3; k++) {
               printf("arr[%i][%i][%i] = %d\n",
i,j,k,arr[i][j][k]);
           }
       }
   }
   printf("\n---Acces unidimensional-------\n");
   for(i=0; i<24;i++) {
        printf("arr[0][0][%i] = %d\n",
i,arr[0][0][i]);
   }
   return 0;
}
```

https://onlinegdb.com/ByM0U5Ya7

```
arr[0][0][0] = 2
arr[0][0][1] = 3
arr[0][0][2] = 5
arr[0][1][0] = 6
arr[0][1][1] = 7
arr[0][1][2] = 4
arr[0][2][0] = 3
arr[0][2][1] = 9
arr[0][2][2] = 11
arr[0][3][0] = 0
arr[0][3][1] = 17
arr[0][3][2] = 10
arr[1][0][0] = 22
arr[1][0][1] = 23
arr[1][0][2] = 25
arr[1][1][0] = 26
arr[1][1][1] = 27
arr[1][1][2] = 24
arr[1][2][0] = 23
arr[1][2][1] = 29
arr[1][2][2] = 211
arr[1][3][0] = 20
arr[1][3][1] = 217
arr[1][3][2] = 210
```

```
arr[0][0][0] = 2
arr[0][0][1] = 3
arr[0][0][2] = 5
arr[0][0][3] = 6
arr[0][0][4] = 7
arr[0][0][5] = 4
arr[0][0][6] = 3
arr[0][0][7] = 9
arr[0][0][8] = 11
arr[0][0][9] = 0
arr[0][0][10] = 17
arr[0][0][11] = 10
arr[0][0][12] = 22
arr[0][0][13] = 23
arr[0][0][14] = 25
arr[0][0][15] = 26
arr[0][0][16] = 27
arr[0][0][17] = 24
arr[0][0][18] = 23
arr[0][0][19] = 29
arr[0][0][20] = 211
arr[0][0][21] = 20
arr[0][0][22] = 217
arr[0][0][23] = 210
```

Returnarea unei matrici dintr-o funcție

- Matricile și șirurile care sunt declarate în main și trimise către funcții, sunt trimise ca referințe (pointeri). Dacă acestea sunt modificate în funcții, modificările persistă și după ce funcția se încheie.

- Matricile declarate în interiorul funcțiilor sunt variabile locale - acestea nu persistă după încheierea funcțiilor. Dacă dorim să declarăm o matrice în interiorul unei funcții și să o returnăm, trebuie să folosim o metodă diferită de declarare, folosind funcția **malloc**.

- Declararea unei matrici de dimensiuni mari într-o funcție (adică în stack) poate depăși dimensiunea stackului provocând eroarea “stack overflow”.

- Malloc alocă memoria în alt spațiu (heap) care nu se resetează la ieșirea din funcție. Heap-ul este de obicei semnificativ mai mare decât stackul.

- La părăsirea funcției, se poate returna un pointer către matricea declarata cu malloc si aceasta va fi disponibilă după încheierea funcției.

Returnarea unei matrici dintr-o funcție

insertion_copy.c

```c
#include <stdio.h>
#include <stdlib.h>

int main()

{
   int data[] = {12,23,5,7,2,9,11,13,9};
   int *sorted = NULL;
   int num = sizeof(data)/sizeof(data[0]);

   printf("Inainte de sort:\n");
   printstr(data,num);
   sorted = insertion(data, num);
   printf("Dupa sort :\n");
   printstr(data,num);
   printf("Sorted :\n");
   printstr(sorted,num);
   return 0;
}
```

https://onlinegdb.com/SygL1OwpQ

```c
int * insertion(int data[], int size)
{
   int *newArr;
   int i,temp,j;
   newArr=(int*)malloc(size*sizeof(int));
   for (i = 1 ; i <= size - 1; i++)
   {
       newArr[i] = data[i];
   }
   for (i = 1 ; i <= size - 1; i++)
   {
     j = i;
    while ( j > 0 && newArr[j] < newArr[j-1])
       {
           temp = newArr[j];
           newArr[j] = newArr[j-1];
           newArr[j-1] = temp;
           j--;
       }
    }
    return newArr;
}
```

```c
void printstr(int data[], int size)
{
    int i;
    for (i = 0; i <= size - 1; i++)
    {
          printf("%d\n", data[i]);
    }
}
```

WOULD YOU LIKE TO KNOW MORE?

Floating Point Numbers (Part1: Fp vs Fixed) - Computerphile

Cât de mult crește viteza calculelor prin utilizarea unui procesor de numere flotante?

https://www.youtube.com/watch?v=f4ekifyijIg

Fixed Point Arithmetic 1: Intro to Fixed Point

Detalii despre modalitatea de a stoca numerele fracționare cu ajutorul virgulei fixe

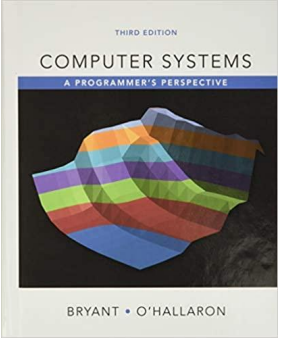
https://www.youtube.com/watch?v=S12qx1DwjVk

Where did Bytes Come From? - Computerphile

Despre originea conceptului de octet.

https://www.youtube.com/watch?v=ixJCo0cyAuA

WOULD YOU LIKE TO KNOW MORE?



Computer Systems: A Programmer's Perspective

Randal Bryant, David O'Hallaron

ISBN: 978-0134092669

GitHub - shihyu/CSAPP2e

21:Everything u need 2 know about pointers -Richard Buckland

Pointeri, referințe - o lecție la tablă

https://www.youtube.com/watch?v=RxvvrECNw

5. C to Assembly

Detalii despre cum este tradus codul C în limbajul de asamblare. Se vorbeste si despre LLVM - un compilator modern care poate compila mai multe limbaje de programare

https://www.youtube.com/watch?v=wt7a5BOztuM

The Next Decade of Software Development - Richard Campbell - NDC Oslo 2021

O incercare de a imagina viitorul imediat al dezvoltarii software pe baza celor intamplate pana acum, împreună cu ceva sfaturi cu privire la modul în care ar trebui abordată o carieră în programare.

https://www.youtube.com/watch?v=a_LM3FJozr0

```
return 0;
```